VERIQTA

DEVSECOPS HANDBOOK

FROM BEGINNER TO PRODUCTION SECURITY

Enterprise DevSecOps for Modern Cloud, Kubernetes, CI/CD, and Platform Engineering

SECURE BY DESIGN
Build security into every stage of the SDLC

SHIFT LEFT & RIGHT
Integrate security early and validate in production

KUBERNETES SECURITY
Secure clusters, workloads, networks, and identities

CLOUD SECURITY
AWS, Azure, GCP best practices

AUTOMATE & ENFORCE
Policy as Code, scanning, compliance, and monitoring

DETECT & RESPOND
Monitoring, threat detection, and incident response

60 PAGES OF PRACTICAL PRODUCTION KNOWLEDGE

REAL WORLD EXAMPLES
Production-tested practices and architectures

ENTERPRISE FOCUSED
Designed for scale, compliance, and reliability

HANDS-ON GUIDES
Step-by-step commands, configs, and workflows

BEST PRACTICES
Proven frameworks and security strategies

SECURE. AUTOMATE. MONITOR. RESPOND.
BUILD TRUST. DELIVER FASTER. PROTECT EVERYTHING.

FOR DEVELOPERS, DEVOPS ENGINEERS, PLATFORM ENGINEERS & SECURITY ENGINEERS

WELCOME TO DEVSECOPS

This handbook is your practical guide to building secure systems, shipping faster, and protecting what matters most - your users, data, and business.

DEVSECOPS IS NOT A TOOL. IT IS A MINDSET, A CULTURE, AND A CONTINUOUS PRACTICE.

WHAT IS DEVSECOPS?

DevSecOps integrates security into every stage of the software delivery lifecycle. It combines Development speed, Security best practices, and Operations reliability.

DEVELOP SECURELY. DELIVER RELIABLY. OPERATE SAFELY.

WHY DEVSECOPS MATTERS

- ✓ Threats are automated. We must be too.
- ✓ Security incidents are expensive and damaging.
- ✓ Shift-left security reduces risk and cost.
- ✓ Compliance is easier with automation.
- ✓ Customers trust secure, reliable systems.
- ✓ Security is everyone's responsibility.

HANDBOOK OBJECTIVES

Understand DevSecOps principles and culture

- ☑ Integrate security into CI/CD and operations
- ☑ Learn practical tools and real-world patterns
- ☑ Implement security across cloud and Kubernetes
- ☑ Prevent, detect, and respond to security issues
- ☑ Build secure, scalable, and compliant systems
- ☑ Think like a security-aware engineer

BUILD SECURELY. AUTOMATE EVERYTHING. MONITOR CONTINUOUSLY. IMPROVE ALWAYS.

LEARNING ROADMAP

1. **FOUNDATIONS** — Core concepts, culture, and security mindset — PAGES 2-9
2. **SECURE DEVELOPMENT** — Secure coding, Git security, secrets, and threat modeling — PAGES 10-16
3. **SECURE CI/CD** — Secure pipelines, scans, policies, and artifact integrity — PAGES 17-24
4. **CONTAINER & KUBERNETES SECURITY** — Containers, K8s security, policies, runtime protection — PAGES 25-34
5. **CLOUD & INFRASTRUCTURE SECURITY** — AWS, Azure, GCP security, IaC security, and compliance — PAGES 35-42
6. **MONITORING, DETECTION & RESPONSE** — Logging, detection engineering, incident response — PAGES 43-50
7. **GOVERNANCE & BEST PRACTICES** — Policies, compliance, metrics, risk management — PAGES 51-57
8. **PRODUCTION EXCELLENCE** — Real-world lessons, case studies, architecture, and checklists — PAGES 58-60

60 PAGES. ONE COMPLETE DEVSECOPS JOURNEY.

PRODUCTION MINDSET

SECURITY IS EVERYONE'S JOB
- No silos
- Collaborate early
- Own the outcome

AUTOMATE EVERYTHING
- Manual = error
- Automate tests
- Automate policies

MEASURE CONTINUOUSLY
- Monitor everything
- Track risk
- Improve constantly

ASSUME BREACH
- Verify always
- Least privilege
- Defense in depth

BUILD FOR RESILIENCE
- Fail securely
- Recover fast
- Learn and adapt

REMEMBER

Fast delivery without security is a vulnerability. Security without speed is friction. DevSecOps balances both.

BUILD SECURE. SHIP FAST. STAY COMPLIANT. PROTECT YOUR USERS. PROTECT YOUR BUSINESS.

DEVSECOPS FUNDAMENTALS

DevSecOps extends DevOps by integrating security into every phase of the software delivery lifecycle. Security is not a separate phase. It is everyone's responsibility, everywhere.

SECURITY IS NOT A GATE.
SECURITY IS A PRACTICE.
SECURITY IS A CULTURE.

1. DEVOPS VS DEVSECOPS

DEVOPS

PLAN › CODE › BUILD › TEST › DEPLOY › OPERATE › MONITOR

Security is tested late (if at all). Risk of vulnerabilities reaches production.

DEVSECOPS

PLAN › CODE › BUILD › TEST › DEPLOY › OPERATE › MONITOR

Security is integrated early and continuously. Vulnerabilities are prevented, not just detected.

LEFT = PREVENT PROBLEMS
RIGHT = DETECT & RESPOND TO PROBLEMS

2. SHIFT LEFT SECURITY

Move security activities earlier in the lifecycle to prevent vulnerabilities before they become expensive to fix.

WHAT THIS INCLUDES:

- ☑ Threat modeling during planning
- ☑ Secure coding practices
- ☑ SAST, SCA, secret scanning in CI
- ☑ Design reviews and security testing early
- ☑ Developer security training

Fixing a vulnerability in production can cost 30–100x more than fixing it earlier.

3. SHIFT RIGHT SECURITY

Extend security activities into production to detect, respond, and continuously improve.

WHAT THIS INCLUDES:

- ☑ Runtime application protection
- ☑ Continuous monitoring and observability
- ☑ Vulnerability management in production
- ☑ Threat detection and incident response
- ☑ Feedback loop to improve development

PRODUCTION REALITY

Attackers focus on what you ship, not what you intend.

4. SECURITY CULTURE

DevSecOps succeeds only when security is part of the culture, not a checklist.

REMEMBER

Tools help, but culture protects. People build secure systems.

5. SHARED RESPONSIBILITY

Security is a shared responsibility across the entire stack.

WHO	RESPONSIBILITIES	EXAMPLES
DEVELOPERS </>	• Secure code • Fix vulnerabilities • Secure dependencies	• Input validation • Secrets management • Secure libraries
OPERATIONS / PLATFORM	• Secure infrastructure • Access control • Monitoring & patching	• Hardened images • Network policies • Logging & alerts
SECURITY TEAM	• Provide guardrails • Threat modeling • Risk management	• Policies • Training • Security tools
PRODUCT OWNERS	• Define security requirements • Prioritize risk	• User data protection • Compliance needs • Business risk

KEY TAKEAWAYS

- ✓ DevSecOps integrates security into every step, not at the end.
- ✓ Shift left to prevent. Shift right to detect and respond.
- ✓ Security culture makes DevSecOps sustainable.
- ✓ Everyone owns security. The organization wins.

PRODUCTION NOTE

The goal is not zero risk.
The goal is to reduce risk continuously and respond quickly when something inevitable happens.

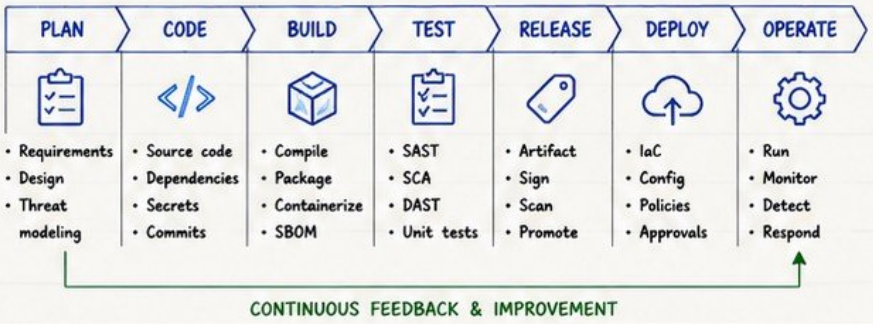
BUILD SECURE. SHIP FAST. STAY COMPLIANT. PROTECT YOUR USERS. PROTECT YOUR BUSINESS.

UNDERSTANDING THE SOFTWARE SUPPLY CHAIN

Every piece of software you run in production has a journey. Security must follow that journey from source to production.

YOU ARE ONLY AS STRONG AS YOUR SUPPLY CHAIN.

1. SOURCE TO PRODUCTION

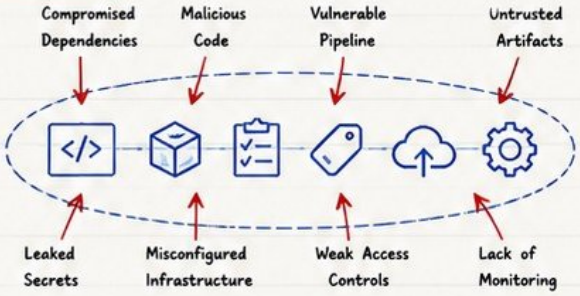


2. SUPPLY CHAIN RISKS

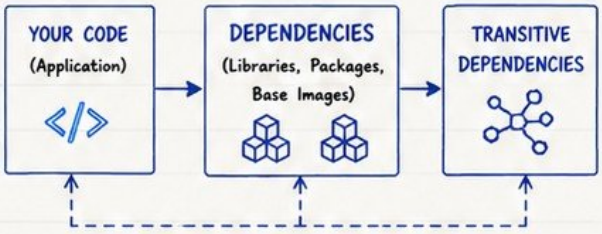
- ⚠ Compromised dependencies (malicious packages)
- ⚠ Weak or stolen credentials
- ⚠ Untrusted or tampered artifacts
- ⚠ Insecure build or CI/CD systems
- ⚠ Misconfigured infrastructure
- ⚠ Insider threats
- ⚠ Third-party and vendor risks
- ⚠ Lack of visibility and monitoring

3. ATTACK SURFACE

Attackers target the weakest link in your chain.



4. DEPENDENCY FLOW

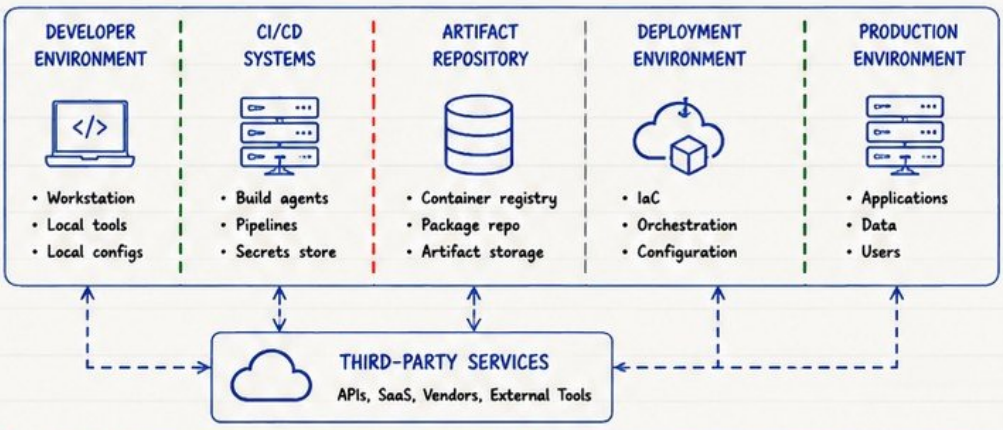


A vulnerable transitive dependency can compromise your application.

5. TRUST BOUNDARIES

Every boundary is a potential point of compromise.

- --- Internal Trust
- --- External Trust
- --- Third-Party Trust



PRODUCTION NOTE
Most breaches today come from the supply chain, not your code. Know every component you trust.

REAL WORLD LESSON
A single malicious dependency, a leaked token, or a misconfigured pipeline can lead to full production compromise.

SENIOR ENGINEER TIP
Map your supply chain. Own your dependencies. Automate trust. Continuously verify everything.

SECURE THE CHAIN. PROTECT THE BUILD. DEFEND THE PRODUCTION.

THREAT MODELING

Threat modeling helps you think like an attacker so you can build and operate systems that resist real-world threats.

YOU CAN'T PROTECT WHAT YOU DON'T UNDERSTAND.
MODEL THE THREAT.
REDUCE THE RISK.

1. ASSETS

Identify what you need to protect.

TYPE		EXAMPLES
</>	Application	Web app, API, Services
	Data	User data, PII, Financial data
	Infrastructure	Servers, Containers, VPCs
	Identity	Users, Service accounts, API keys
	Processes	CI/CD pipelines, Deployments
	Third-Party	SaaS, Libraries, Payment gateways

2. THREAT ACTORS

Who might attack and why?

EXTERNAL ATTACKERS
Hackers, script kiddies, nation-states.

MALICIOUS INSIDERS
Disgruntled employees, contractors.

COMPROMISED ACCOUNTS
Stolen credentials, phished users.

AUTOMATED THREATS
Bots, scanners, brute force tools.

THIRD PARTIES
Vendors, supply chain partners.

3. STRIDE OVERVIEW

Microsoft STRIDE is a practical framework to categorize threats.

S		**SPOOFING** Pretending to be something or someone else.
T		**TAMPERING** Altering data or systems maliciously.
R		**REPUDIATION** Denying an action with no audit trail.
I		**INFORMATION DISCLOSURE** Exposing sensitive information to unauthorized users.
D		**DENIAL OF SERVICE** Making a system unavailable to legitimate users.
E		**ELEVATION OF PRIVILEGE** Gain higher access than intended.

4. ATTACK PATHS

Map how an attacker could move from entry to impact.

Break the path. Reduce the blast radius.

5. RISK PRIORITIZATION

Not all risks are equal. Focus on what matters most.

LIKELIHOOD \ IMPACT	LOW	MEDIUM	HIGH
HIGH	MEDIUM	HIGH	CRITICAL
MEDIUM	LOW	MEDIUM	HIGH
LOW	LOW	LOW	MEDIUM

Score = Likelihood x Impact

HOW TO DO THREAT MODELING (SIMPLE PROCESS)

1. **Define Scope** — What system are you modeling?
2. **Identify Assets** — What needs protection?
3. **Identify Threat Actors** — Who might attack and why?
4. **Identify Threats** — Use STRIDE to find possible threats.
5. **Map Attack Paths** — How can these threats be exploited?
6. **Prioritize Risks** — Focus on high risk first.
7. **Define Mitigations** — Implement controls and monitor.

PRODUCTION NOTE
Threat models are not documents.
They are living models.
Review after every major change.

REAL WORLD LESSON
Most breaches succeed because simple attack paths were ignored during design and deployment.

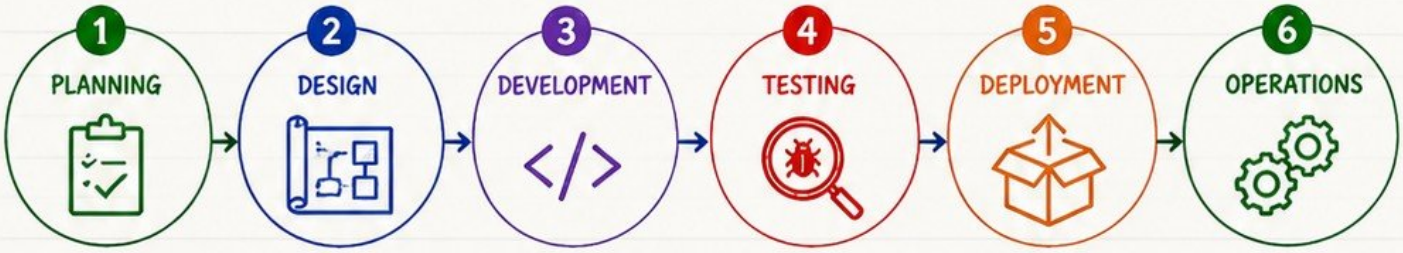
SENIOR ENGINEER TIP
The goal is not to eliminate all risk.
The goal is to understand, prioritize, and reduce what matters.

THINK LIKE AN ATTACKER. BUILD LIKE A DEFENDER. OPERATE LIKE AN OWNER.

SECURE SOFTWARE DEVELOPMENT LIFECYCLE (SSDLC)

SSDLC integrates security into every phase of the software development lifecycle. Security is not a phase, it is continuous.

BUILD SECURITY IN.
NOT BOLT IT ON.
THINK CONTINUOUS.
DELIVER CONFIDENTLY.



CONTINUOUS FEEDBACK & IMPROVEMENT ACROSS ALL PHASES

1 PLANNING

- Define security requirements
- Identify assets and data
- Perform initial risk assessment
- Define policies, standards, and compliance needs
- Establish security goals and metrics

KEY OUTCOME
Security is part of the plan.

2 DESIGN

- Perform threat modeling
- Apply secure design principles
- Design for least privilege
- Plan for data protection and privacy
- Define controls and validations

KEY OUTCOME
Security is built into architecture.

3 DEVELOPMENT

- Follow secure coding standards
- Use code analysis tools (SAST)
- Manage dependencies securely (SCA)
- Protect secrets and credentials
- Peer reviews and secure commits

KEY OUTCOME
Code is secure by default.

4 TESTING

- Run SAST, SCA, DAST, IAST
- Perform fuzzing and API testing
- Validate authentication and authorization
- Check configuration and infrastructure security
- Fix issues and re-test

KEY OUTCOME
Vulnerabilities are found early.

5 DEPLOYMENT

- Use secure pipelines
- Scan containers and artifacts
- Sign and verify artifacts
- Enforce policy as code
- Deploy with least privilege access

KEY OUTCOME
Only trusted, secure builds reach production.

6 OPERATIONS

- Monitor and detect threats
- Manage vulnerabilities
- Respond to incidents
- Audit and log continuously
- Improve and update controls

KEY OUTCOME
Systems stay secure in production.

SSDLC PRINCIPLES

- ✓ Security is everyone's responsibility
- ✓ Automate security wherever possible
- ✓ Shift security left and right
- ✓ Continuous validation and feedback
- ✓ Improve continuously

COMMON MISTAKES

- ✗ Adding security only at the end
- ✗ Ignoring threat modeling
- ✗ Hardcoding secrets
- ✗ Skipping dependency scanning
- ✗ Deploying without visibility

SENIOR ENGINEER TIP

The most secure system is not the one with the most tools, but the one with the right controls applied consistently at every stage of the lifecycle.

PLAN SECURE. DESIGN SECURE. CODE SECURE. TEST SECURE.
DEPLOY SECURE. OPERATE SECURE. IMPROVE CONTINUOUSLY.

IDENTITY AND ACCESS MANAGEMENT (IAM)

IAM ensures the right people and systems have the right access to the right resources for the right reasons at the right time. Strong IAM is the foundation of a secure system.

IF YOU CAN CONTROL ACCESS,
YOU CAN CONTROL RISK.
SECURE ACCESS. SECURE EVERYTHING.

1. LEAST PRIVILEGE

Grant users and systems only the minimum access needed to perform their job.

DO

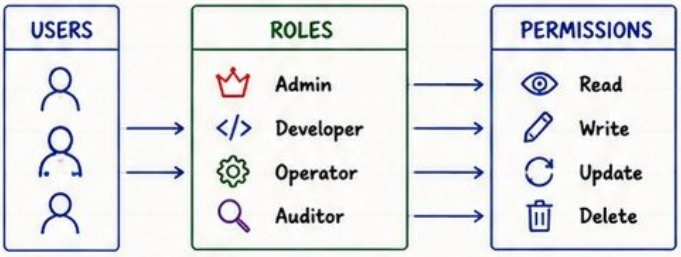
- Grant minimum access
- Review access regularly
- Remove unused access
- Automate access reviews
- Just-in-time access

DON'T

- Excessive permissions
- Default admin access
- Shared accounts
- Long-lived access
- Overly broad roles

2. RBAC (ROLE-BASED ACCESS CONTROL)

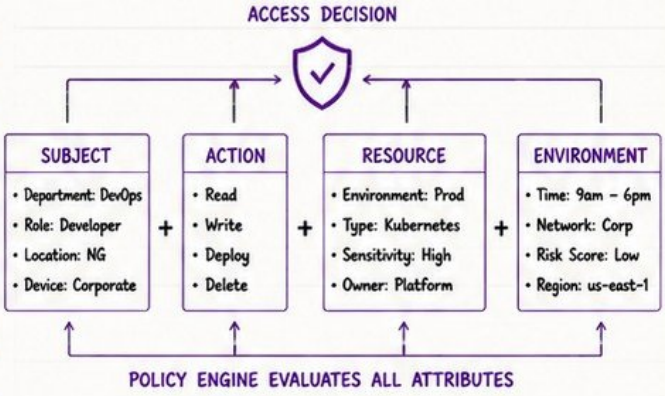
Access is granted based on roles assigned to a user.
Simple, scalable, and widely used.



Users → Roles → Permissions → Resources

3. ABAC (ATTRIBUTE-BASED ACCESS CONTROL)

Access is granted based on attributes of the user, resource, action, and environment.



4. MULTI-FACTOR AUTHENTICATION (MFA)

Add an extra layer of security beyond password.
MFA significantly reduces the risk of account takeover.

BEST PRACTICES

- Enforce MFA for all users, especially admins
- Use phishing-resistant MFA (FIDO2, WebAuthn)
- Do not allow MFA bypass
- Monitor and alert on failed MFA attempts

5. SERVICE IDENTITIES

Use dedicated identities for applications, services, and workloads instead of human accounts.

WHY SERVICE IDENTITIES MATTER

- Remove credentials from code
- Enable fine-grained access
- Improve auditability
- Rotate credentials automatically
- Limit blast radius

EXAMPLES

BEST PRACTICES

- Use least privilege for services
- Do not use long-lived keys
- Rotate credentials automatically
- Store secrets securely
- Monitor service identity usage

STRONG IAM. RIGHT ACCESS. RIGHT TIME. RIGHT REASON. EVERY TIME.

SECRETS MANAGEMENT

Secrets are keys to your kingdom.
Manage them securely across their entire lifecycle:
create, store, use, rotate, and revoke.

A SECRET EXPOSED IS AN INCIDENT WAITING TO HAPPEN.

1. WHY SECRETS LEAK

- Hardcoded in code or configs
- Committed to version control
- Logged in plaintext
- Exposed in environment variables
- Overly broad access
- Unencrypted storage
- Lack of rotation
- Human error
- Third-party breaches

REALITY CHECK
Secrets leak through people, processes, and systems. Assume exposure and design for containment.

2. ENVIRONMENT VARIABLES

Environment variables are easy to use but easy to expose.

GOOD USES
- Non-sensitive configuration
- Feature flags
- Log levels
- Public endpoints
- Safe defaults

RISKS
- Visible via /proc
- Shown in process listings
- May leak in logs or dumps
- Stored in plaintext
- Inherited by child processes

EXAMPLE

```
export APP_ENV=production          export DB_PASSWORD=MyPass123 ✗
export LOG_LEVEL=info              export API_KEY=abcd1234      ✗
```

Rule: Only store non-sensitive data in environment variables.

3. SECRET STORES

Use dedicated secret stores to encrypt, control access, and audit secret usage.

AWS	Azure	Google	HashiCorp	Kubernetes
AWS Secrets Manager	Azure Key Vault	Google Secret Manager	Vault	Secrets

KEY CAPABILITIES
- ✓ Encrypt secrets at rest and in transit
- ✓ Fine-grained access control
- ✓ Centralized management
- ✓ Auditing and logging
- ✓ Integration with CI/CD and workloads

4. SECRET ROTATION

Rotate secrets regularly to reduce the impact of exposure.

CREATE → UPDATE → VALIDATE → REVOKE → AUDIT

- **CREATE** — Generate new secret
- **UPDATE** — Update in applications and services
- **VALIDATE** — Verify new secret works correctly
- **REVOKE** — Disable old secret
- **AUDIT** — Log and monitor changes

ROTATION FREQUENCY GUIDE

HIGH RISK	MEDIUM RISK	LOW RISK
Every 7–30 days	Every 30–90 days	Every 90–180 days

5. PRODUCTION PRACTICES

LEAST PRIVILEGE	NO HARDCODING	SCAN & DETECT	SECURE DELIVERY	AUDIT EVERYTHING	EDUCATE TEAM
Grant only the minimum access required. No standing privileges.	Never hardcode secrets in code, scripts, or configurations.	Scan code and repos for leaked secrets continuously.	Inject secrets at runtime from secret stores, not in images.	Enable audit logs and monitor secret access regularly.	Train developers on secure secret handling and risks.

SENIOR ENGINEER TIP
Treat secrets like production code. Version, review, audit, and rotate. Security is a continuous discipline.

COMMON MISTAKES
- ✗ Hardcoding secrets
- ✗ Storing secrets in plain text
- ✗ Using default or shared accounts
- ✗ Skipping rotation
- ✗ Over-permissive access policies

PRODUCTION MINDSET
Assume breach
- ✓ Minimize blast radius
- ✓ Automate rotation
- ✓ Monitor and alert
- ✓ Review and improve continuously

PROTECT YOUR SECRETS. PROTECT YOUR SYSTEMS. PROTECT YOUR BUSINESS.

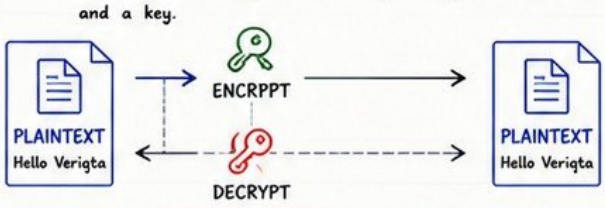
CRYPTOGRAPHY FUNDAMENTALS

Cryptography is the science of protecting information. It ensures confidentiality, integrity, authenticity, and non-repudiation in an untrusted world.

CRYPTOGRAPHY DOESN'T PROVIDE SECURITY. IT PROVIDES ASSURANCE THAT SECURITY CONTROLS WORK.

1. ENCRYPTION

Encryption converts readable data (plaintext) into unreadable data (ciphertext) using an algorithm and a key.

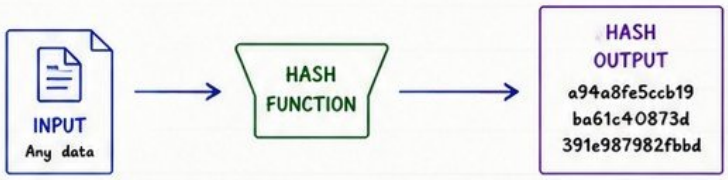


KEY POINTS

- ✓ Symmetric encryption: Same key for encrypt and decrypt (e.g., AES)
- ✓ Asymmetric encryption: Public key to encrypt, private key to decrypt (e.g., RSA)
- ✓ Protects confidentiality (data cannot be read by unauthorized users)

2. HASHING

Hashing converts data of any size into a fixed-length fingerprint. It is one-way and irreversible.

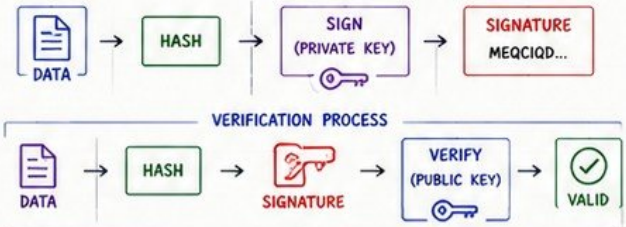


KEY POINTS

- ✓ Used for integrity verification, password storage, and digital signatures
- ✓ Common algorithms: SHA-256, SHA-512, BLAKE2, MD5 (not recommended)
- ✓ Same input = same hash, different input = different hash

3. DIGITAL SIGNATURES

Digital signatures verify the authenticity and integrity of data using asymmetric cryptography.



KEY POINTS

- ✓ Ensures integrity (data not changed) and authenticity (sender is real)
- ✓ Provides non-repudiation (sender cannot deny)
- ✓ Relies on public key infrastructure (PKI)

4. CERTIFICATES

Digital certificates bind an identity to a public key. They are issued and signed by a trusted Certificate Authority (CA).

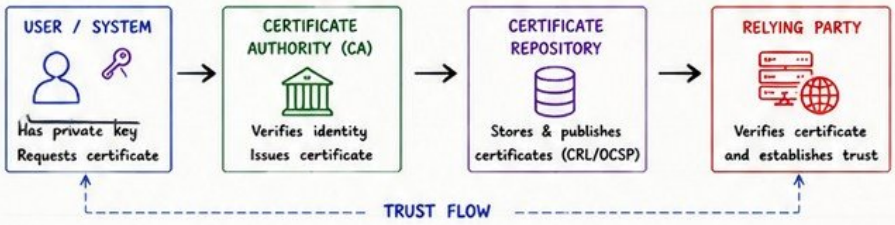
- → Subject: veriqta.com
- → Public Key: ----- BEGIN PUBLIC KEY -----
- → Issuer (CA): DigiCert Inc
- → Valid From: 2024-01-01
- → Valid To: 2025-01-01
- → Signature: -----

KEY POINTS

- ✓ Used in TLS/SSL, code signing, email encryption, and more
- ✓ Certificates prove identity and enable encrypted trust
- ✓ Always verify certificate validity and chain

5. PKI BASICS

Public Key Infrastructure (PKI) is the framework that manages keys, certificates, and trust.



PKI COMPONENTS

- Root CA – Self-signed, highest trust
- Intermediate CA – Issued by Root CA
- End-Entity Certificate – For users, servers, devices
- CRL (Certificate Revocation List) – List of revoked certificates
- OCSP – Online status check for certificate validity

SENIOR ENGINEER TIP

Use encryption to protect data.
Use hashing to verify data.
Use signatures to prove data.
Use certificates to establish trust.

COMMON MISTAKES

- ✗ Using weak or outdated algorithms (e.g., MD5, SHA-1)
- ✗ Storing keys in code or version control
- ✗ Not rotating keys or certificates
- ✗ Ignoring certificate expiry and revocation
- ✗ Assuming encryption alone = security

BEST PRACTICES

- ✓ Use strong algorithms (AES-256, RSA-2048+, SHA-256)
- ✓ Protect private keys like passwords
- ✓ Rotate keys and certificates regularly
- ✓ Validate certificate chains and expiry
- ✓ Automate PKI management where possible

ENCRYPT TO PROTECT. HASH TO VERIFY. SIGN TO PROVE. CERTIFY TO TRUST.

VERIQTA

GIT SECURITY

PAGE 9

Your code is only as secure as your Git workflows. Secure your repositories, commits, branches, and pipelines from the first line of code.

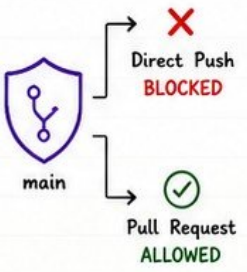
INSECURE CODE TODAY
BECOMES A BREACH
IN PRODUCTION TOMORROW.
SECURE GIT. SECURE EVERYTHING.

1. BRANCH PROTECTION

Prevent unsafe changes to important branches like main or production.

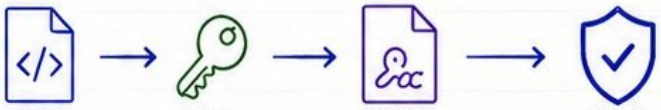
PROTECT MAIN BRANCH

- ✓ Require pull request reviews
- ✓ Require status checks to pass
- ✓ Require branches to be up to date
- ✓ Restrict who can push
- ✓ Dismiss stale reviews
- ✓ Prevent force pushes
- ✓ Require signed commits
- ✓ Include administrators



2. COMMIT SIGNING

Verify the author and ensure code integrity using cryptographic signatures.



BENEFITS

- ✓ Confirms the author's identity
- ✓ Ensures code was not tampered with
- ✓ Provides non-repudiation and accountability

EXAMPLE COMMANDS

```
git config --global user.signingkey <KEY_ID>
git config --global commit.gpgsign true
git log --show-signature
```

3. SECRET DETECTION

Detect and prevent secrets from being committed into repositories.

COMMON SECRETS

- API Keys
- Cloud credentials
- Passwords / Tokens
- Private keys
- Database URLs
- OAuth tokens

TOOLS YOU CAN USE

- GitHub Advanced Security (Secret Scanning)
- GitLab Secret Detection
- TruffleHog
- Gitleaks
- AWS Secrets Manager

BEST PRACTICE

Scan on push, in CI, and in pull requests. Block and rotate if leaked.

4. REPOSITORY PERMISSIONS

Control who can access, modify, and administer your repositories.

ROLE	ACCESS LEVEL	WHAT THEY CAN DO
ADMIN	Full Access	• Manage settings • Add / remove collaborators • Force push, delete branches
MAINTAIN	Write Access	• Push code • Manage branches • Create tags, releases
WRITE	Write Access	• Push code • Create branches • Open Pull Requests
READ	Read Only	• Clone repository • View code and issues • No write access

BEST PRACTICES

- ✓ Use least privilege – grant only what is needed.
- ✓ Review access regularly and remove unused accounts.
- ✓ Use teams / groups for scalable access management.

5. SECURE WORKFLOWS

Build secure Git workflows from code to production.

- ✓ Require reviews and status checks before merge
- ✓ Run SAST, dependency, and secret scans in CI
- ✓ Enforce signed commits and verified contributors
- ✓ Use protected branches and environments
- ✓ Deploy with approvals and audit logs

CI / CD PIPELINE

- Security Gates
- Automated Scans
- Policy Enforcement

COMMON MISTAKES

- ✗ Allowing direct pushes to main
- ✗ Disabling branch protection
- ✗ Committing secrets in code
- ✗ Using weak or unsigned commits
- ✗ Over-permissive repository access

SENIOR ENGINEER TIP

Git is the source of truth. Secure it like you secure production. Every commit is a change request to your infrastructure.

PRODUCTION MINDSET

- ✓ Treat code as a critical asset.
- ✓ Automate security checks early and continuously.
- ✓ Assume every commit is public.
- ✓ Monitor, audit, and improve continuously.

SECURE YOUR GIT TODAY. PROTECT YOUR CODE. PROTECT YOUR BUSINESS.

VERIQTA

SECURE CI/CD PIPELINES

CI/CD pipelines automate delivery.
If the pipeline is compromised,
attackers get a direct path to production.
Secure the pipeline. Protect the product.

A COMPROMISED PIPELINE CAN COMPROMISE EVERYTHING.
SECURE THE PIPELINE.
SECURE THE OUTCOME.

1. PIPELINE SECURITY

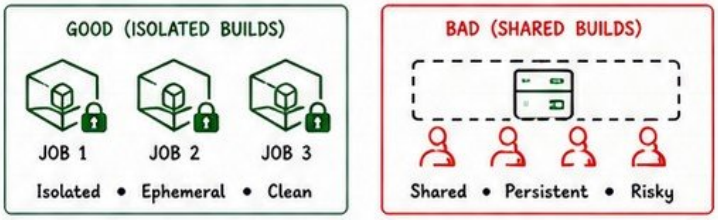
Secure every stage of the pipeline from code to production.

KEY POINTS

- ✓ Validate and verify everything (code, dependencies, configs, artifacts)
- ✓ Scan early and continuously
- ✓ Use approvals and gates for high-risk actions
- ✓ Enforce least privilege at every stage
- ✓ Log, monitor, and alert on suspicious activity

2. BUILD ISOLATION

Isolate build environments to prevent cross-contamination and privilege abuse.



KEY POINTS

- ✓ Use ephemeral runners or build environments
- ✓ Clean environment for every job
- ✓ No shared workspaces between untrusted jobs
- ✓ Limit network access during builds

3. SECURE RUNNERS

Runners execute your code.
Secure the machine, the network, and the identity.

RUNNER HARDENING CHECKLIST

- ✓ Use dedicated runners
- ✓ Keep OS and tools updated
- ✓ Disable unnecessary services
- ✓ Restrict outbound network access
- ✓ Use containers or VMs for isolation
- ✓ Auto-scale and auto-terminate
- ✓ Store nothing on persistent disk

4. PIPELINE PERMISSIONS

Use least privilege for pipeline identities and service accounts.

WHAT	BEST PRACTICE	AVOID
Pipeline Identity	Use dedicated service account per pipeline	Using personal credentials
Access Scope	Grant only required permissions	Over-permissive roles
Cloud Access	Use OIDC / short-lived credentials	Long-lived access keys
Secrets Access	Allow access only to specific secrets	Broad access to secret stores

KEY POINTS

- ✓ Never hardcode credentials in pipelines
- ✓ Use scoped roles and short-lived tokens
- ✓ Review and rotate permissions regularly

5. ARTIFACT INTEGRITY

Verify that what you build is what you deploy.

BUILD	SIGN	STORE	VERIFY	DEPLOY
Create artifact	Sign artifact (Digital signature)	Store in secure repository	Verify signature before deploy	Deploy only verified artifact

KEY POINTS

- ✓ Sign all artifacts (packages, images, Terraform plans, etc.)
- ✓ Use trusted repositories (not public buckets)
- ✓ Verify signatures before promotion or deployment
- ✓ Enable provenance (SLSA, SBOM) where possible

COMMON RISKS

- ✗ Untrusted pipeline code
- ✗ Using public or shared runners
- ✗ Exposed secrets in logs or env
- ✗ Over-permissive pipeline roles
- ✗ Skipping artifact verification
- ✗ No approval for production deploys
- ✗ No monitoring or alerting

SENIOR ENGINEER TIP

Treat your pipeline like production.
Because it is.
Attackers target the pipeline because it has access to everything.
Secure the pipeline, secure your supply chain.

PRODUCTION PRACTICES

- ✓ Implement pipeline as code with version control
- ✓ Use policy-as-code to enforce security rules
- ✓ Scan code, dependencies, images, and IaC
- ✓ Enforce approvals for high-risk changes
- ✓ Monitor pipeline activity and failures
- ✓ Continuously review and improve pipeline security

SECURE THE PIPELINE. PROTECT THE CODE. DELIVER WITH CONFIDENCE.

STATIC APPLICATION SECURITY TESTING (SAST)

SAST analyzes source code, bytecode, or binaries without executing the application to find security vulnerabilities early in the SDLC.

FIND VULNERABILITIES EARLY.
FIX CHEAPLY.
PREVENT BREACHES.
BUILD SECURE BY DESIGN.

1. HOW SAST WORKS

SAST tools scan your codebase using static analysis techniques and pattern matching against vulnerability rules.

ANALYSIS TECHNIQUES

- ✓ Pattern matching (signatures)
- ✓ Data flow analysis
- ✓ Control flow analysis
- ✓ Taint analysis
- ✓ Symbolic execution

2. COMMON FINDINGS

SAST tools detect a wide range of code-level issues.

CATEGORY	EXAMPLE FINDINGS	IMPACT
INJECTION	• SQL Injection • Command Injection • NoSQL Injection	Data breach System compromise
AUTHENTICATION	• Hardcoded credentials • Weak password handling • Missing MFA	Account takeover Unauthorized access
CRYPTOGRAPHY	• Weak algorithms (MD5, SHA-1) • Hardcoded keys • Insecure random	Data exposure Broken encryption
INPUT VALIDATION	• Cross-Site Scripting (XSS) • Path Traversal • Deserialization	Code execution Data manipulation
ERROR HANDLING	• Information leakage • Verbose errors • Insecure exception handling	Reconnaissance Attack facilitation
CODE QUALITY	• Dead code • Unsafe functions • Resource leaks	Unstable app Future vulnerabilities

3. FALSE POSITIVES

SAST tools may report issues that are not real vulnerabilities.

COMMON CAUSES

- ✗ Lack of context
- ✗ Complex business logic
- ✗ Unreachable code paths
- ✗ Framework abstractions
- ✗ Outdated rule sets

HOW TO MANAGE

- ✓ Triage and validate findings
- ✓ Use suppression only with reason
- ✓ Improve code quality
- ✓ Tune rules for your stack
- ✓ Educate developers

TIP The goal is not zero findings.
The goal is zero risk.

4. DEVELOPER WORKFLOW

Make SAST a natural part of everyday development.

BEST PRACTICES

- ✓ Scan early and often
- ✓ Fail builds on high/critical issues
- ✓ Track fixes and prevent regression
- ✓ Keep rules up to date
- ✓ Integrate with IDEs and PR gates
- ✓ Measure and improve over time

5. ENTERPRISE PRACTICES

Scale SAST across teams, codebases, and environments.

- **CENTRALIZED POLICY MANAGEMENT** — Enforce consistent rules and standards
- **INTEGRATE INTO CI/CD** — Automate scans on every pipeline run
- **QUALITY GATES** — Block merges on high-risk vulnerabilities
- **REPORTING & METRICS** — Track trends, MTTR, and risk posture
- **RISK BASED PRIORITIZATION** — Focus on exploitable and impactful issues
- **TRAIN & EMPOWER DEVELOPERS** — Provide training and secure coding guidance

SAST IN THE SDLC

PLAN — Security requirements
↓
CODE — SAST in IDE / Local
↓
BUILD — CI pipeline scan
↓
RELEASE — Verified & secure code
↓
OPERATE — Monitor & improve

WHAT SAST DOES NOT DO

- ✗ Does not find runtime issues
- ✗ Does not detect logic flaws fully
- ✗ May miss business logic vulnerabilities
- ✗ Cannot replace dynamic testing
- ✗ Cannot protect running systems

SENIOR ENGINEER TIP

Use SAST to shift security left. Catching issues in code is 10x cheaper than fixing them in production.
Security is everyone's responsibility.

POPULAR SAST TOOLS

SonarQube · Semgrep · Checkmarx · GitHub Code Scanning · Veracode

CATCH VULNERABILITIES IN CODE. FIX EARLY. SHIP SECURE. STAY AHEAD.

VERIQTA

SOFTWARE COMPOSITION ANALYSIS (SCA)

PAGE 12

SCA identifies and manages risks in open source components and third-party libraries used in your applications. You can't secure what you don't know you're using.

YOU CAN'T MANAGE WHAT YOU DON'T SEE.
SCAN YOUR DEPENDENCIES.
UNDERSTAND YOUR RISKS.
KEEP YOUR SOFTWARE SUPPLY CHAIN SAFE.

1. OPEN SOURCE RISKS

Open source accelerates development, but introduces risks if not managed properly.

VULNERABILITIES	MALICIOUS PACKAGES	ABANDONED PROJECTS	LICENSE RISKS
Known flaws in libraries	Typosquatting, dependency hijacking	No updates, unmaintained code	Incompatible or restrictive licenses

KEY POINTS

- ✓ Every dependency is part of your attack surface
- ✓ Transitive dependencies (dependencies of dependencies) matter
- ✓ Risks come from vulnerabilities, licenses, and supply chain threats

2. DEPENDENCY SCANNING

SCA tools build a software bill of materials (SBOM) and scan for known issues.

SOURCE CODE → DEPENDENCIES DETECTED → SCAN & ANALYZE → RISK REPORT & REMEDIATION

WHAT SCA SCANS

- ✓ Direct dependencies (declared in project files)
- ✓ Transitive dependencies (nested dependencies)
- ✓ Container base images and their packages
- ✓ IaC modules and plugins

POPULAR SCA TOOLS

- Snyk
- SonarQube
- OWASP Dependency-Check
- Anchore
- GitHub Dependabot
- JFrog Xray

3. LICENSE COMPLIANCE

Using open source means following the rules of the license.

LICENSE TYPES	RISKS	BEST PRACTICES
• Permissive (MIT, Apache 2.0)	• GPL in closed-source apps	• Inventory all licenses
• Copyleft (GPL, AGPL)	• Missing attributions	• Follow license obligations
• Weak Copyleft (LGPL)	• Incompatible licenses	• Get legal review for high risk
• Proprietary / Restricted	• Unknown license types	• Automate license checks

TIP A clean build can still be non-compliant. Always review licenses before release.

4. VULNERABILITY DATABASES

SCA tools rely on vulnerability databases to identify known issues.

DATABASE	FOCUS	USED BY
NVD (National Vulnerability Database)	General CVEs	All major tools
GitHub Advisory Database	Open source advisories	GitHub, Dependabot
OSS Index	Vulnerability data for OSS	Snyk, SonarQube
Red Hat OVAL / CVE	Linux / Enterprise packages	Enterprise scanners
Vendor Advisories	Specific product advisories	Tool vendors

KEY POINTS

- ✓ Databases update daily – keep tools updated
- ✓ Match CVEs to the exact library version and context
- ✓ Severity alone is not enough – consider exploitability

5. PACKAGE UPDATES

Keeping dependencies up to date reduces risk.

UPDATE STRATEGY

- ✓ Monitor vulnerabilities continuously
- ✓ Apply non-breaking updates regularly
- ✓ Test updates in isolated environments
- ✓ Automate update PRs
- ✓ Track and close stale dependencies

UPDATE WORKFLOW

DETECT VULNERABILITY → CREATE UPDATE → TEST CHANGES → MERGE & DEPLOY

COMMON MISTAKES

- ✗ Ignoring transitive dependencies
- ✗ Blindly updating without testing
- ✗ Not tracking licenses
- ✗ Delaying updates for too long

6. ENTERPRISE BEST PRACTICES

- ✓ Generate and maintain SBOMs for all applications
- ✓ Integrate SCA in CI/CD pipelines (fail builds on high risk)
- ✓ Set policies for allowed licenses and risk thresholds
- ✓ Centralize and review exceptions
- ✓ Continuously monitor and respond to new vulnerabilities
- ✓ Educate developers about secure dependency usage

REMEMBER

Every dependency is a promise.
Know what you use.
Trust what you ship.

KNOW YOUR DEPENDENCIES. MANAGE YOUR RISKS. SECURE YOUR SUPPLY CHAIN.

VERIQTA

VERIQTA

DYNAMIC APPLICATION SECURITY TESTING (DAST)

PAGE 13

DAST tests running applications from the outside.
It simulates real attacker behavior to find vulnerabilities that exist at runtime.
It does not require source code.

TEST LIKE AN ATTACKER.
FIND WEAKNESSES BEFORE ATTACKERS DO.
SECURE AT RUNTIME.

1. RUNTIME TESTING

DAST tools interact with your running application just like a real user or attacker would.

WHAT DAST TESTS FOR

- ✓ How the application behaves at runtime
- ✓ How inputs are handled by the application
- ✓ How the application communicates with backend services
- ✓ Authentication, authorization, and session management
- ✓ API endpoints, business logic, and error handling

2. AUTHENTICATION TESTING

DAST tools test authentication mechanisms for weaknesses.

- ✓ Brute force / credential stuffing
- ✓ Weak password policies
- ✓ Account lockout bypass
- ✓ Multi-factor authentication bypass
- ✓ Session fixation
- ✓ Remember me / token issues
- ✓ Password reset flaws
- ✓ JWT / token manipulation

EXAMPLE CHECKS

- Try default credentials
- Test weak passwords
- Attempt login rate bypass
- Test token expiry
- Modify tokens
- Test password reset flow
- Check session invalidation

TIP Strong authentication is the first line of defense. Attackers target it first.

3. COMMON WEB VULNERABILITIES

DAST tools look for the most common and dangerous issues.

VULNERABILITY	DESCRIPTION	IMPACT
SQL INJECTION	Attacker injects malicious SQL in inputs.	Data theft, data modification, bypass
CROSS-SITE SCRIPTING (XSS)	Attacker injects scripts into web pages viewed by others.	Session theft, defacement, malware
BROKEN AUTHENTICATION	Flaws in login and session management.	Account takeover, unauthorized access
BROKEN ACCESS CONTROL	Users can access resources they should not.	Privilege escalation, data exposure
INSECURE FILE UPLOAD	Uploading malicious files to the server.	Remote code execution, system compromise
SECURITY MISCONFIGURATION	Insecure settings in server, frameworks, or apps.	Information leakage, attack surface increase
SENSITIVE DATA EXPOSURE	Sensitive data exposed in responses or error messages.	Data breaches, compliance failure
SERVER-SIDE REQUEST FORGERY (SSRF)	Attacker forces server to make unexpected requests.	Internal access, data leakage
BUSINESS LOGIC FLAWS	Abuse of application logic to perform unauthorized actions.	Financial loss, fraud, data abuse

4. PRODUCTION CONSIDERATIONS

Running DAST in production requires careful planning.

- ✓ Avoid performance impact on users
- ✓ Exclude sensitive or high-transaction endpoints
- ✓ Use authenticated scans with test accounts
- ✓ Coordinate scan windows (low traffic)
- ✓ Rate limit and throttle scans
- ✓ Protect production data (mask or exclude)
- ✓ Ensure legal and compliance approvals
- ✓ Monitor logs and alerts during scans

WHAT TO AVOID

- ✗ Running aggressive scans during peak hours
- ✗ Scanning admin endpoints without permission
- ✗ Using destructive payloads in production
- ✗ Ignoring alerts triggered by the scan
- ✗ Leaving test accounts or tokens active

5. BEST PRACTICES

- ✓ Integrate DAST into your SDLC and CI/CD pipelines
- ✓ Combine DAST with SAST and SCA for full coverage
- ✓ Use authenticated scans for deeper visibility
- ✓ Keep scanners and vulnerability rules updated
- ✓ Verify and triage findings – not every finding is real
- ✓ Re-test after fixes to confirm vulnerabilities are resolved
- ✓ Track trends and fix the root cause
- ✓ Document exceptions and acceptances properly

TYPES OF DAST SCANS

- Black Box (No knowledge of app)
- Gray Box (Partial knowledge)
- White Box (Full knowledge)
- Authenticated (With login)
- API Scanning (REST / GraphQL)
- Mobile App Scanning
- Interactive / Manual Assisted

6. DAST TOOLS

OWASP ZAP · Burp Suite Professional · Acunetix · HCL AppScan Standard · Invicti (Netsparker)

Arachni · Nikto · Nuclei · Wapiti · ffuf

TEST AT RUNTIME. FIND WEAKNESSES. FIX EARLY. SHIP SECURE.

VERIQTA

VERIQTA

INTERACTIVE APPLICATION SECURITY TESTING (IAST)

PAGE 14

IAST combines the accuracy of runtime testing with the depth of static analysis.
It instruments your application and analyzes vulnerabilities as they are discovered.

SEE THE RISK.
SEE THE CONTEXT.
FIX THE VULNERABILITY.
IAST MAKES SECURITY ACTIONABLE.

1. HOW IAST WORKS

IAST instruments the application at runtime and correlates real-time analysis with source code.

TEST TRAFFIC (User / Scanner) → INSTRUMENTED APPLICATION → RUNTIME ANALYSIS (Vulnerability Detection) → DETAILED REPORT (Code + Context)

IAST COMBINES

- ✓ Real-time vulnerability detection (like DAST)
- ✓ Source code visibility (like SAST)
- ✓ Reachability context (only what runs)
- ✓ Reduced noise and false positives

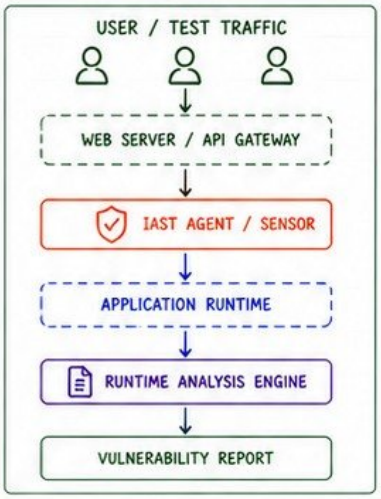
2. RUNTIME INSTRUMENTATION

IAST uses agents or sensors to watch the application while it runs.

INSTRUMENTATION METHODS

- Agent-based (on host or container)
- Agentless (in-memory injection)
- Bytecode instrumentation
- API hooking
- Library / classloader hooks

TIP IAST sees what attackers can reach and how data flows in real time.



3. ADVANTAGES

- ✓ Highly accurate with low false positives
- ✓ Finds vulnerabilities in running code paths
- ✓ Provides deep context (stack trace, user, data)
- ✓ Detects complex vulnerabilities (e.g., deserialization)
- ✓ Works with modern apps (APIs, microservices, containers)
- ✓ Continuous visibility in dev, test, and production
- ✓ Speeds up remediation with precise code locations

4. LIMITATIONS

- ✗ Requires the application to be running
- ✗ Needs agent installation or runtime access
- ✗ Can add performance overhead
- ✗ May not cover 100% of code (untested paths)
- ✗ Complex to deploy in some environments
- ✗ Requires skilled tuning for best results
- ✗ Not a replacement for SAST or DAST

5. ENTERPRISE ADOPTION

IAST delivers the most value when integrated into the SDLC.

1. PLAN — Define scope and goals
2. INTEGRATE — Add IAST to pipelines and environments
3. TEST — Run functional and security tests
4. ANALYZE — Review findings with context and severity
5. REMEDIATE — Fix issues and validate continuously

BEST PRACTICES FOR ENTERPRISES

- ✓ Use IAST early in development and in CI/CD pipelines
- ✓ Combine IAST with SAST, SCA, and DAST for full coverage
- ✓ Baseline results and track trends over time
- ✓ Set policies and thresholds for risk acceptance
- ✓ Integrate with ticketing and vulnerability management
- ✓ Monitor performance impact and tune agents
- ✓ Train developers to read and act on IAST findings

6. IAST VS OTHER TESTING TYPES

FEATURE	SAST	DAST	IAST
When it runs	Before runtime	During runtime	During runtime
Visibility	Source code	Black box	Inside the app
Accuracy	Medium	Medium	High
False positives	High	Medium	Low
Coverage	All code	Exposed endpoints	Executed code paths
Context	Limited	Limited	Full (code + runtime)
Setup	Easy	Easy	Moderate
Best use	Code quality	External attack view	Deep runtime security

TIP IAST gives you the context to fix the right issue without wasting time on noise.

INSTRUMENT. OBSERVE. UNDERSTAND. SECURE IN REAL TIME.

VERIQTA

API SECURITY

APIs power modern applications.
They also expose your business.
Secure every endpoint. Protect every interaction.

APIs ARE CODE.
APIs ARE DATA.
APIs ARE BUSINESS.
SECURE THEM ALL.

1. REST SECURITY

Follow security best practices for RESTful APIs.

AREA	BEST PRACTICES
TRANSPORT SECURITY	• Enforce HTTPS with TLS 1.2+ • Disable insecure protocols and ciphers
INPUT VALIDATION	• Validate all inputs (type, length, format) • Prevent injection (SQL, NoSQL, Command)
DATA EXPOSURE	• Return only necessary fields • Avoid verbose error messages • Mask sensitive data
HTTP HEADERS	• Set security headers (HSTS, CSP, etc.) • Use Content-Security-Policy • Prevent MIME sniffing
VERSIONING	• Version your APIs (/v1/, /v2/) • Deprecate old versions securely

2. GRAPHQL SECURITY

GraphQL is powerful. Secure it by design.

RISK	MITIGATION
INTROSPECTION	• Disable introspection in production • Restrict schema visibility
DEEP QUERIES	• Limit query depth and complexity • Use query cost analysis
EXCESSIVE DATA	• Enforce pagination and limits • Avoid returning large datasets
SENSITIVE FIELDS	• Use field-level authorization • Never expose sensitive fields
ERROR HANDLING	• Avoid detailed error messages • Log errors internally only

TIP Use persisted queries and allow only whitelisted operations to reduce attack surface.

3. AUTHENTICATION

Verify who is calling your API.

API KEYS	JWT TOKENS	OAUTH 2.0	OPENID CONNECT	MUTUAL TLS
Simple to use. Rotate regularly.	Stateless. Include expiry.	Standard for delegated access.	Authentication + identity layer.	Strong for service-to-service

AUTHENTICATION CHECKLIST

- ✓ Enforce strong token policies (expiry, rotation)
- ✓ Store secrets securely (never in code)
- ✓ Use HTTPS for all authentication flows
- ✓ Revoke compromised tokens immediately
- ✓ Use secure libraries for token handling

4. AUTHORIZATION

Control what users and systems can access.

RBAC	ABAC	SCOPES	FIELD-LEVEL AUTHZ	ROW-LEVEL SECURITY
Role-Based Access Control access based on roles.	Attribute-Based Access Control Use attributes (user, resource, environment).	Define scopes for tokens. Limit access to required resources.	Restrict access to specific fields or data.	Limit data based on user context.

BEST PRACTICES

- ✓ Default deny. Allow explicitly.
- ✓ Enforce authorization on every endpoint.
- ✓ Validate permissions on the server.
- ✓ Do not trust client-side checks.
- ✓ Log authorization failures.
- ✓ Review access regularly.

5. RATE LIMITING

Protect your APIs from abuse and overload.

WHY IT MATTERS

- ✓ Prevents brute force attacks
- ✓ Protects from denial of service
- ✓ Ensures fair usage
- ✓ Reduces cost and resource abuse

COMMON STRATEGIES

- **Fixed Window** — Simple and easy to implement.
- **Sliding Window** — More accurate, smooth limits.
- **Token Bucket** — Allows bursts, maintains average rate.
- **Per User / IP / API Key** — Apply limits based on identity.

EXAMPLE RATE LIMIT POLICY

IDENTITY	LIMIT	WINDOW	ACTION ON EXCEED
Anonymous	100 requests	Per hour	429 Too Many Requests
Authenticated User	1,000 requests	Per hour	429 Too Many Requests
Admin User	10,000 requests	Per hour	Log + Alert
Internal Service	Unlimited	Trusted	Log only

TIP Return standard 429 status with Retry-After header. Help clients behave well.

SECURE YOUR APIS. PROTECT DATA. PROTECT USERS. PROTECT BUSINESS.

CONTAINER SECURITY FUNDAMENTALS

Containers move fast.
Security must move faster.
Secure the build. Secure the image.
Secure the runtime. Secure everything.

BUILD SECURE.
RUN SECURE.
STAY SECURE.
CONTAIN RISK.
REDUCE ATTACK SURFACE.

1. CONTAINER ATTACK SURFACE

Containers have a unique attack surface across build, image, registry, runtime, and orchestration.

CODE	BUILD	REGISTRY	RUNTIME	ORCHESTRATION
• Malicious code • Secrets in code	• Insecure Dockerfile • Vulnerable deps	• Image tampering • Access control	• Escape to host • Privilege abuse	• Misconfigurations • Weak policies

KEY ATTACK VECTORS

- ✓ Vulnerable base images
- ✓ Excessive privileges
- ✓ Weak secrets management
- ✓ Insecure network exposure
- ✓ Misconfigured orchestrator
- ✓ Untrusted images / registries

2. BASE IMAGE SECURITY

Your base image is the foundation. Start secure.

CHECK	WHAT TO DO
Use Official Images	Prefer official images from Docker Hub or vendor registries.
Minimal Base Images	Use minimal/distroless images (Alpine, Distroless, or Scratch).
Keep Updated	Use images with regularly patched packages and base layers.
Scan for Vulnerabilities	Scan base images before use (Trivy, Grype, Clair, etc).
Verify Integrity	Verify image signatures and digests.

TIP Choose small, trusted, and updated base images. Every unnecessary package is a risk.

3. IMAGE HARDENING

Build images that are minimal, non-root, and immutable.

HARDENING CHECKLIST

- ✓ Use minimal base images
- ✓ Remove unnecessary packages
- ✓ Run as non-root user
- ✓ Use read-only filesystem
- ✓ Drop all unnecessary capabilities
- ✓ Set secure file permissions
- ✓ Avoid storing secrets in image
- ✓ Use .dockerignore
- ✓ Multi-stage builds

EXAMPLE: GOOD DOCKERFILE

```
FROM python:3.12-alpine AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM gcr.io/distroless/python3-debian12
WORKDIR /app
COPY --from=builder /usr/local /usr/local
COPY app.py .
USER 10001
ENTRYPOINT ["python3", "app.py"]

# Minimal final image
# Non-root user
# No shell, no package manager
```

4. RUNTIME RISKS

Risks exist when containers are running.

RISK	DESCRIPTION	IMPACT	EXAMPLE
Container Escape	Breaking out of the container to the host.	Full host compromise	Kernel exploit, misconfig
Excessive Privileges	Running with root or unnecessary capabilities.	Privilege escalation	--privileged, CAP_SYS_ADMIN
Network Exposure	Exposing unnecessary ports or services.	External attacks	Open admin endpoints
Secrets in Runtime	Secrets stored in env, files or image.	Credential theft	API keys, DB passwords
Vulnerable Runtime	Unpatched packages or insecure configurations.	Exploits, data loss	Outdated libs, weak configs

RUNTIME PRINCIPLE
Minimize privileges, isolate workloads, and monitor everything.

5. BEST PRACTICES

- ✓ Scan images in CI/CD pipelines
- ✓ Sign images (cosign, Notary v2)
- ✓ Use private registries
- ✓ Enforce image policies (OPA/Gatekeeper)
- ✓ Run as non-root user
- ✓ Use read-only root filesystem
- ✓ Drop all capabilities by default
- ✓ Use secrets management tools
- ✓ Apply least privilege everywhere
- ✓ Continuously update base images
- ✓ Monitor containers and runtime behavior

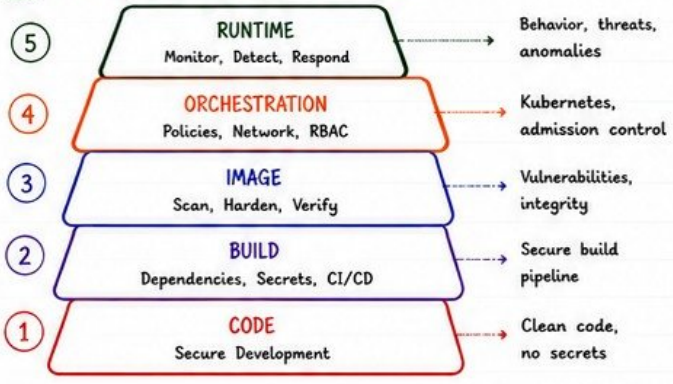
SECURE BY DEFAULT

Security is not a feature.
It is the foundation.
Bake it in, don't bolt it on.

USE TOOLS

- Trivy / Grype (Scanning)
- Cosign (Signing)
- Falco (Runtime Security)
- OPA / Gatekeeper (Policy)
- Docker Bench / Kube Bench

6. CONTAINER SECURITY LAYERS



SECURE THE BUILD. SECURE THE IMAGE. SECURE THE RUNTIME. SECURE EVERYTHING.

VERIQTA

DOCKER SECURITY

Secure containers. Secure images.
Secure the build. Secure the runtime.
A secure Docker environment starts with how you build and how you run.

SECURE DOCKER.
SECURE IMAGES.
SECURE CONTAINERS.
SECURE YOUR FUTURE.

1. SECURE DOCKERFILES

Follow best practices when building images.

BEST PRACTICE	EXAMPLE / HOW TO
Use official base images	FROM python:3.12-slim Avoid unknown or outdated images.
Pin image digests	FROM nginx@sha256:ab12cd... Avoid tag drift and supply chain risk.
Minimize layers and size	Combine RUN commands. Use multi-stage builds.
Run as non-root user	RUN useradd -m app USER app
Remove unnecessary packages	Clean package cache and temp files. Keep images small and clean.
Set secure permissions	Use least privilege on files and dirs. Avoid 777 permissions.
Scan during build	Integrate image scanning in CI/CD pipelines.

2. ROOTLESS CONTAINERS

Run containers without root privileges.

WHY ROOTLESS?

- ✓ Reduces impact of container breakout
- ✓ Prevents root inside container
- ✓ Better isolation and safety
- ✓ Meets compliance requirements

HOW TO ENABLE ROOTLESS DOCKER

1. Install Docker Engine (rootful once)
   `sudo apt install docker.io`
2. Create a non-root user
   `sudo useradd -m dockuser`
3. Enable rootless mode
   `dockerd-rootless-setuptool.sh install`
4. Login to user session
   `export DOCKER_HOST=unix:///run/user/1000/docker.sock`
5. Verify
   `docker info | grep "Rootless"`

ROOTLESS BENEFITS

- No root daemon
- User namespace isolation
- Safer file system access
- Works well with Kubernetes
- Better for multi-tenant hosts

TIP Use rootless for dev, CI runners, and multi-tenant environments.

3. IMAGE SCANNING

Scan images for vulnerabilities before pushing or running.

TOOLS

- Trivy
- Grype
- Docker Scout
- Clair
- Snyk
- Anchore
- AWS ECR Scan
- GitHub Dependency Scanning

EXAMPLE: TRIVY SCAN

```
# Scan local image
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock
aquasec/trivy image myapp:latest

# Scan and show only HIGH and CRITICAL
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock
aquasec/trivy image --severity HIGH,CRITICAL myapp:latest

# Fail build on critical vulns
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock
aquasec/trivy image --exit-code 1 --severity CRITICAL myapp:latest
```

TIP Scan early, scan often. Never ship unscanned images.

4. CAPABILITIES

Drop unnecessary Linux capabilities.

COMMON DANGEROUS CAPABILITIES

- CAP_SYS_ADMIN
- CAP_NET_ADMIN
- CAP_SYS_PTRACE
- CAP_DAC_OVERRIDE
- CAP_CHOWN
- CAP_MKNOD

Too many capabilities = more risk.

BEST PRACTICES

- ✓ Drop all, add only what you need
- ✓ Use --cap-drop ALL
- ✓ Add required caps explicitly
- ✓ Avoid privileged containers
- ✓ Review caps regularly
- ✓ Use seccomp + AppArmor

EXAMPLE

```
docker run -d \
  --cap-drop ALL \
  --cap-add NET_BIND_SERVICE \
  nginx:alpine
```

5. RUNTIME PROTECTION

Protect containers while they are running.

AREA	RISK	PROTECTION
Container Escape	Break out to host	Keep Docker updated Use user namespaces Drop capabilities
Filesystem Access	Write sensitive files	Use read-only root fs Mount only what is needed
Network Access	Exfiltration Lateral movement	Restrict network Use custom networks Block unnecessary ports
Process / Privilege	Privilege escalation	No privileged mode Run as non-root Drop capabilities
Monitoring	Undetected activity	Enable logging Use runtime security tools Monitor containers

RUNTIME SECURITY TOOLS

- Falco — Runtime threat detection
- Tracee — eBPF based security
- Sysdig Secure — Runtime protection platform
- Aqua Security — Prevent and detect threats
- Tetragon — eBPF security enforcement

EXAMPLE: SECURE RUN COMMAND

```
docker run -d \
  --name web \
  --user 10001:10001 \
  --read-only \
  --tmpfs /tmp:rw,noexec,nosuid,size=64m \
  --cap-drop ALL \
  --cap-add NET_BIND_SERVICE
  --security-opt no-new-privileges \
  --security-opt seccomp=default.json \
  --pids-limit 200
  --memory 512m \
  --memory-swap 512m \
  --network appnet
  nginx:alpine
```

TIP Least privilege + isolation = secure containers.

6. DOCKER SECURITY CHECKLIST

- ☑ Use official and minimal base images
- ☑ Pin image versions with digests
- ☑ Scan images in CI/CD
- ☑ Remove unnecessary packages
- ☑ Use multi-stage builds
- ☑ Run containers as non-root
- ☑ Drop all capabilities by default
- ☑ Add only required capabilities
- ☑ Avoid privileged mode
- ☑ Use read-only root filesystem
- ☑ Limit mounted volumes
- ☑ Restrict network access
- ☑ Enable logging and monitoring
- ☑ Keep Docker and host updated
- ☑ Use runtime security tools
- ☑ Enforce security policies
- ☑ Review and audit regularly
- ☑ Automate security checks
- ☑ Fail builds on critical issues
- ☑ Security is a continuous process

BUILD SECURE. RUN SECURE. STAY SECURE.

VERIQTA

KUBERNETES SECURITY FUNDAMENTALS

SECURE THE CLUSTER.
SECURE THE WORKLOADS.
SECURE THE DATA.
SECURE YOUR FUTURE.

Kubernetes is powerful, but insecure by default.
Security must be designed, not assumed.
Secure the cluster. Secure the pods. Secure everything.

1. CLUSTER SECURITY

Secure the control plane and cluster components.

AREA	BEST PRACTICES	EXAMPLES / NOTES
API SERVER	• Enable TLS everywhere • Disable anonymous access • Use Strong Authentication	--anonymous-auth=false --authorization-mode=RBAC --client-ca-file=ca.crt
AUTHENTICATION	• Use OIDC / SSO / MFA • Avoid static tokens • Rotate credentials	AWS IAM Authenticator Azure AD / GCP IAM Short-lived tokens
AUTHORIZATION	• Enforce RBAC • Apply least privilege	Use Roles, ClusterRoles Avoid cluster-admin
AUDITING	• Enable audit logging • Monitor admin actions	--audit-log-path=/var/log/k8s/ Use centralized logging
NETWORKING	• Encrypt etcd • Restrict control plane access	etcd --cert-file / --key-file Allow only trusted IPs
ETCD SECURITY	• Enable TLS • Regular backups • Limit access	--client-cert-auth Backup & test restore

TIP The control plane is the crown jewel.
Lock it down and monitor it closely.

2. POD SECURITY

Harden workloads and reduce risk at the pod level.

POD SECURITY CONTROLS	
CONTROL	BEST PRACTICES
RUN AS NON-ROOT	• Set runAsNonRoot: true • Use non-root user
READ-ONLY FILESYSTEM	• readOnlyRootFilesystem: true • Prevent unnecessary writes
DROP CAPABILITIES	• Drop ALL by default • Add only what is needed
RESOURCE LIMITS	• Set CPU & memory requests/limits • Prevent noisy neighbor issues
SECRETS MANAGEMENT	• Use Kubernetes Secrets / External Secrets • Never hardcode secrets
HEALTH PROBES	• Use liveness & readiness probes • Improve stability and recovery

EXAMPLE: SECURE POD SPEC (SNIPPET)

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]
resources:
  requests:
    cpu: "200m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

POD SECURITY CHECKLIST

- ✓ Run as non-root
- ✓ Read-only root filesystem
- ✓ Drop ALL capabilities
- ✓ Set resource limits
- ✓ Use secrets, not hardcoded
- ✓ Health probes configured
- ✓ No privilege escalation
- ✓ Image is minimal and trusted

3. NODE SECURITY

Secure the hosts running your containers.

HARDEN NODES

- Keep OS updated
- Disable SSH password login
- Use key-based authentication
- Enable firewall (allow minimal ports)
- Disable swap (required for kubelet)
- Use AppArmor / SELinux
- Enable audit logging
- Restrict Docker/Kubelet access

CRITICAL COMPONENTS

- Kubelet: secure access, rotate certs
- Container Runtime: keep updated
- CNI Plugin: use trusted, updated CNI
- Host Network: restrict node access
- Disk Encryption: encrypt sensitive data

⚠ Avoid privileged containers.
They can break out of the node.

4. NAMESPACE ISOLATION

Use namespaces and policies to isolate workloads.

ISOLATION STRATEGIES

- Use namespaces per team/app/env
- Enforce ResourceQuota
- Enforce LimitRange
- Use NetworkPolicies
- Prevent cross-namespace access
- Use Pod Security Admission

EXAMPLE: DENY ALL TRAFFIC BY DEFAULT

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
mettdata:
  name: default-deny
  namespace: app-namespace
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
```

TIP Default deny + explicit allow = secure by design.

5. PRODUCTION RECOMMENDATIONS

Operational security practices for production clusters.

SECURE BY DEFAULT
- Apply security settings to all clusters
- Use secure baselines
- Enforce guardrails

MONITOR & DETECT
- Enable logging
- Monitor API activity
- Use runtime threat detection

PATCH REGULARLY
- Update Kubernetes
- Update OS & runtimes
- Update add-ons
- Automate patching

BACKUP & DR
- Backup etcd regularly
- Store offsite
- Test restore
- Plan for disaster

LEAST PRIVILEGE
- Apply least privilege everywhere
- Review permissions
- Avoid cluster-admin

AUDIT & REVIEW
- Regular security audits
- Review RBAC
- Review NetworkPolicies
- Review images & tools

SECURITY TRAINING
- Train teams
- Promote secure Kubernetes practices
- Build security culture

SECURE CLUSTERS. SECURE WORKLOADS. SECURE DATA. SECURE BUSINESS.

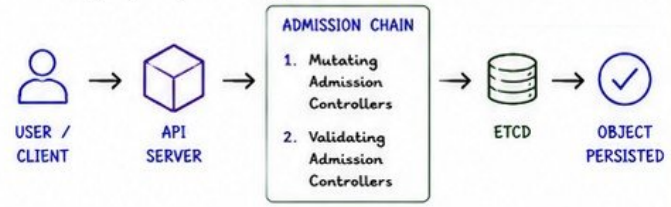
KUBERNETES ADMISSION CONTROLLERS

ADMIT SECURELY.
ENFORCE POLICIES.
PREVENT RISK.
PROTECT THE CLUSTER.

Admission controllers are the gatekeepers of the cluster.
They intercept API requests before persistence.
They enforce policies, mutate objects, and validate configurations.
Secure by design. Validate every request.

1. ADMISSION WORKFLOW

Every request passes through the admission chain.



1. Request received by API Server
2. Mutating controllers can modify the object
3. Validating controllers can accept or reject the request
4. If allowed, the object is persisted to etcd
5. The response is returned to the user

TIP If any validating controller rejects the request, the object is NOT persisted.

2. POLICY ENFORCEMENT

Enforce security, compliance, and operational standards.

POLICY TYPE	DESCRIPTION	EXAMPLES
SECURITY	Block risky configurations and insecure settings	Block privileged containers Require non-root users
COMPLIANCE	Enforce regulatory and organizational rules	Enforce labels Enforce resource limits
ACCESS CONTROL	Restrict who can create what	Restrict hostPath Enforce namespaces
OBSERVABILITY	Inject metadata and monitoring	Add labels / annotations Inject sidecars
COST CONTROL	Prevent resource waste and abuse	Limit CPU / Memory Restrict node selectors

TIP Admission controllers provide guardrails.
They do not replace RBAC or network policies.

3. MUTATION

Mutating controllers modify the object before it is stored.

WHAT MUTATION CAN DO

- Add or modify labels / annotations
- Set default values
- Inject sidecars
- Append security settings
- Normalize configurations

EXAMPLES

- Default Namespace
- LimitRanger
- PodSecurity (baseline)
- MutatingAdmissionWebhook
- Istio Sidecar Injector

EXAMPLE: MUTATING WEBHOOK (ADD LABEL)

```yaml
apiVersion: admissionregistration.k8s.io/v1
kind: MutatingWebhookConfiguration
metadata:
  name: add-label-webhook
webhooks:
- name: add-company-label.example.com
  rules:
  - apiGroups: [""]
    apiVersions: ["v1"]
    operations: ["CREATE"]
    resources: ["pods"]
  clientConfig:
    service:
      name: label-webhook-svc
      namespace: kube-system
      path: /mutate
```

BEFORE

```yaml
metadata:
  name: myapp
  labels: {}
```

↓

AFTER (MUTATED)

```yaml
metadata:
  name: myapp
  labels:
    company: veriqta
  managed-by: admission-webhook
```

TIP Mutation happens first. Validation happens after mutation.

4. VALIDATION

Validating controllers accept or reject the request.

WHAT VALIDATION CAN DO

- Enforce security policies
- Validate resource requests
- Enforce naming conventions
- Prevent use of risky fields
- Ensure compliance rules

EXAMPLES

- PodSecurity (enforce)
- ValidatingAdmissionWebhook
- ResourceQuota
- Gatekeeper / OPA
- Kyverno

EXAMPLE: VALIDATING POLICY (PREVENT PRIVILEGED)

```yaml
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sPSPPrivilegedContainer
metadata:
  name: disallow-privileged
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
  parameters:
    allowedHostPaths: []
    exemptImages: []
```

VIOLATION EXAMPLE

```yaml
containers:
  - name: app
    image: nginx
    securityContext:
      privileged: true
```

REJECTED
Request denied by validating controller

5. ENTERPRISE USE CASES

Real-world patterns for production environments.

SECURE BASELINE

- Enforce non-root users
- Drop all capabilities
- Block privileged containers
- Prevent hostPath usage

COMPLIANCE ENFORCEMENT

- Enforce labels
- Require resource limits
- Enforce image registries
- Enforce namespaces

PLATFORM STANDARDIDATION

- Inject logging sidecars
- Inject monitoring agents
- Add default labels
- Set default security context

COST & RESOURCE CONTROL

- Enforce quotas
- Limit CPU / Memory
- Restrict node selectors
- Prevent noisy neighbors

MULTI-TENANCY ISOLATION

- Enforce namespace policies
- Restrict cross-namespace access
- Enforce network policies
- Isolate workloads

ADMIT WITH INTENT. ENFORCE CONSISTENTLY. PROTECT THE CLUSTER.

VERIQTA

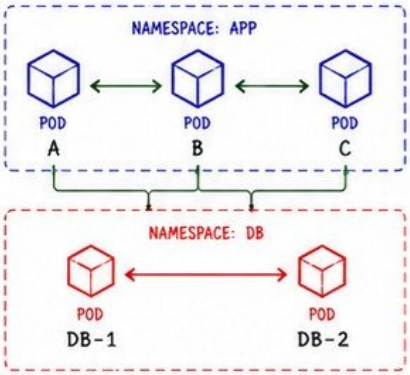
NETWORK POLICIES

Control traffic between pods and namespaces.
Implement micro-segmentation.
Default deny is your friend.
Design with intent. Test. Monitor. Improve.

CONTROL TRAFFIC.
LIMIT BLAST RADIUS.
ENFORCE SEGMENTATION.
SECURE COMMUNICATION.

1. EAST-WEST TRAFFIC

Traffic between workloads inside the cluster.



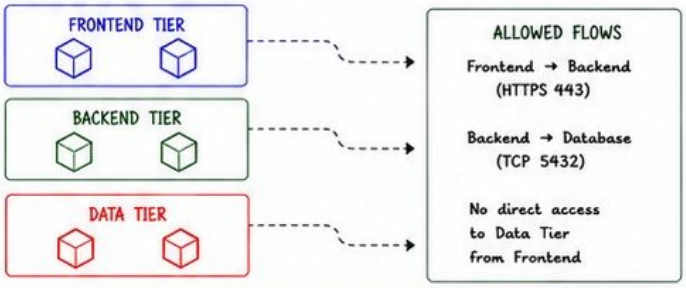
EXAMPLES

- Frontend → Backend
- Backend → Database
- Service → Service
- Job → API

TIP
Without Network Policies, all pods can talk to all pods by default.

2. MICRO-SEGMENTATION

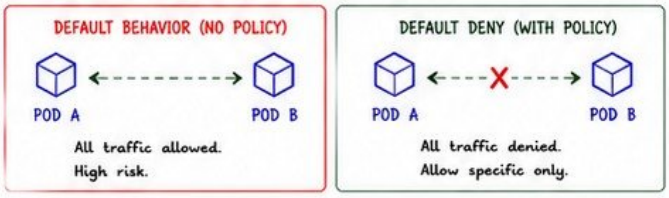
Apply policies per namespace, per tier, per application, or even per pod.



TIP Micro-segmentation limits movement and reduces impact of compromise.

3. DEFAULT DENY

Start from deny-all. Allow only what is required.



EXAMPLE: DEFAULT DENY POLICY

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: app
spec:
  podSelector: {}          # Select all pods in namespace
  policyTypes:
    - Ingress
    - Egress
  ingress: []              # Deny all incoming
  egress: []               # Deny all outgoing
```

TIP
Apply this first, then open what is needed.

4. POLICY DESIGN

Design policies with least privilege and clear intent.

DEFINE FLOWS	• List all required communications • Identify source, destination, ports, protocols
START WITH DENY	• Apply default deny policy • Verify all traffic is blocked
ALLOW REQUIRED	• Add rules for required traffic only • Use specific selectors and ports
SCOPE TIGHTLY	• Use labels, namespaces, and podSelectors • Avoid broad CIDR ranges
TEST & VALIDATE	• Test connectivity • Validate no unintended access
DOCUMENT	• Keep diagrams and policy inventory • Review and audit regularly

TIP Good policies are simple, specific, and documented.
Complex policies become security risks.

5. TROUBLESHOOTING

Use this checklist when traffic is blocked.

STEP	CHECK	HOW TO CHECK
1	Policy Exists	kubectl get networkpolicies -n <ns>
2	Pod Labels	kubectl get pod <pod> -n <ns> --show-labels
3	Policy Selectors	Verify podSelector and namespaceSelector
4	Ingress / Egress Rules	Check ports, protocols, and endpoints
5	DNS Resolution	kubectl exec <pod> -- nslookup <service>
6	Network Plugin	Ensure CNI supports NetworkPolicy
7	Logs & Events	kubectl describe pod <pod> -n <ns>

TIP
Most issues are labels, selectors, or missing egress rules.
Test from source pod to destination pod.

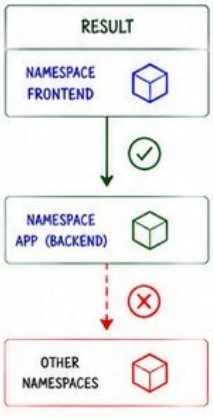
COMMON COMMANDS

```
# List policies
kubectl get networkpolicies -A

# Describe policy
kubectl describe networkpolicy <name> -n <ns>

# Test connectivity
kubectl exec -it <pod> -n <ns> -- \
  curl http://<service>:<port>

# Watch events
kubectl get events -n <ns>
--sort-by=.lastTimestamp

# CNI logs (example: Calico)
kubectl -n kube-system logs
-l k8s-app=calico-node
```

6. EXAMPLE: ALLOW FRONTEND TO BACKEND ONLY

Allow traffic from frontend pods to backend pods on port 8080.

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: app
spec:
  podSelector:
    matchLabels:
      app: backend          # Select backend pods
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: frontend    # Only from frontend namespace
    ports:
      - protocol: TCP
        port: 8080
```



DESIGN WITH INTENT. DENY BY DEFAULT. ALLOW WHAT IS NEEDED. VERIFY EVERYTHING.

VERIQTA

VERIQTA

RBAC IN KUBERNETES

PAGE 21

RBAC controls who can do what in the cluster.
It is Kubernetes' native authorization system.
Grant the minimum access required.
Protect the cluster. Reduce risk. Stay compliant.

RIGHT PEOPLE.
RIGHT ACCESS.
RIGHT RESOURCES.
RIGHT NOW.

1. ROLES

Grant permissions within a namespace.

KEY CONCEPT	DETAILS
Scope	Namespace scoped
Used For	Giving permissions inside a single namespace
Contains	API Groups, Resources, Verbs
Example Use	Allow a dev to manage deployments in "dev" namespace

EXAMPLE: ROLE (YAML)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: dev-role
  namespace: dev
rules:
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list", "watch", "create",
          "update", "patch", "delete"]
```

TIP Roles only apply inside the namespace where they are created.

2. CLUSTERROLES

Grant permissions cluster-wide.

KEY CONCEPT	DETAILS
Scope	Cluster wide
Used For	Managing cluster-level resources or across all namespaces
Contains	API Groups, Resources, Verbs
Example Use	View nodes, manage storage classes, or cluster roles

EXAMPLE: CLUSTERROLE (YAML)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: view-cluster
rules:
- apiGroups: [""]
  resources: ["pods", "services",
              "configmaps", "nodes"]
  verbs: ["get", "list", "watch"]
```

TIP ClusterRoles are not bound to a namespace. They apply across the entire cluster.

3. BINDINGS

Bindings connect users, groups, or service accounts to roles.

ROLEBINDING (Namespace Scoped)

Binds a Role to a subject within a namespace.

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dev-rolebinding
  namespace: dev
subjects:
- kind: User
  name: dev-user
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: dev-role
  apiGroup: rbac.authorization.k8s.io
```

CLUSTERROLEBINDING (Cluster Scoped)

Binds a ClusterRole to a subject cluster-wide.

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: view-cluster-binding
subjects:
- kind: Group
  name: devs
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: view-cluster
  apiGroup: rbac.authorization.k8s.io
```

TIP Bindings do not contain permissions. They only reference Roles or ClusterRoles.

4. SERVICE ACCOUNTS

Identities for processes running in Pods.

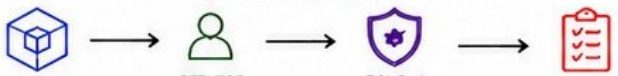
KEY POINTS

- ✓ Used by Pods to access the API
- ✓ Bound to Roles/ClusterRoles
- ✓ Has a token mounted by default
- ✓ Each namespace gets "default" SA
- ✓ Use dedicated SAs for workloads

EXAMPLE: SERVICE ACCOUNT (YAML)

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: prod
```

SERVICE ACCOUNT FLOW



Never use the default service account in production. Create dedicated SAs with only required permissions.

5. LEAST PRIVILEGE

Grant the minimum permissions required to perform the job.

LEAST PRIVILEGE PRINCIPLES

- ✓ Grant only what is needed
- ✓ Use Namespaced Roles over ClusterRoles when possible
- ✓ Avoid wildcards (*) in resources and verbs
- ✓ Don't grant "*" or "cluster-admin"
- ✓ Review permissions regularly
- ✓ Remove unused access
- ✓ Audit and monitor RBAC changes

AVOID THESE

- ✗ cluster-admin
- ✗ * wildcard in verbs
- ✗ * wildcard in resources
- ✗ Default service account
- ✗ Broad ClusterRoleBindings

EXAMPLE: GOOD VS BAD

TOO MUCH ACCESS (BAD)

```yaml
kind: ClusterRole
metadata:
  name: admin
rules:
- apiGroups: ["*"]
  resources: ["*"]
  verbs: ["*"]
```

LEAST PRIVILEGE (GOOD)

```yaml
kind: Role
metadata:
  name: read-pods
  namespace: app
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

6. RBAC BEST PRACTICES

- ✓ Use Roles for namespace-level access
- ✓ Use ClusterRoles only when necessary
- ✓ Bind to users, groups, or service accounts
- ✓ Follow least privilege principle
- ✓ Use dedicated service accounts
- ✓ Audit RBAC regularly
- ✓ Monitor access and API calls
- ✓ Document and review permissions

QUICK COMMANDS

```
kubectl get roles -A                          # List roles in all namespaces
kubectl get clusterroles                      # List clusterroles
kubectl get rolebindings -A                   # List rolebindings
kubectl get clusterrolebindings               # List clusterrolebindings
kubectl describe role <name> -n <ns>          # Describe a role
kubectl auth can-i <verb> <resource>          # Check your permissions
kubectl auth can-i --list                     # List all your permissions
```

GRANT LEAST PRIVILEGE. REVIEW REGULARLY. SECURE ALWAYS.

VERIQTA

VERIQTA

POD SECURITY

Pods are the smallest deployable units in Kubernetes.
Secure the Pod. Protect the workload. Reduce the blast radius.
Use security contexts, drop privileges, and enforce policies.
Secure by default. Harden everywhere.

LOCK DOWN PODS.
DROP PRIVILEGES.
REDUCE THE RISKS.
SECURE BY DEFAULT.

1. SECURITY CONTEXTS

Define privilege and access for containers and Pods.

KEY FIELDS	PURPOSE
runAsUser	User ID to run the process
runAsGroup	Group ID to run the process
fsGroup	Group ID for mounted volumes
allowPrivilegeEscalation	Prevent gaining more privileges
readOnlyRootFilesystem	Make root filesystem read-only
capabilities	Add or drop Linux capabilities
privileged	Run container in privileged mode

EXAMPLE: SECURITY CONTEXT (POD LEVEL)

```
apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
spec:
  securityContext:
    runAsUser: 1000
    runAsGroup: 1000
    fsGroup: 2000
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
containers:
- name: app
  image: nginx
  securityContext:
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: true
    capabilities:
      drop: ["ALL"]
      privileged: false
```

TIP

Use runAsNonRoot: true and drop ALL capabilities by default.

Only add what is required.

2. PRIVILEGED CONTAINERS

Privileged containers have full access to the host.

WHAT PRIVILEGED MODE GIVES

- ✗ All Linux capabilities
- ✗ Access to all devices
- ✗ Bypass AppArmor/SELinux
- ✗ Mount the host filesystem
- ✗ Access to kernel interfaces
- ✗ High risk of container escape

BEST PRACTICES

- ✓ Avoid privileged containers
- ✓ Use DaemonSets only when required
- ✓ Use specific capabilities instead
- ✓ Review workloads regularly
- ✓ Enforce Pod Security Standards

Privileged containers can break out and take over the node.
Use only when absolutely necessary.

EXAMPLE: DO NOT USE PRIVILEGED (UNLESS REQUIRED)

```
securityContext:
  privileged: true    # ✗ Avoid this
  allowPrivilegeEscalation: true    # ✗ Risky
  capabilities:
    add: ["ALL"]    # ✗ Too much access
```

3. LINUX CAPABILITIES

Linux capabilities provide fine-grained privilege control.

COMMON CAPABILITIES

CAP_CHOWN	Change file ownership
CAP_NET_BIND_SERVICE	Bind to low ports (<1024)
CAP_NET_ADMIN	Network administration
CAP_SYS_TIME	Set system time
CAP_DAC_OVERRIDE	Bypass file permissions
CAP_SYS_ADMIN	System administration (powerful)
CAP_SYS_PTRACE	Trace processes

BEST PRACTICES

- ✓ Drop ALL by default
- ✓ Add only what is needed
- ✓ Avoid CAP_SYS_ADMIN
- ✓ Use audit to find requirements
- ✓ Review capabilities regularly

EXAMPLE: DROP ALL, ADD SPECIFIC

```
securityContext:
  capabilities:
    drop: ["ALL"]
    add: ["NET_BIND_SERVICE"]  # Only what is needed
```

TIP

Most apps do NOT need ANY capabilities. Drop ALL.

4. SECCOMP

Seccomp filters system calls to reduce attack surface.

SECCOMP MODES

- Unconfined - No filtering (default)
- RuntimeDefault - Kubernetes default
- Localhost - Custom profile

BENEFITS

- ✓ Restricts dangerous syscalls
- ✓ Prevents kernel exploits
- ✓ Blocks unexpected behavior
- ✓ Reduces container escape risks

EXAMPLE: SECCOMP PROFILE

```
securityContext:
  seccompProfile:
    type: RuntimeDefault    # Recommended
```

CUSTOM PROFILE EXAMPLE

```
seccompProfile:
  type: Localhost
  localhostProfile:
    localhost/profiles/
    myprofile.json
```

TIP Use RuntimeDefault if you don't have a custom profile.
Create custom profiles for high security environments.

5. APPARMOR

AppArmor restricts what a process can do on the system.

KEY POINTS

- Linux kernel security module
- Controls file access, capabilities, network, and more
- Uses profiles to allow/deny actions
- Works per container or per process
- Should be used with seccomp

APPARMOR MODES

- Unconfined - No restrictions
- RuntimeDefault - Kubernetes default
- Localhost - Custom profile

EXAMPLE: APPARMOR PROFILE

```
metadata:
  annotations:
    container.apparmor.security.beta.kubernetes.io/app: runtime/default
spec:
  containers:
  - name: app
    image: nginx
    securityContext:{ }
```

TIP

Use RuntimeDefault as a minimum. Create custom profiles for sensitive apps.

6. POD SECURITY BEST PRACTICES

- ✓ Run as non-root user
- ✓ Drop ALL capabilities and add only what is needed
- ✓ Disable privilege escalation
- ✓ Use read-only root filesystem
- ✓ Use seccomp RuntimeDefault or custom profile
- ✓ Use AppArmor RuntimeDefault or custom profile
- ✓ Avoid privileged containers
- ✓ Use Pod Security Standards
- ✓ Scan images and keep them updated
- ✓ Audit and monitor regularly

POD SECURITY STANDARDS

PRIVILEGED
Unrestricted (not recommended)

BASELINE
Minimal restriction
Good for general use

RESTRICTED
Strong restrictions
For high security

TIP Secure Pods. Secure Nodes. Secure Everything.
Defense in depth is the only real security.

TIGHTEN EVERY POD. DROP PRIVILEGES. LIMIT ACCESS. STAY SECURE.

VERIQTA

VERIQTA

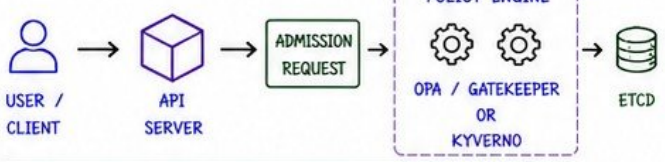
ADMISSION POLICY ENGINES

Admission policy engines enforce rules before objects are persisted.
They prevent misconfigurations, reduce risk, and ensure compliance.
Policy as Code makes security repeatable and auditable.
Automate governance. Shift security left.

POLICIES AS CODE.
SECURE BY DESIGN.
COMPLIANT BY DEFAULT.
GOVERNED ALWAYS.

PAGE 23

① 1. ADMISSION WORKFLOW



1. API Server receives the request.
2. Sends the request to Admission Policy Engine.
3. Engine evaluates the policy.
4. Engine returns ALLOW or DENY (or MUTATE).
5. If allowed, the object is persisted to etcd.

TIP Admission controllers are your last line of defense before a bad configuration enters the cluster.

② 2. OPA GATEKEEPER

Open Policy Agent (OPA) Gatekeeper enforces policies using Rego.

KEY FEATURES

- ✓ Constraint-based policies
- ✓ Uses Rego language
- ✓ Validating & Mutating
- ✓ Audit and dry-run
- ✓ Excellent for compliance

EXAMPLE: K8S REQUIRED LABEL POLICY

```
package k8srequiredlabels
violation[{"msg": msg}] {
  input.review.kind.kind == "Pod"
  not input.review.object.metadata.labels.app
  msg = "label 'app' is required"
}
```

SAMPLE CONSTRAINT

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
  name: require-app-label
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
  parameters:
    labels: ["app"]
```

TIP Gatekeeper works well for platform teams enforcing enterprise standards.

③ 3. KYVERNO

Kyverno is a Kubernetes-native policy engine.
It supports validation, mutation, and generation.

KEY FEATURES

- ✓ Simple YAML policies
- ✓ Validate, mutate, generate
- ✓ Background scans
- ✓ Policy reports
- ✓ Great developer experience

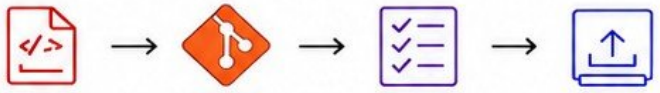
EXAMPLE: DISALLOW LATEST TAG

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-latest-tag
spec:
  validationFailureAction: enforce
  rules:
  - name: block-latest-tag
    match:
      resources:
        kinds: ["Pod"]
    validate:
      message: "Images with :latest tag are not allowed"
      pattern:
        spec:
          containers:
          - image: "!*:latest"
```

TIP Kyverno is easy to start and ideal for both platform and app teams.

④ 4. POLICY AS CODE

Store policies in Git. Review like application code.



BEST PRACTICES

- ✓ Version control all policies
- ✓ Use pull requests and code reviews
- ✓ Automate tests in CI/CD pipelines
- ✓ Promote policies across environments
- ✓ Document intent and examples

TIP Policies are code. Treat them with the same discipline as apps.

⑤ 5. COMPLIANCE

Use policy engines to enforce standards and frameworks.

COMMON FRAMEWORKS

- CIS Kubernetes Benchmark
- NIST 800-53 / 800-190
- PCI DSS
- SOC 2
- ISO 27001
- HIPAA

EXAMPLE CONTROLS ENFORCED

- No privileged containers
- Drop ALL capabilities
- Require resource requests
- Disallow hostPath volumes
- Enforce readOnlyRootFilesystem
- Require labels / annotations

EXAMPLE: DISALLOW PRIVILEGED CONTAINERS

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-privileged
spec:
  validationFailureAction: enforce
  rules:
  - name: no-privileged
    match:
      resources:
        kinds: ["Pod"]
    validate:
      message: "Privileged containers are not allowed"
      pattern:
        spec:
          =(securityContext):
            privileged: false
```

TIP Compliance is not a document. It is enforced continuously.

⑥ 6. GOVERNANCE

Strong governance reduces risk and improves reliability.

GOVERNANCE PILLARS

- People
- Process
- Policy
- Monitoring
- Audit

GOVERNANCE PRACTICES

- ✓ Define policy ownership
- ✓ Establish change management
- ✓ Enforce least privilege
- ✓ Monitor policy violations
- ✓ Report and audit regularly
- ✓ Continuously improve policies

TIP Governance is about consistency, visibility, and accountability. Automate everything you can.

⑦ QUICK REFERENCE

OPA GATEKEEPER COMMANDS

```
kubectl get constraints                  # List all constraints
kubectl get constrainttemplates          # List templates
kubectl get -A violations                # List policy violations
kubectl describe constraint <name>       # Describe a constraint
kubectl apply -f constraint.yaml         # Apply a constraint
```

KYVERNO COMMANDS

```
kubectl get clusterpolicies              # List policies
kubectl get -n kyverno policyreports     # View reports
kubectl get -A polr                      # List policy reports
kubectl describe clusterpolicy <name>    # Describe policy
kubectl apply -f policy.yaml             # Apply policy
```

COMMON POLICY ACTIONS

ALLOW	Permit the request
DENY	Block the request
MUTATE	Modify the object
AUDIT	Report in audit mode

POLICY ENGINES + POLICY AS CODE = SECURITY, COMPLIANCE, AND GOVERNANCE AT SCALE.

VERIQTA

VERIQTA

INFRASTRUCTURE AS CODE SECURITY

SECURE IaC
SECURE CLOUD
SECURE PRODUCTION

PAGE 24

Secure your infrastructure before it reaches production.
Treat IaC as application code. Scan. Validate. Enforce. Monitor.
Build secure, compliant and repeatable infrastructure.

1 WHY IaC SECURITY MATTERS

- ✓ IaC defines critical cloud resources
- ✓ Misconfigurations create security risks
- ✓ Developers can deploy insecure resources fast
- ✓ Security must shift to code and automation
- ✓ Prevents incidents before deployment

KEY IDEA

Your cloud is only as secure as your infrastructure code.

2 TERRAFORM SECURITY

BEST PRACTICES

- ✓ Use remote backend with encryption
- ✓ Enable state file locking
- ✓ Encrypt state with KMS
- ✓ Avoid hardcoded secrets
- ✓ Use least privilege IAM
- ✓ Validate configurations

SECURE BACKEND EXAMPLE

```
terrafrom {
 backend "s3" {
   bucket         = "veriqta-terraform-state"
   key            = "prod/terraform.tfstate"
   region         = "us-east-1"
   encrypt        = true
   dynamodb_table = "terraform-locks"
 }
}
```

3 CLOUDFORMATION SECURITY

BEST PRACTICES

- ✓ Use parameter validation
- ✓ Encrypt sensitive values
- ✓ Apply IAM least privilege
- ✓ Enable stack termination protection
- ✓ Use change sets for review
- ✓ Follow AWS Well-Architected

EXAMPLE: ENCRYPTED S3 BUCKET

```
Resources:
  SecureBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketEncryption:
        ServerSideEncryptionConfiguration:
          - ServerSideEncryptionByDefault:
              SSEAlgorithm: AES256
```

4 IaC SCANNING

POPULAR TOOLS

- ✓ Checkov
- ✓ tfsec
- ✓ Trivy
- ✓ Terrascan
- ✓ Snyk

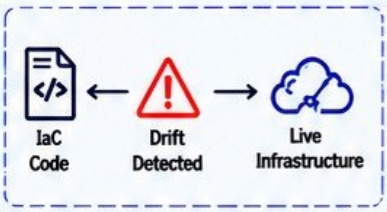
TIP Scan IaC in CI/CD to catch misconfigurations early.

5 DRIFT DETECTION

Detect when real infrastructure no longer matches your code.

TOOLS

- ✓ Terraform plan
- ✓ Treraform Cloud
- ✓ AWS CloudFormation drift detection
- ✓ Atlantis
- ✓ Spacelift



BENEFITS

- ✓ Maintain consistency
- ✓ Detect manual changes
- ✓ Reduce configuration drift
- ✓ Improve compliance
- ✓ Strengthen auditability

6 SECURITY RISKS TO AVOID

COMMON MISTAKES

- ✗ Hardcoding secrets in code
- ✗ Using overly permissive IAM
- ✗ Exposing resources publicly
- ✗ Skipping code review
- ✗ No scanning or validation
- ✗ Storing state files locally

SECURE APPROACH

- ✓ Use secret managers
- ✓ Apply least privilege
- ✓ Restrict public access
- ✓ Enforce pull requests
- ✓ Enable automated scanning
- ✓ Use remote encrypted state

7 IaC SECURITY BEST PRACTICES

- ✓ Treat IaC as application code
- ✓ Use version control (Git)
- ✓ Enforce code review
- ✓ Use secure defaults
- ✓ Apply least privilege
- ✓ Validate and scan in CI/CD
- ✓ Encrypt all sensitive data
- ✓ Use modules for consistency
- ✓ Track and detect drift
- ✓ Document and audit changes
- ✓ Follow security frameworks
- ✓ Continuously improve

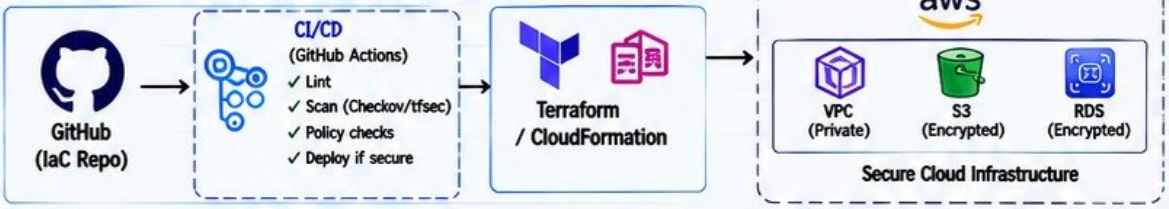
REMEMBER

Secure IaC builds secure cloud environments at scale.

8 CI/CD SECURITY INTEGRATION

- ✓ Automate security checks
- ✓ Block insecure deployments
- ✓ Enforce policies as code
- ✓ Build secure by design pipelines

9 REAL-WORLD SECURITY STACK



KEY OUTCOME

- ✓ Secure code
- ✓ Secure deployment
- ✓ Secure infrastructure
- ✓ Secure production

CODE SECURE. DEPLOY SECURE. OPERATE SECURE.

VERIQTA

TERRAFORM SECURITY

Terraform manages real infrastructure. Secure your code, state, and access to prevent leaks, tampering, and unauthorized changes. Security must be built in from day one.

SECURE CODE
SECURE STATE
SECURE ACCESS
SECURE INFRASTRUCTURE

① SENSITIVE VARIABLES

Never hardcode secrets in .tf files.
Use variables + secure secret stores.

```
# variables.tf
variable "db_password" {
  type        = string
  sensitive   = true
  description = "Database password"
}

# terraform.tfvars (DO NOT COMMIT)
db_password = "SuperSecret123!"
```

BEST PRACTICES

- ✓ Mark variables as sensitive
- ✓ Never commit .tfvars
- ✓ Use environment variables
- ✓ Use secret managers
- ✓ Rotate secrets regularly

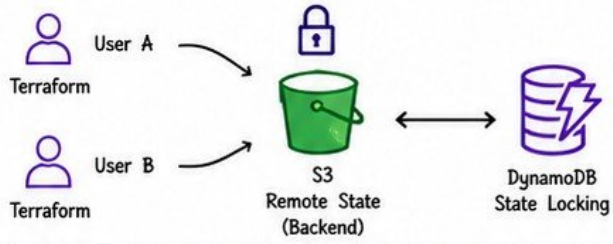
TIP

Export from environment:

```
export TF_VAR_db_password="SuperSecret123!"
```

② REMOTE STATE

Use remote state to protect integrity and enable collaboration.



BENEFITS

- ✓ Centralized state storage
- ✓ Prevents state corruption
- ✓ Enables team collaboration
- ✓ State locking prevents conflicts

EXAMPLE: BACKEND CONFIG (S3 + DYNAMODB)

```
terraform {
  backend "s3" {
    bucket         = "veriqta-terraform-state"
    key            = "prod/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt        = true
  }
}
```

③ STATE ENCRYPTION

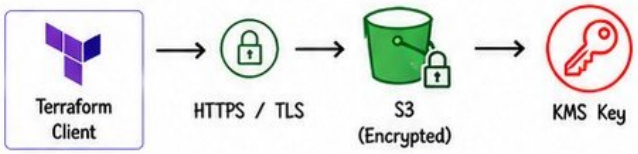
Protect your state file— it contains all resource details.

AT REST (S3)

- ✓ Enable S3 server-side encryption (SSE-KMS)
- ✓ Use a dedicated KMS key
- ✓ Restrict bucket access

IN TRANSIT

- ✓ Use HTTPS endpoint
- ✓ Enforce TLS 1.2+
- ✓ Block public access



TIP Never store state locally in team environments.

④ IAM

Use least privilege IAM for Terraform users and roles.

BEST PRACTICES

- ✓ Use dedicated IAM user or role
- ✓ Attach only required policies
- ✓ Avoid AdministratorAccess
- ✓ Use AssumeRole for CI/CD
- ✓ Enable MFA for humans

EXAMPLE: LEAST PRIVILEGE POLICY (S3 READ/WRITE STATE)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::veriqta-terraform-state",
        "arn:aws:s3:::veriqta-terraform-state/*"
      ]
    }
  ]
}
```

Never use root credentials.
Rotate access keys regularly.

⑤ SECURE MODULES

Modules are reusable—make them secure by design.

- ✓ Use trusted module sources
- ✓ Pin module versions
- ✓ Scan modules for vulnerabilities
- ✓ Avoid dynamic code execution
- ✓ Validate inputs with variable constraints

EXAMPLE: MODULE SOURCE

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"

  cidr = "10.0.0.0/16"
  azs  = ["us-east-1a", "us-east-1b"]
}
```

TIP Review module code before use. Prefer official or well maintained modules.

TERRAFORM SECURITY BEST PRACTICES

- Treat Terraform code as production code
- Store secrets in dedicated secret managers
- Use remote state with locking
- Encrypt state at rest and in transit
- Apply least privilege IAM
- Scan code and modules regularly
- Audit Terraform plans and changes
- Backup state files regularly
- Monitor and alert on changes

TOOLS TO CONSIDER

- tflint
- tfsec
- Checkov
- Terrascan
- Trivy IaC
- AWS Config
- CloudTrail

REMEMBER

Secure IaC.
Secure state.
Secure everything.
Automate security.
Stay protected.

VERIQTA

CLOUD SECURITY FUNDAMENTALS

Cloud security is about protecting data, applications, and infrastructure while enabling speed, availability, and innovation.
Strong foundations across identity, networking, storage, and monitoring reduce risk and build trust.

SECURE TODAY.
RESILIENT TOMORROW.
TRUST ALWAYS.

PAGE 26

① SHARED RESPONSIBILITY MODEL

Security in the cloud is a partnership.

AWS / CLOUD PROVIDER	CUSTOMER / YOU
Physical facilities	Data classification
Hardware / Infrastructure	Identity & access management
Global network	Operating systems
Virtualization layer	Applications & code
Managed services	Network configurations
Compliance of the cloud	Endpoint & device security
	Monitoring & logging
	Patches & updates

TIP You control security IN the cloud.
The provider secures. security OF the cloud.

② IDENTITY

Strong identity is the first line of defense.

People · Permissions · Policies · MFA · Federation (SSO)

BEST PRACTICES

- ✓ Use least privilege access
- ✓ Enable MFA everywhere
- ✓ Use IAM roles (not users)
- ✓ Rotate credentials regularly
- ✓ Use federated identity (SSO)
- ✓ Review access continuously
- ✓ Remove unused access

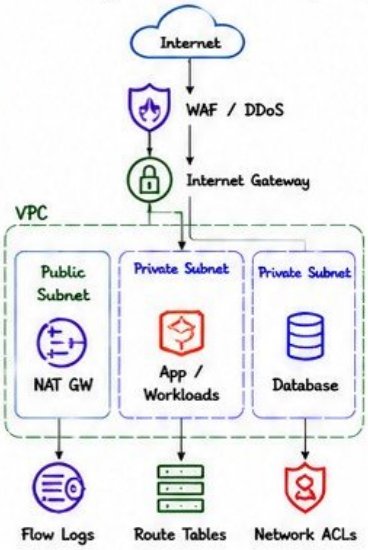
EXAMPLE: LEAST PRIVILEGE POLICY

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": ["arn:aws:s3:::veriqta-bucket"]
    }
  ]
}
```

TIP Give minimum access needed, for minimum time needed.

③ NETWORKING

Design networks that are private, controlled, and monitored.



BEST PRACTICES

- ✓ Use VPC and private subnets
- ✓ Use security groups
- ✓ Use NACLs for subnet control
- ✓ Use NAT Gateway for egress
- ✓ Enable VPC Flow Logs
- ✓ Restrict inbound traffic
- ✓ Use private endpoints
- ✓ Segment with multiple subnets

AVOID

- ✗ 0.0.0.0/0 on inbound rules
- ✗ Exposing databases to Internet
- ✗ Overly permissive security groups
- ✗ No network logging

④ STORAGE

Protect data at rest, in transit, and in use.

Object Storage (S3) · Block Storage (EBS) · File Storage (EFS) · Backup & Archive · KMS Encryption

BEST PRACTICES

- ✓ Encrypt all data at rest (KMS)
- ✓ Encrypt data in transit (TLS)
- ✓ Use bucket policies & ACLs
- ✓ Block public access
- ✓ Enable versioning
- ✓ Enable access logging
- ✓ Use lifecycle policies
- ✓ Regular backups & tests

EXAMPLE: S3 BUCKET POLICY

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": "arn:aws:s3:::veriqta-bucket/*",
      "Condition": {
        "Bool": {"aws:SecureTransport": "false"}
      }
    }
  ]
}
```

TIP Classify data. Encrypt it. Control access. Back it up.

⑤ MONITORING & DETECTION

You can't protect what you don't see.

Logs & Metrics · Alerts · Dashboards · Threat Detection · Audit Trail

BEST PRACTICES

- ✓ Enable CloudTrail (API activity)
- ✓ Enable CloudWatch (metrics)
- ✓ Centralize and retain logs
- ✓ Create meaningful alerts
- ✓ Use GuardDuty for threats
- ✓ Automate response actions
- ✓ Review logs and investigate

TOOLS (AWS EXAMPLES)

- CloudTrail
- CloudWatch
- AWS Config
- GuardDuty
- Security Hub
- Inspector
- VPC Flow Logs

TIP Collect → Monitor → Detect → Alert → Respond → Improve

⑥ CLOUD SECURITY PRINCIPLES

Simple principles. Massive impact.

- ✓ **DEFENSE IN DEPTH** — Use multiple layers of security.
- ✓ **LEAST PRIVILEGE** — Grant minimal access required.
- ✓ **SECURE BY DEFAULT** — Build with security from the start.
- ✓ **AUTOMATE SECURITY** — Use tools, policies, and guardrails.
- ✓ **CONTINUOUS MONITORING** — Visibility drives security.
- ✓ **SHARE RESPONSIBILITY** — Understand your role in the cloud.

REMEMBER

- ✓ People
- ✓ Process
- ✓ Technology
- ✓ Data
- ✓ Visibility
- ✓ Compliance

ALL MATTER.

STRONG FOUNDATIONS. SECURE SYSTEMS. TRUSTED CLOUD.

VERIQTA

AWS SECURITY

AWS provides powerful security services. Use them together to protect identities, data, workloads, and infrastructure. Security is everyone's responsibility.

SECURE IDENTITIES.
ENCRYPT EVERYTHING.
MONITOR CONTINUOUSLY.
RESPOND QUICKLY.

① IAM (IDENTITY AND ACCESS MANAGEMENT)

Control who can access AWS and what they can do.

KEY COMPONENTS

- **Users** — Individual identities
- **Groups** — Collection of users
- **Roles** — Assume permissions
- **Policies** — JSON permission documents

IAM FLOW

User → Policy → Permissions → AWS Resources

EXAMPLE POLICY (S3 READ ONLY)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket","s3:GetObject"],
      "Resource": ["arn:aws:s3:::my-bucket", "arn:aws:s3:::my-bucket/*"]
    }
  ]
}
```

BEST PRACTICES

- ✓ Use least privilege
- ✓ Enable MFA
- ✓ Use IAM roles (no long-term keys)
- ✓ Rotate access keys
- ✓ Prefer policies over inline permissions
- ✓ Review access regularly
- ✓ Use IAM Access Analyzer

PRINCIPLE

Grant only the minimum access needed to perform a task.

② KMS (KEY MANAGEMENT SERVICE)

Create and manage encryption keys.

HOW KMS WORKS

Plaintext Data → KMS Key (Encrypt) → Ciphertext Data

KEY FEATURES

- ✓ Customer managed keys (CMK)
- ✓ AWS managed keys
- ✓ Key rotation
- ✓ Fine-grained key policies
- ✓ Integrates with many AWS services
- ✓ Hardware Security Modules (HSM)

EXAMPLE: ENCRYPT S3 WITH KMS

S3 Bucket → KMS Key → Encrypted Objects

TIP Use customer managed keys for sensitive data.

③ CLOUDTRAIL

Records API activity and account events.

WHAT IT RECORDS

- ✓ API calls (management & data events)
- ✓ Console sign-in events
- ✓ Resource changes
- ✓ AWS service events

User / Service → AWS API Request → CloudTrail → S3 Bucket (Logs) → Analyze / Monitor

BEST PRACTICES

- ✓ Enable organization trail
- ✓ Log to S3 with versioning
- ✓ Enable CloudTrail Insights
- ✓ Create CloudWatch Alarms
- ✓ Protect log files (S3 bucket policy + KMS)

TIPS

CloudTrail is your audit log. Always enable it.

④ GUARDDUTY

Intelligent threat detection service.

KEY FEATURES

- ✓ Continuous monitoring
- ✓ Detects malicious activity
- ✓ ML-powered threat detection
- ✓ No agents to manage
- ✓ Works across accounts and regions

HOW IT WORKS

AWS Environment → GuardDuty Analyzes → Findings → Take Action

EXAMPLE FINDINGS

- EC2 instance communicating with malicious IP
- Unauthorized access: IAM user behavior
- Crypto mining activity
- Reconnaissance activities

BEST PRACTICES

- ✓ Enable in all accounts
- ✓ Review findings daily
- ✓ Integrate with Security Hub
- ✓ Automate response (Lambda, SNS, SSM Automation)

TIP GuardDuty uses threat intelligence, anomaly detection, and ML to identify threats automatically.

⑤ SECURITY HUB

Centralize, aggregate, and prioritize security findings.

OVERVIEW

- ✓ Aggregates findings from multiple sources
- ✓ Standardizes and prioritizes findings
- ✓ Provides security score
- ✓ Integrates with AWS and third-party tools
- ✓ Supports compliance standards (CIS, PCI, ISO, etc.)

AGGREGATION FLOW

GuardDuty, Inspector, AWS Config, Third-party Tools

↓

Security Hub (Aggregates & Prioritizes)

↓

Dashboard / Findings / Actions

BEST PRACTICES

- ✓ Enable across all accounts
- ✓ Use automation for remediation
- ✓ Review security score
- ✓ Enable compliance standards
- ✓ Integrate with CloudWatch and Ticketing systems

PRINCIPLE

Visibility + Context = Better Security Decisions

TIP Security Hub gives you a single pane of glass for security posture across your AWS environment.

SECURE IDENTITIES. PROTECT DATA. DETECT THREATS. RESPOND FAST.

VERIQTA

AZURE SECURITY

Azure provides integrated security services to protect identities, data, applications, and infrastructure across your cloud environment.

SECURE IDENTITIES.
PROTECT DATA.
DETECT THREATS.
RESPOND FAST.

1 MICROSOFT ENTRA ID (IDENTITY AND ACCESS)

Cloud identity and access management for Azure resources.

KEY FEATURES

- Centralized identity for all Azure resources
- Single Sign-On (SSO)
- Multi-Factor Authentication (MFA)
- Conditional Access
- Privileged Identity Management (PIM)
- Identity Protection

AUTHENTICATION FLOW

User → Sign In → Microsoft Entra ID → Access to Azure Resources

EXAMPLE: CONDITIONAL ACCESS POLICY

```
{
  "conditions": {
    "users": [ "All" ],
    "applications": [ "All" ],
    "clientAppTypes": [ "browser" ],
    "locations": { "exclude": [ "0.0.0.0/0" ] }
  },
  "grantControls": {
    "operator": "AND",
    "builtInControls": [ "mfa" ]
  }
}
```

BEST PRACTICES

- ✓ Enforce MFA for all users
- ✓ Use Conditional Access principles
- ✓ Apply least privilege access
- ✓ Regularly review roles
- ✓ Monitor sign-in logs
- ✓ Use Identity Protection for risky sign-ins

PRINCIPLE

Verify users continuously. Grant least privilege access.

2 DEFENDER FOR CLOUD

Cloud-native security posture management (CSPM) and workload protection.

KEY CAPABILITIES

- ✓ Security posture assessment
- ✓ Vulnerability management
- ✓ Just-in-time VM access
- ✓ Application security
- ✓ Container security
- ✓ Threat protection

HOW IT WORKS

Resources (Azure & Multi-cloud) → Defender for Cloud → Recommendations & Alerts → Secure & Compliant Environment

BEST PRACTICES

- ✓ Enable on all subscriptions
- ✓ Review secure score
- ✓ Fix high security recommendations first
- ✓ Enable JIT access for VMs
- ✓ Integrate with Sentinel

TIP

High secure score does not mean zero risk. Investigate continuously.

3 KEY VAULT

Securely store and manage secrets, keys, and certificates.

KEY FEATURES

- ✓ Secrets management
- ✓ Keys management
- ✓ Certificates management
- ✓ RBAC + Access policies
- ✓ Soft delete & purge protection
- ✓ HSM-backed keys

HOW IT WORKS

Applications → Key Vault → Secrets / Keys / Certificates → Secure Access

BEST PRACTICES

- ✓ Use RBAC over access policies
- ✓ Enable soft delete
- ✓ Enable purge protection
- ✓ Rotate keys and secrets regularly
- ✓ Use managed identities for access

EXAMPLE: STORE SECRET (CLI)

```
az keyvault secret set \
--vault-name myVault \
--name db-password \
--value "P@sswOrd123!"
```

TIP Never hardcode secrets in code. Use Key Vault.

4 AZURE POLICY

Enforce organizational standards and assess compliance.

KEY FEATURES

- ✓ Create custom policies
- ✓ Assign policies to scopes
- ✓ Audit and deny actions
- ✓ Initiative (policy set)
- ✓ Effect types: Audit, Deny, Append, Modify

POLICY FLOW

Policy Definition → Assign to Scope → Evaluate Resources → Compliant or Non-compliant

BEST PRACTICES

- ✓ Use built-in policies first
- ✓ Apply deny for critical standards
- ✓ Use initiative for grouping
- ✓ Tag and document policies
- ✓ Review compliance dashboard regularly

EXAMPLE: SENY PUBLIC IP

```
{
  "if": {
    "field": "type",
    "equals": "Microsoft.Network/
    publicIPAddresses"
  },
  "then": {
    "effect": "Deny"
  }
}
```

5 AZURE SENTINEL (OVERVIEW)

Cloud-native SIEM and SOAR for intelligent security operations.

CAPABILITIES

- ✓ Collect and analyze logs
- ✓ Threat detection
- ✓ Security dashboards
- ✓ Incident management
- ✓ Automated response (SOAR)
- ✓ Hunting and investigation

SENTINEL ARCHITECTURE

Data Connectors → Log Analytics → Analytics & Threat Detection → Alerts & Response

BEST PRACTICES

- ✓ Connect all relevant data sources
- ✓ Use analytic rules
- ✓ Tune alerts to reduce noise
- ✓ Automate responses for common threats
- ✓ Review incidents and improve

TIP

Good visibility + fast response = strong security outcomes.

AZURE SECURITY PILLARS

- **IDENTITY** — Secure who can access
- **DATA** — Encrypt and protect data
- **THREAT PROTECTION** — Detect and stop threats
- **COMPLIANCE** — Meet standards and regulations
- **MONITORING** — Observe, analyze, and respond

VERIQTA

GOOGLE CLOUD SECURITY

Google Cloud provides integrated security services to help you protect identities, data, applications, and infrastructure across your cloud environment.

SECURE IDENTITIES.
PROTECT DATA.
DETECT THREATS.
RESPOND FAST.

1 IAM (IDENTITY AND ACCESS MANAGEMENT)

Manage access to Google Cloud resources securely.

KEY COMPONENTS

- **Principals** — Users, groups, service accounts
- **Roles** — Permissions bundled together
- **Policies** — Allow or deny access
- **Resource Hierarchy** — Organization → Folder → Project → Resource

IAM FLOW

Principal → Authentication → Authorization → Resource Access

EXAMPLE: IAM POLICY (ALLOW)

```
{
 "bindings": [
  {
   "role": "roles/storage.objectViewer",
   "members": ["user:ann@veriqta.com"]
  }
 ]
}
```

BEST PRACTICES

- ✓ Enable IAM Recommender
- ✓ Use least privilege
- ✓ Use service accounts
- ✓ Use conditional IAM
- ✓ Avoid using Owner role
- ✓ Review access regularly
- ✓ Use IAM Deny for guardrails
- ✓ Enable audit logs

PRINCIPLE

Grant the minimum access needed to perform a task.

2 CLOUD ARMOR

Protect applications from DDoS attacks and web threats.

KEY FEATURES

- DDoS protection
- WAF for HTTP(S) traffic
- Geo-based filtering
- Rate limiting & throttling
- Bot management
- Adaptive protection

HOW IT WORKS

Internet → Cloud Armor → Load Balancer → Backends (VMs, GKE, etc.)

BEST PRACTICES

- ✓ Enable DDoS protection
- ✓ Use managed rule sets
- ✓ Create custom rules
- ✓ Enable logging & monitoring
- ✓ Apply rate limiting
- ✓ Review security policies

TIP

Place Cloud Armor in front of external applications using HTTP(S) Load Balancing.

3 SECRET MANAGER

Store, manage, and access secrets securely.

KEY FEATURES

- Centralized secret storage
- Automatic secret rotation
- Fine-grained IAM access
- Replicate secrets
- Integrates with services
- Audit logging

HOW IT WORKS

Create Secret → Store in Secret Manager → Rotate (Optional) → Access via API / SDK

BEST PRACTICES

- ✓ Use automatic rotation
- ✓ Grant least privilege
- ✓ Use CMEK for encryption
- ✓ Avoid hardcoding secrets
- ✓ Monitor access logs
- ✓ Disable unused versions

EXAMPLE: ACCESS SECRET

```
# gcloud CLI
gcloud secrets versions access latest \
  --secret=my-secret
```

TIP

Use Workload Identity to access secrets without keys.

4 SECURITY COMMAND CENTER

Unified security posture management and threat detection.

KEY CAPABILITIES

- Asset inventory
- Vulnerability management
- Threat detection
- Misconfiguration findings
- Web security scanner
- Compliance monitoring

HOW IT WORKS

Collect Assets → Analyze Findings → Prioritize Risks → Remediate & Monitor

BEST PRACTICES

- ✓ Enable all sources
- ✓ Review findings daily
- ✓ Integrate with SIEM
- ✓ Use notification channels
- ✓ Track remediation progress
- ✓ Enforce organization policies

TIP

Use SCC dashboards to track security posture across projects.

5 WORKLOAD IDENTITY

Securely access Google Cloud services from workloads.

KEY FEATURES

- Eliminates service account keys
- Federated identity
- Works with GKE, Cloud Run, GCE, and more
- Automatic token rotation
- Fine-grained IAM control

HOW IT WORKS

Workload (GKE / Cloud Run) → Workload Identity Federation → Short-lived Token → Access Google Cloud Resources

BEST PRACTICES

- ✓ Use Workload Identity
- ✓ Disable service account keys
- ✓ Grant least privilege
- ✓ Audit access logs
- ✓ Rotate and review roles

PRINCIPLE

Workloads get access without long-lived credentials.

VERIQTA

ARTIFACT SECURITY

Artifacts are the outputs of your build pipelines. Protecting artifacts ensures only trusted, verified, and untampered software is promoted to production.

TRUST YOUR ARTIFACTS.
VERIFY EVERYTHING.
PROMOTE SAFELY.

PAGE 30

1 ARTIFACT REPOSITORIES

Centralized storage for build outputs and dependency artifacts.

COMMON REPOSITORIES

- JFrog Artifactory
- Sonatype Nexus
- AWS CodeArtifact
- Azure Artifacts
- Google Artifact Registry
- GitHub Packages

WHAT THEY STORE

- Container images
- Helm charts
- Libraries & dependencies
- Build packages
- Binary files
- Infrastructure modules

KEY CONTROLS

- Authentication & Authorization
- Fine-grained access control
- Immutable artifacts
- Repository segmentation
- Retention policies
- Audit logging

BEST PRACTICES

- ✓ Separate repos per environment
- ✓ Enforce least privilege
- ✓ Enable immutability
- ✓ Scan artifacts for vulnerabilities
- ✓ Enable audit logs
- ✓ Use lifecycle policies

TIP
Never pull artifacts from untrusted or public repositories in production.

2 SIGNING

Cryptographically sign artifacts to prove authenticity.

HOW SIGNING WORKS

Build Artifact → Sign with Private Key → Signed Artifact → Verify with Public Key

SIGNING TOOLS & STANDARDS

- ✓ GPG / PGP
- ✓ Cosign (Container signing)
- ✓ Notary v2
- ✓ Sigstore (Fulcio + Rekor)
- ✓ SLSA Provenance
- ✓ X.509 Certificates

BEST PRACTICES

- ✓ Sign every build artifact
- ✓ Store public keys securely
- ✓ Verify before use or deployment
- ✓ Automate signing in CI/CD
- ✓ Rotate signing keys regularly

TIP
If it's not signed, don't trust it.

3 PROVENANCE

Provenance is the story of how an artifact was built.

PROVENANCE CAPTURES

- ✓ Source code commit
- ✓ Build pipeline details
- ✓ Builder identity
- ✓ Dependencies used
- ✓ Build environment
- ✓ Timestamps
- ✓ Parameters & configs

PROVENANCE FLOW

Source Code → Build Pipeline → Provenance (SLSA / in-toto) → Artifact

STANDARDS & TOOLS

- ✓ SLSA (Supply-chain Levels for Software Artifacts)
- ✓ in-toto
- ✓ Cosign attestations
- ✓ Tekton Chains
- ✓ GitHub OIDC + Attestations

BEST PRACTICES

- ✓ Generate provenance for every build
- ✓ Store provenance with artifacts
- ✓ Verify provenance before promotion
- ✓ Use OIDC to bind builds to identity
- ✓ Keep provenance tamper-proof

TIP
Provenance gives you confidence in where the artifact came from.

4 INTEGRITY

Ensure artifacts are not tampered with after creation.

INTEGRITY CONTROLS

- ✓ Checksums (SHA256/SHA512)
- ✓ Digital signatures
- ✓ Immutable storage
- ✓ Access controls
- ✓ Tamper-evident logs
- ✓ Audit & monitoring

INTEGRITY FLOW

Artifact → Generate Checksum → Store & Protect (Immutable) → Verify Before Use/Deploy

INTEGRITY TOOLS

- ✓ SHA256 / SHA512
- ✓ Sigstore / Cosign
- ✓ Notary v2
- ✓ Immutable repositories
- ✓ WORM storage

BEST PRACTICES

- ✓ Enforce immutability
- ✓ Verify checksums before use
- ✓ Protect repository creds
- ✓ Monitor access and changes
- ✓ Alert on integrity violations

TIP
Integrity ensures the artifact is exactly what you expect.

5 PROMOTION

Safely promote artifacts through environments.

PROMOTION STRATEGY

- ✓ Promote, don't rebuild
- ✓ Use immutable artifacts
- ✓ Automated quality gates
- ✓ Scan before promotion
- ✓ Require approvals
- ✓ Rollback ready

PROMOTION FLOW

Dev/ Build → Quality Gate → Promote Artifact → Staging → Production

BEST PRACTICES

- ✓ Use the same artifact across environments
- ✓ Store promote history
- ✓ Enforce approval workflows
- ✓ Validate before production
- ✓ Automate rollback paths

PRINCIPLE

Promote trusted artifacts. Never rebuild for production.

ARTIFACT SECURITY CHECKLIST

Store securely | Sign everything | Capture provenance | Verify integrity | Promote safely | Monitor always

TRUSTED ARTIFACTS. SECURE PIPELINES. PRODUCTION CONFIDENCE.

VERIQTA @veriqta

VERIQTA

SBOM (SOFTWARE BILL OF MATERIALS)

An SBOM is a formal record of the components, dependencies, and libraries that make up your software. It is a foundation for security, compliance, and supply chain transparency.

KNOW WHAT YOU BUILD.
MANAGE RISK EARLY.
PROTECT YOUR SUPPLY CHAIN.
BUILD TRUST.

1 WHY SBOM MATTERS

KEY REASONS

- ✓ Identify vulnerable components
- ✓ Improve incident response
- ✓ Meet compliance requirements
- ✓ Reduce software supply chain risk
- ✓ Enable license compliance
- ✓ Increase customer trust
- ✓ Required by governments and industry standards

THE PROBLEM WITHOUT SBOM

Software Application → Unknown Dependencies → Vulnerabilities & Risks → Security, Compliance & Business Impact

WITH SBOM

Software Application → SBOM Visibility → Risk Assessment → Secure, Compliant & Trusted

BEST PRACTICES

- ✓ Generate SBOM for every build
- ✓ Keep SBOMs up to date
- ✓ Store SBOMs securely
- ✓ Integrate SBOM in CI/CD
- ✓ Scan SBOM for vulnerabilities
- ✓ Share SBOMs with consumers
- ✓ Review and validate SBOMs
- ✓ Continuously improve processes

TIP

You can't secure what you can't see. An SBOM gives you a complete map of your software.

2 SBOM FORMATS

Common formats used to represent SBOM data.

SPDX

- ✓ Open standard
- ✓ Widely adopted
- ✓ Human & machine readable
- ✓ Supports licensing and compliance

CYCLONEDX

- ✓ Modern & flexible
- ✓ Native JSON & XML
- ✓ Security focused
- ✓ Easy for tools to consume

SWID TAG

- ✓ ISO / IEC 19770-2
- ✓ Enterprise asset management
- ✓ Hardware & software inventory

OTHER FORMATS

- ✓ Proprietary formats
- ✓ May exist in legacy systems
- ✓ Consider conversion to open standards

FORMAT COMPARISON

Format	Best For	Data Model	Adoption
SPDX	Compliance & Licensing	Rich	High
CycloneDX	Security & DevSecOps	Rich	High
SWID Tag	IT Asset Management	Moderate	Medium
Proprietary	Legacy Systems	Varies	Low

3 SBOM GENERATION

How SBOMs are created in modern pipelines.

GENERATION METHODS

- ✓ Build tool plugins (Maven, Gradle, npm)
- ✓ Language-specific tools
- ✓ Container image scanning
- ✓ Binary analysis
- ✓ Dependency scanning
- ✓ CI/CD pipeline integration

TYPICAL PIPELINE

Code Commit → Build → Generate SBOM → Scan SBOM → Store & Distribute

POPULAR TOOLS

- ✓ Syft (by Anchore)
- ✓ CycloneDX CLI
- ✓ SPDX SBOM Generator
- ✓ Trivy
- ✓ Grype
- ✓ Dependency-Track

TIP

Automate SBOM generation. Manual SBOMs are incomplete and unreliable.

4 SBOM COMPLIANCE

Regulations and standards driving SBOM adoption.

REGULATIONS & STANDARDS

- ✓ US Executive Order 14028
- ✓ NIST SP 800-218
- ✓ CISA Secure Software Development Framework (SSDF)
- ✓ EU CRA (Cyber Resilience Act)
- ✓ ISO / IEC 5962
- ✓ Industry specific mandates

COMPLIANCE BENEFITS

- Meet Legal Requirements
- Pass Audits Faster
- Reduce Risk Exposure
- Build Customer Trust

BEST PRACTICES

- ✓ Understand applicable regulations
- ✓ Generate SBOMs continuously
- ✓ Maintain historical records
- ✓ Provide SBOMs to customers
- ✓ Respond quickly to requests
- ✓ Document processes clearly

TIP

Regulators are asking for SBOMs today. Start now so you're not rushed later.

5 ENTERPRISE USE CASES

How organizations use SBOMs at scale.

COMMON USE CASES

- ✓ Vulnerability management
- ✓ Risk assessment & prioritization
- ✓ Incident response
- ✓ License compliance
- ✓ Third-party risk management
- ✓ Mergers & acquisitions
- ✓ Software transparency initiatives

ENTERPRISE WORKFLOW

Collect SBOMs → Centralize & Store → Analyze & Correlate → Act on Risks → Report & Improve

SUCCESS FACTORS

- ✓ Executive sponsorship
- ✓ Tooling integration
- ✓ Standardized processes
- ✓ Cross-team collaboration
- ✓ Continuous monitoring
- ✓ Measure and improve

PRINCIPLE

An SBOM is not a document. It is an ongoing practice.

KNOW YOUR COMPONENTS. MANAGE YOUR RISKS. BUILD SECURE SOFTWARE.

IMAGE SIGNING

VERIQTA

Image signing ensures the integrity, authenticity, and provenance of container images. It prevents tampering and builds trust across your software supply chain.

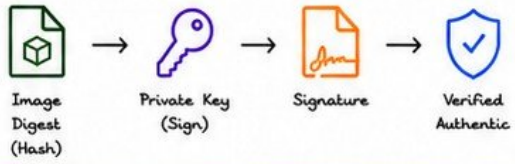
SIGN IT. VERIFY IT. TRUST IT. SECURE DEPLOYMENTS START HERE.

PAGE 32

① DIGITAL SIGNATURES

Prove an image came from you and wasn't changed.

HOW IT WORKS



COMMON SIGNING TOOLS

- ✓ Cosign (Sigstore)
- ✓ Notary v2
- ✓ Docker Content Trust
- ✓ GPG (traditional)
- ✓ AWS Signer
- ✓ Azure Notary (ACR)
- ✓ Google Binary Authorization

BEST PRACTICES

- ✓ Always sign images
- ✓ Use strong key management
- ✓ Protect private keys
- ✓ Automate signing in CI/CD
- ✓ Enforce verification
- ✓ Rotate keys regularly

TIP

Unsigned images are an open door for attackers. Make signing mandatory.

② VERIFICATION

Verify signatures before pulling or deploying images.

VERIFICATION FLOW

VERIFICATION COMMAND (COSIGN)

```
$ cosign verify ghcr.io/veriqta/app:1.0.0 \
  --key cosign.pub

Verify OK

Signed by: veriqta@veriqta.com
Using key: cosign.pub

Certificate: valid
Creation Time: 2024-06-20T12:00:00Z
```

BEST PRACTICES

- ✓ Verify before deploy
- ✓ Enforce in admission controls
- ✓ Fail closed (deny if unverified)
- ✓ Log verification results
- ✓ Use transparency logs

TIP

Verification must happen in the runtime environment, not just in CI.

③ TRUST CHAINS

Build a chain of trust from builder to runtime.

TRUST CHAIN MODEL

KEY ELEMENTS

- ✓ Root of Trust
- ✓ Signing Authority
- ✓ Certificates
- ✓ Identity Providers (OIDC)
- ✓ Transparency Logs
- ✓ Time Stamps

BEST PRACTICES

- ✓ Use short-lived credentials
- ✓ Leverage OIDC federation
- ✓ Store keys in HSM / KMS
- ✓ Log to transparency logs
- ✓ Review trust chain regularly

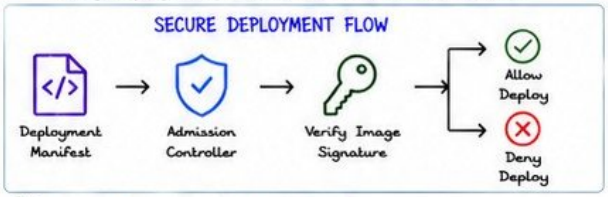
TIP

Sigstore + OIDC eliminates long-lived keys and strengthens trust at every step.

④ SECURE DEPLOYMENT

Only deploy images that are verified and trusted.

SECURE DEPLOYMENT FLOW



ENFORCEMENT METHODS

- ✓ Kubernetes Admission Policies
- ✓ Kyverno / OPA / Gatekeeper
- ✓ Cosign Policy Controller
- ✓ Docker Content Trust
- ✓ AWS EKS Image Signing
- ✓ Azure Policy (ACR)
- ✓ Binary Authorization (GCP)

BEST PRACTICES

- ✓ Enforce verification at runtime
- ✓ Use policy-as-code
- ✓ Deny unsigned images
- ✓ Monitor and alert
- ✓ Audit all deployments

TIP

Block at the gate. Never run what you haven't verified.

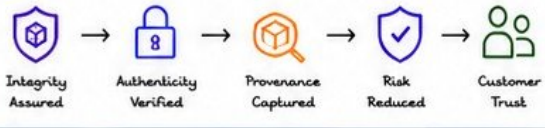
⑤ SUPPLY CHAIN SECURITY

Image signing is a core control for supply chain security.

SUPPLY CHAIN RISKS

- ⚠ Tampered images
- ⚠ Impersonated publishers
- ⚠ Compromised registries
- ⚠ Malicious dependencies
- ⚠ Untrusted build systems
- ⚠ Drift and unauthorized changes

HOW SIGNING MITIGATES RISKS



BEST PRACTICES

- ✓ Sign every image
- ✓ Capture provenance (SBOM)
- ✓ Use attestation & SLSA
- ✓ Monitor supply chain
- ✓ Continuously validate trust

TIP

Signing + SBOM + Provenance = Stronger, verifiable supply chain.

⑥ ENTERPRISE CHECKLIST

A complete image signing program.

KEY CONTROLS

- ✓ Key management & rotation
- ✓ Automated signing in CI/CD
- ✓ Signature verification enforcement
- ✓ Transparency log monitoring
- ✓ Policy enforcement
- ✓ Audit & reporting

CHECKLIST

- ☐ All images are signed
- ☐ Signatures are verified in runtime
- ☐ Policy enforcement is active
- ☐ Transparency logs are monitored
- ☐ Keys are stored securely
- ☐ Incidents are investigated

SUCCESS OUTCOMES

- ✓ Trusted software only
- ✓ Reduced attack surface
- ✓ Compliance & audit readiness
- ✓ Stronger customer confidence
- ✓ Secure, resilient supply chain

PRINCIPLE

You can't secure what you can't trust. Sign everything. Verify always.

SIGN IMAGES. VERIFY ALWAYS. BUILD TRUST. SECURE YOUR SUPPLY CHAIN.

VERIQTA @veriqta

VERIQTA

RUNTIME SECURITY

Runtime security protects your applications and workloads while they are running. It provides visibility, detects threats in real time, and helps you respond quickly to incidents.

SEE EVERYTHING.
DETECT EARLY.
RESPOND FAST.
PROTECT RUNTIME.

PAGE 33

1 RUNTIME MONITORING

Continuously monitor workloads, containers, and hosts.

WHAT WE MONITOR

- Processes
- System Calls
- File Integrity
- Network Connections
- User Activity
- Container / Pod Status
- Configuration Changes
- Resource Usage

RUNTIME MONITORING FLOW

Collect Telemetry → Normalize & Enrich Data → Analyze in Real Time → Alert & Notify

BEST PRACTICES

- ✓ Monitor all layers (Host, Container, App)
- ✓ Use eBPF for deep visibility
- ✓ Enable audit logging
- ✓ Baseline normal behavior
- ✓ Alert on anomalies
- ✓ Retain logs for investigations
- ✓ Continuously tune monitoring

TIP

You can't protect what you can't see. Visibility is the foundation of runtime security.

2 BEHAVIORAL DETECTION

Detect abnormal behavior that indicates potential threats.

BEHAVIOR EXAMPLES

- ✓ Unexpected process execution
- ✓ Privilege escalation attempts
- ✓ Suspicious file modifications
- ✓ Outbound connections to unknown IPs
- ✓ Unusual resource consumption
- ✓ Lateral movement behavior

BEHAVIORAL DETECTION FLOW

Learn Normal Behavior → Detect Anomalies → Score Risk → Generate Alert

DETECTION TECHNIQUES

- ✓ Machine Learning / UEBA
- ✓ Statistical Analysis
- ✓ Rule-based Detection
- ✓ Threat Intelligence
- ✓ Anomaly Scoring
- ✓ Correlation Engine

TIP

Attackers try to blend in. Behavior analytics helps you catch what signatures miss.

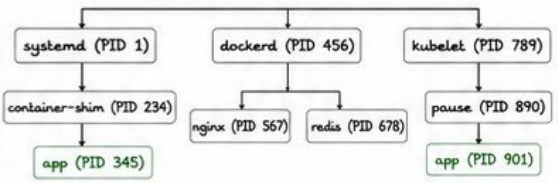
3 PROCESS VISIBILITY

Understand what is running in your environment at all times.

WHAT WE SEE

- Running Processes
- Process Tree (Parent/Child)
- User & Permissions
- Open Files & Sockets
- Loaded Libraries
- Command Line Args

PROCESS VISIBILITY EXAMPLE



BEST PRACTICES

- ✓ Maintain full process tree
- ✓ Track user & privilege context
- ✓ Monitor command lines
- ✓ Detect unauthorized binaries
- ✓ Alert on suspicious activity
- ✓ Store process history

TIP

Process visibility is critical for forensics and threat hunting.

4 THREAT DETECTION

Identify real threats before they cause damage.

THREAT TYPES

- Malware
- Ransomware
- Privilege Escalation
- Container Escape
- Lateral Movement
- Data Exfiltration
- Zero-day Exploits

THREAT DETECTION FLOW

Collect Events → Correlate & Analyze → Detect Threat → Alert & Escalate → Contain (Optional)

SOURCES

- ✓ System Calls
- ✓ File Events
- ✓ Network Traffic
- ✓ Authentication Logs
- ✓ Cloud Audit Logs
- ✓ Application Logs

TIP

Detect early, contain fast, and investigate thoroughly.

5 INCIDENT RESPONSE

Respond quickly and effectively to security incidents.

RESPONSE STEPS

1. Detect & Triage
2. Investigate
3. Contain
4. Eradicate
5. Recover
6. Lessons Learned

INCIDENT RESPONSE WORKFLOW

Alert Received → Investigate & Validate → Contain Threat → Eradicate & Clean → Recover Services → Post-Incident Review

BEST PRACTICES

- ✓ Have an IR playbook
- ✓ Automate containment
- ✓ Preserve evidence
- ✓ Communicate clearly
- ✓ Review and improve
- ✓ Run regular drills

PRINCIPLE

Speed + Clarity + Evidence = Effective Incident Response.

RUNTIME VISIBILITY. THREAT DETECTION. FAST RESPONSE. STRONGER SECURITY.

VERIQTA @veriqta

VERIQTA

CLOUD WORKLOAD PROTECTION

Cloud Workload Protection (CWPP) secures workloads running across your cloud environments—on hosts, in containers, and in Kubernetes. It provides visibility, threat detection, and response to protect what matters most: your runtime.

PROTECT EVERY WORKLOAD.
DETECT THREATS EARLY.
RESPOND FAST.
STAY SECURE AT RUNTIME.

PAGE 34

① CWPP OVERVIEW

CWPP unifies protection across hosts, containers, and Kubernetes.

CWPP IN ACTION

CWPP CAPABILITIES

- ✓ Asset discovery & inventory
- ✓ Vulnerability assessment
- ✓ Runtime threat detection
- ✓ Behavioral analytics
- ✓ Configuration monitoring
- ✓ Compliance & policy enforcement
- ✓ Automated response
- ✓ Security posture management

BEST PRACTICES

- ✓ Deploy agents on all workloads
- ✓ Enable least privilege
- ✓ Keep workloads updated
- ✓ Monitor continuously
- ✓ Integrate with SIEM/SOAR
- ✓ Automate responses
- ✓ Review alerts daily
- ✓ Test your detections

TIP

You can't secure what you can't see. Visibility is the foundation of workload protection.

② HOST PROTECTION

Protect virtual machines, bare metal servers, and cloud instances.

WHAT CWPP PROTECTS

- ✓ Malware & ransomware
- ✓ Privilege escalation
- ✓ Unauthorized access
- ✓ File integrity
- ✓ Rootkits & persistence
- ✓ Weak configurations
- ✓ Exploits & zero-days

HOST PROTECTION FEATURES

BEST PRACTICES

- ✓ Harden OS baselines
- ✓ Apply patches regularly
- ✓ Restrict sudo access
- ✓ Disable unnecessary services
- ✓ Monitor system calls
- ✓ Alert on suspicious activity
- ✓ Use immutable infrastructure

TIP

Hosts are the foundation of your workloads. Secure them first.

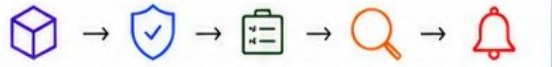
③ CONTAINER PROTECTION

Protect containers throughout their lifecycle.

CONTAINER RISKS

- ⚠ Vulnerable base images
- ⚠ Malicious images
- ⚠ Excessive privileges
- ⚠ Secrets in images
- ⚠ Insecure configs
- ⚠ Lateral movement

CONTAINER PROTECTION LIFECYCLE



BEST PRACTICES

- ✓ Scan images before deploy
- ✓ Use trusted base images
- ✓ Drop unnecessary privileges
- ✓ Run as non-root
- ✓ Use read-only file systems
- ✓ Monitor container behavior
- ✓ Rotate secrets & tokens

TIP

A vulnerable image can compromise your entire environment.

④ KUBERNETES PROTECTION

Secure Kubernetes clusters, nodes, and workloads.

PROTECT WHAT MATTERS

- ✓ Nodes & control plane
- ✓ Pods & containers
- ✓ Namespaces
- ✓ Secrets & ConfigMaps
- ✓ RBAC & service accounts
- ✓ Network policies

K8S PROTECTION FEATURES

BEST PRACTICES

- ✓ Enable NetworkPolicies
- ✓ Use least privilege (RBAC)
- ✓ Scan workloads & images
- ✓ Keep cluster & nodes updated
- ✓ Monitor audit logs
- ✓ Enable pod security standards
- ✓ Detect anomalous behavior

TIP

Kubernetes is powerful. Security must be built into every layer.

⑤ CLOUD-NATIVE SECURITY

Leverage cloud-native controls and integrations for stronger protection.

CLOUD-NATIVE CONTROLS

- ✓ IAM least privilege
- ✓ Security groups / NACLs
- ✓ Workload identity
- ✓ Secrets management
- ✓ Encryption everywhere
- ✓ Zero trust networking

CLOUD-NATIVE SECURITY STACK

BEST PRACTICES

- ✓ Follow shared responsibility
- ✓ Automate security controls
- ✓ Use native security services
- ✓ Enable logging & monitoring
- ✓ Review access regularly
- ✓ Continuously improve posture

TIP

Cloud-native security is automated, integrated, and scalable.

CWPP VALUE

Stronger Protection | Better Visibility | Faster Detection | Rapid Response | Lower Risk | Business Continuity

PROTECT WORKLOADS. SECURE RUNTIMES. STAY AHEAD OF THREATS.

VERIQTA @veriqta

VERIQTA

VULNERABILITY MANAGEMENT

Vulnerability Management is the continuous process of finding, assessing, prioritizing, and fixing security weaknesses across your infrastructure, applications, and workloads.

Goal: Reduce risk by fixing the right vulnerabilities, in the right order, fast.

FIND EARLY.
PRIORITIZE SMART.
FIX FAST.
VERIFY ALWAYS.

1 DISCOVERY

Find all assets and their vulnerabilities.

DISCOVERY SOURCES

- Cloud Assets (VMs, Containers, DBs)
- Applications & APIs
- Network Devices
- Code Repositories
- Third-party & Dependencies

DISCOVERY FLOW

Identify Assets → Scan Targets → Collect Data → Normalize Results → Asset Inventory Updated

BEST PRACTICES

- ✓ Maintain complete asset inventory
- ✓ Scan continuously
- ✓ Include external & internal assets
- ✓ Use authenticated scans
- ✓ Integrate with CMDB
- ✓ Track cloud & on-prem assets

TIP

You can't fix what you don't know exists.

2 PRIORITIZATION

Focus on what matters most.

PRIORITIZATION FACTORS

- ✓ Asset Criticality
- ✓ Vulnerability Severity
- ✓ Exploit Availability
- ✓ Business Impact
- ✓ Exposure (Internet Facing)
- ✓ Threat Intelligence
- ✓ Compensating Controls

PRIORITIZATION MATRIX

IMPACT \ LIKELIHOOD	Low	Medium	High
High	MEDIUM	HIGH	CRITICAL
Medium	LOW	MEDIUM	HIGH
Low	LOW	LOW	MEDIUM

RISK LEVELS: Critical, High, Medium, Low

BEST PRACTICES

- ✓ Use risk-based approach
- ✓ Combine CVSS + Context
- ✓ Review exploit activity
- ✓ Prioritize business-critical assets
- ✓ Reassess risk continuously

TIP

High CVSS doesn't always mean high risk. Context is everything.

3 RISK SCORING

Calculate and score the risk.

COMMON SCORING MODELS

- ✓ CVSS v3.1 (Severity)
- ✓ EPSS (Exploit Probability)
- ✓ Asset Criticality Score
- ✓ Business Impact Score
- ✓ Threat Intelligence Score
- ✓ Environmental Score

RISK SCORING FORMULA (EXAMPLE)

Risk Score = (Severity Score × Exploit Prob) × Asset Criticality × Business Impact

EXAMPLE

Severity 8.0 × Exploit Prob 0.7 × Asset Criticality 0.9 × Business Impact 0.8 = 4.03 Risk Score

BEST PRACTICES

- ✓ Standardize scoring model
- ✓ Automate score calculation
- ✓ Use weighted factors
- ✓ Adjust based on environment
- ✓ Review scores regularly

TIP

Risk scoring turns noise into action.

4 REMEDIATION

Fix vulnerabilities the right way.

REMEDIATION METHODS

- Patch Management
- Code Fixes
- Configuration Changes
- Compensating Controls
- Workaround / Mitigation

REMEDIATION WORKFLOW

Plan → Fix → Test → Deploy → Document

BEST PRACTICES

- ✓ Fix root cause, not symptoms
- ✓ Test before production
- ✓ Use change management
- ✓ Verify dependencies
- ✓ Document everything

TIP

Good remediation is safe, tested, and repeatable.

5 VERIFICATION

Confirm vulnerabilities are fully resolved.

VERIFICATION METHODS

- Rescan & Validate
- Configuration Validation
- Penetration Testing
- Continuous Monitoring
- Compliance Checks

VERIFICATION FLOW

Rescan → Verify Fixed → Validate Controls → Update Risk Score → Close or Reopen

BEST PRACTICES

- ✓ Verify in the same way you scanned
- ✓ Check all affected assets
- ✓ Validate compensating controls
- ✓ Monitor for reoccurrence
- ✓ Keep audit trail

TIP

Verification closes the loop. Never assume it's fixed.

VULNERABILITY MANAGEMENT LIFECYCLE

1. Discovery → 2. Prioritization → 3. Risk Scoring → 4. Remediation → 5. Verification → Continuous Improvement

KEY TAKEAWAY

Vulnerability Management is not a one-time activity.
It is a continuous cycle of visibility, intelligence, action, and verification.

MANAGE VULNERABILITIES. REDUCE RISK. BUILD RESILIENT SYSTEMS.

VERIQTA

SECURITY MONITORING

Security monitoring provides real-time visibility into your environment. It collects, analyzes, and correlates data to detect threats early and help you respond quickly.

MONITOR CONTINUOUSLY.
DETECT EARLY.
RESPOND FASTER.
STAY SECURE.

1 SIEM (SECURITY INFORMATION AND EVENT MANAGEMENT)

Centralize events, detect threats, and support incident response.

WHAT IS SIEM?

SIEM collects, analyzes, correlates, and stores security events from across your environment. It helps you detect threats, investigate incidents, and ensure compliance.

SIEM PROCESS

Collect Events → Analyze Events → Correlate & Detect → Alert & Respond

SIEM CAPABILITIES

- ✓ Real-time monitoring
- ✓ Threat detection
- ✓ Incident investigation
- ✓ Compliance reporting
- ✓ User & entity behavior
- ✓ Analytics & dashboards

TIP

A good SIEM turns raw data into actionable security intelligence.

2 LOG AGGREGATION

Collect logs from many sources in one place.

LOG SOURCES

- Cloud Services
- Servers / Hosts
- Containers
- Kubernetes
- Applications
- Network Devices

LOG AGGREGATION FLOW

Sources → Collect → Store → Search & Analyze

BEST PRACTICES

- ✓ Centralize all logs
- ✓ Use standardized log format
- ✓ Time sync all systems
- ✓ Encrypt log transport
- ✓ Retain logs appropriately
- ✓ Monitor log pipeline health

TIP

If it isn't logged, it didn't happen. Good logs are your evidence.

3 ALERTING

Get notified about important events.

ALERT TYPES

- Critical (Immediate Action)
- High (High Priority)
- Medium (Investigate)
- Low (Informational)

ALERTING FLOW

Event Occurs → Rules & Thresholds → Alert Generated → Notify Team

BEST PRACTICES

- ✓ Alert on suspicious behavior
- ✓ Avoid alert fatigue
- ✓ Use severity & context
- ✓ Route to the right team
- ✓ Test alerts regularly
- ✓ Automate where possible

TIP

Fewer, smarter alerts are better than thousands of noisy ones.

4 CORRELATION

Connect the dots to find real threats.

CORRELATION USES

- ✓ Detect complex attacks
- ✓ Reduce false positives
- ✓ Identify patterns
- ✓ Link related events
- ✓ Improve detection accuracy

CORRELATION EXAMPLE

User Login (Success) → Failed Login (Multiple) → New IP (Unknown) → Privilege Escalation → High Risk Alert

BEST PRACTICES

- ✓ Use correlation rules
- ✓ Leverage threat intel
- ✓ Tune rules regularly
- ✓ Review false positives
- ✓ Combine multiple data sources
- ✓ Align with MITRE ATT&CK

TIP

Correlation turns noise into meaningful security events.

5 DASHBOARDS

Visualize your security posture.

DASHBOARD TYPES

- Executive Summary
- Security Operations
- User Activity
- Infrastructure Health
- Cloud Security

DASHBOARD EXAMPLE

BEST PRACTICES

- ✓ Keep dashboards simple
- ✓ Show what matters
- ✓ Update in real time
- ✓ Use meaningful metrics
- ✓ Enable drill-down
- ✓ Share with the right teams

TIP

A good dashboard answers questions before they become problems.

6 MONITORING ARCHITECTURE (HIGH LEVEL)

DATA SOURCES → COLLECTORS / AGENTS → LOG PIPELINE (Ingest → Normalize → Parse) → STORAGE (Logs / Index) → SIEM / ANALYTICS (Search / Analyze, Correlate / Detect) → OUTPUT (Alerts, Dashboards, Reports)

MONITOR EVERYTHING. DETECT EARLY. RESPOND FAST. STAY SECURE.

VERIQTA

DETECTION ENGINEERING

Detection Engineering is the practice of building, testing, and improving detections to find threats early and reduce risk. Good detections are accurate, relevant, and actionable.

BUILD SMART DETECTIONS.
FIND THREATS EARLY.
REDUCE NOISE.
IMPROVE CONTINUOUSLY.

1 DETECTION RULES

Create rules that identify malicious or suspicious activity.

RULE COMPONENTS

- ✓ Data sources
- ✓ Conditions
- ✓ Logic
- ✓ Thresholds
- ✓ Response
- ✓ Severity
- ✓ Tuning

DETECTION RULE LIFECYCLE

Define Use Case → Build Detection → Test & Validate → Tune Detection → Deploy & Monitor

BEST PRACTICES

- ✓ Start with clear use cases
- ✓ Keep rules simple
- ✓ Avoid overly broad logic
- ✓ Tune for low false positives
- ✓ Document rule purpose
- ✓ Review and update regularly

TIP

A good rule finds the right threats without drowning the team in noise.

2 INDICATORS

Use Indicators of Compromise (IOCs) to detect known threats.

TYPES OF INDICATORS

- IP Addresses
- Domains
- File Hashes
- Email Addresses
- URLs
- Registry Keys

INDICATOR WORKFLOW

Collect Indicators → Validate Quality → Enrich Context → Deploy in Detections → Monitor Effectiveness

BEST PRACTICES

- ✓ Use trusted sources
- ✓ Validate before use
- ✓ Enrich with context
- ✓ Expire outdated indicators
- ✓ Avoid blind trust
- ✓ Track indicator performance

TIP

Context turns indicators into powerful detections.

3 THREAT INTELLIGENCE

Leverage external intelligence to improve detections.

INTELLIGENCE SOURCES

- Open Source Feeds
- Commercial Feeds
- ISAC / Information Sharing
- Reports & Research
- Incident Response Data

THREAT INTELLIGENCE FLOW

Collect Intel → Filter & Normalize → Analyze & Enrich → Apply to Detections → Measure & Improve

BEST PRACTICES

- ✓ Understand your data
- ✓ Focus on relevant threats
- ✓ Enrich with business context
- ✓ Share and collaborate
- ✓ Review intelligence quality
- ✓ Turn intel into detections

TIP

Intelligence without action is just information.

4 CONTINUOUS IMPROVEMENT

Detections must evolve with threats and your environment.

IMPROVEMENT ACTIVITIES

- ✓ Review alerts
- ✓ Triage feedback
- ✓ Identify gaps
- ✓ Update rules
- ✓ Reduce false positives
- ✓ Add new detections

IMPROVEMENT CYCLE

Detect & Alert → Investigate & Learn → Update Rules & Content → Measure Results → (back to Detect & Alert)

BEST PRACTICES

- ✓ Review detections regularly
- ✓ Learn from every alert
- ✓ Track what you change
- ✓ Focus on high value rules
- ✓ Use metrics to guide changes
- ✓ Automate where possible

TIP

Continuous improvement keeps you ahead of attackers.

5 METRICS

Measure detection performance and impact.

KEY METRICS

- Detection Coverage
- True Positives
- False Positives
- Mean Time To Detect (MTTD)
- Mean Time To Respond (MTTR)
- Rule Effectiveness

METRICS DASHBOARD EXAMPLE

Total Alerts (24h)	True Positives	False Positives
12,458 ▲ 8.3%	842 ▲ 5.6%	1,034 ▼ -12.1%
MTTD	**MTTR**	**Rule Effectiveness**
18m ▼ -18%	37m ▼ -10%	87% ▲ 4.9%

BEST PRACTICES

- ✓ Track what matters
- ✓ Use metrics in decisions
- ✓ Identify trends early
- ✓ Set improvement goals
- ✓ Report regularly
- ✓ Share with stakeholders

TIP

If you can't measure it, you can't improve it.

6 DETECTION ENGINEERING FRAMEWORK

A structured approach to building effective detections.

IDENTIFY	COLLECT	CREATE	VALIDATE	DEPLOY	MEASURE	IMPROVE
Understand assets, risks, and threats.	Collect high quality data and logs.	Build detections based on use cases.	Test rules with real data.	Deploy to production and monitor.	Measure performance and impact.	Refine and evolve continuously.

ENGINEER DETECTIONS. REDUCE RISK. PROTECT WHAT MATTERS.

VERIQTA @veriqta

VERIQTA

INCIDENT RESPONSE

Incident Response is the structured approach to detecting, handling, and recovering from security incidents to minimize impact and prevent future attacks.

Goal: Respond quickly. Limit damage. Restore safely. Learn continuously.

PREPARE BEFORE.
RESPOND FAST.
RECOVER STRONGER.
IMPROVE ALWAYS.

PAGE 38

VERIQTA VERIQTA VERIQTA VERIQTA

1 DETECTION

Identify that an incident is happening.

DETECTION SOURCES

- SIEM Alerts
- IDS / IPS
- Endpoint Alerts
- User Reports
- Threat Intel Feeds
- Vulnerability Scanners

DETECTION FLOW

Observe Event → Alert Generated → Triage Alert → Validate Incident → Create Incident

BEST PRACTICES

- ✓ Monitor continuously
- ✓ Tune alerts to reduce noise
- ✓ Validate before escalating
- ✓ Enrich with context
- ✓ Document initial findings

TIP

Not every alert is an incident. Context and validation are critical.

2 CONTAINMENT

Stop the incident from spreading.

CONTAINMENT STRATEGIES

- Isolate affected systems
- Block malicious IPs / Domains
- Disable compromised accounts
- Apply network segmentation
- Revoke tokens / Keys
- Enable additional monitoring

CONTAINMENT FLOW

Identify Scope → Isolate Systems → Block Threats → Monitor Activity → Confirm Containment

BEST PRACTICES

- ✓ Act fast, but don't panic
- ✓ Limit business impact
- ✓ Document every action
- ✓ Communicate internally
- ✓ Preserve forensic evidence

TIP

Containment should be surgical, not disruptive.

3 ERADICATION

Remove the root cause and threats.

ERADICATION ACTIONS

- Remove malware / backdoors
- Delete malicious files
- Close exploited vulnerabilities
- Remove persistence mechanisms
- Reset passwords / Keys
- Update security controls

ERADICATION FLOW

Identify Root Cause → Remove Threats → Fix Vulnerabilities → Clean Systems → Validate Clean State

BEST PRACTICES

- ✓ Thoroughly remove all traces
- ✓ Patch and harden
- ✓ Scan and verify systems
- ✓ Check for lateral movement
- ✓ Update detections

TIP

If you don't remove it completely, it will come back.

4 RECOVERY

Restore normal operations securely.

RECOVERY ACTIVITIES

- Restore systems from backups
- Validate system integrity
- Re-enable accounts / Services
- Gradual service restoration
- Monitor for anomalies
- Confirm business impact resolved

RECOVERY FLOW

Restore Systems → Validate Integrity → Re-enable Access → Restore Services → Monitor Stability

BEST PRACTICES

- ✓ Verify before restoring access
- ✓ Test critical functions
- ✓ Monitor closely
- ✓ Communicate with stakeholders
- ✓ Document recovery steps

TIP

Recovery is not complete until you confirm stability.

5 LESSONS LEARNED

Learn and improve to prevent future incidents.

LESSONS LEARNED ACTIVITIES

- Conduct post-incident review
- Identify root cause
- Evaluate response effectiveness
- Update playbooks and SOPs
- Improve detections and controls
- Share knowledge with team

IMPROVEMENT CYCLE

Test & Validate → Identify Gaps → Update Processes → Improve Controls → CONTINUOUS IMPROVEMENT

BEST PRACTICES

- ✓ No blame culture
- ✓ Focus on improving
- ✓ Track action items
- ✓ Measure improvements
- ✓ Repeat and evolve

TIP

Every incident is a lesson. Every lesson makes you stronger.

INCIDENT RESPONSE IS A CYCLE:

DETECTION → CONTAINMENT → ERADICATION → RECOVERY → LESSONS LEARNED

KEY TAKEAWAY

Prepare, detect, respond, recover, and learn. Speed saves you. Process protects you. Improvement makes you stronger.

VERIQTA

COMPLIANCE AND GOVERNANCE

PAGE 39

Compliance ensures you follow regulations.
Governance ensures you do the right things, the right way.
Together, they build trust, reduce risk, and protect your business.

STRONG GOVERNANCE.
CONSISTENT COMPLIANCE.
LOWER RISK.
GREATER TRUST.

1 ISO 27001

International standard for Information Security Management Systems (ISMS).

WHAT IS ISO 27001?

ISO

Provides a framework to manage information security risks and protect data confidentiality, integrity, and availability.

KEY FOCUS AREAS

- Information Security Policies
- Risk Assessment & Treatment
- Security Controls
- Awareness & Training
- Monitoring & Continuous Improvement

BEST PRACTICES

- ✓ Establish an ISMS
- ✓ Conduct regular risk assessments
- ✓ Implement strong controls
- ✓ Review and audit regularly
- ✓ Drive continuous improvement

TIP

ISO 27001 is about building a culture of information security.

2 SOC 2

Service Organization Control 2.

WHAT IS SOC 2?

SOC 2

Assesses how well a service organization protects data based on five Trust Service Criteria (TSC).

TRUST SERVICE CRITERIA

- Security
- Availability
- Processing Integrity
- Confidentiality
- Privacy

BEST PRACTICES

- ✓ Map controls to TSC
- ✓ Implement strong access controls
- ✓ Monitor systems continuously
- ✓ Maintain accurate documentation
- ✓ Undergo regular SOC 2 audits

TIP

SOC 2 builds customer trust and proves operational maturity.

3 PCI DSS

Payment Card Industry Data Security Standard.

WHAT IS PCI DSS?

A set of security requirements for organizations that store, process, or transmit cardholder data.

CORE REQUIREMENTS (12)

- Build & Maintain Secure Network
- Protect Cardholder Data
- Maintain Vulnerability Management
- Implement Strong Access Control
- Monitor & Test Networks

BEST PRACTICES

- ✓ Keep systems and apps secure
- ✓ Encrypt cardholder data
- ✓ Use strong access controls
- ✓ Monitor and test regularly
- ✓ Maintain security policies

TIP

PCI DSS is mandatory for anyone handling payment card data.

4 NIST

National Institute of Standards and Technology Frameworks.

WHAT IS NIST?

NIST

Provides frameworks and guidelines to manage and reduce cybersecurity risk.

NIST CYBERSECURITY FRAMEWORK (CSF)

- Identify (Understand Risk)
- → Protect (Implement Safeguards)
- → Detect (Find Threats)
- → Respond (Take Action)
- → Recover (Restore Operations)

BEST PRACTICES

- ✓ Use NIST CSF as a roadmap
- ✓ Align with business objectives
- ✓ Integrate into risk management
- ✓ Measure and improve maturity
- ✓ Share threat intelligence

TIP

NIST CSF is flexible and works for organizations of any size.

5 CIS BENCHMARKS

Center for Internet Security Benchmarks.

WHAT ARE CIS BENCHMARKS?

CIS.

Configuration guidelines to secure systems, networks, and applications.

BENCHMARK AREAS

- Operating Systems
- Cloud Platforms
- Containers
- Network Devices
- Applications

BEST PRACTICES

- ✓ Use benchmarks for hardening
- ✓ Automate compliance checks
- ✓ Monitor configuration drift
- ✓ Prioritize critical controls
- ✓ Update benchmarks regularly

TIP

CIS Benchmarks provide practical, actionable security configurations.

COMPLIANCE VS GOVERNANCE

COMPLIANCE

- ✓ Follow rules and standards
- ✓ Often required by law
- ✓ Proves you are following external requirements

GOVERNANCE

- ✓ Set policies and direction
- ✓ Driven by leadership
- ✓ Ensures you do the right things

KEY TAKEAWAY

Use these frameworks and standards together.
Compliaace keeps you secure and legal.
Governance keeps you aligned and resilient.
Review, measure, and improve continuously.

STRONG GOVERNANCE. CONSISTENT COMPLIANCE. RESILIENT BUSINESS.

VERIQTA @veriqta

VERIQTA

POLICY AS CODE

Policy as Code turns security and compliance rules into version-controlled, testable, and enforceable code. It ensures consistent governance, reduces risk, and enables continuous compliance at enterprise scale.

WRITE POLICIES.
AUTOMATE GOVERNANCE.
ENFORCE CONSISTENTLY.
STAY COMPLIANT.

PAGE 40

1 WHY POLICIES MATTER

Policies define what is allowed, required, and forbidden.

KEY REASONS

- ✓ Ensure security and compliance
- ✓ Reduce human error
- ✓ Provide consistency
- ✓ Support audits and reporting
- ✓ Enable scale and automation
- ✓ Reduce risk and cost

POLICY EXAMPLES

Access Control · Data Protection · Resource Configuration · Secrets Management · Logging & Monitoring

BEST PRACTICES

- ✓ Design clear and simple policies
- ✓ Align with business objectives
- ✓ Review policies regularly
- ✓ Communicate policies clearly
- ✓ Version and track all policies

TIP

A good policy prevents issues before they become incidents.

2 GOVERNANCE AUTOMATION

Automate the creation, distribution, and enforcement of policies.

AUTOMATION TARGETS

- ✓ Policy creation
- ✓ Policy distribution
- ✓ Policy enforcement
- ✓ Drift detection
- ✓ Remediation
- ✓ Reporting

GOVERNANCE AUTOMATION FLOW

Define Policies → Store in Repository → Run Policy Pipeline → Enforce Policies → Monitor & Report

BEST PRACTICES

- ✓ Automate as much as possible
- ✓ Use version control
- ✓ Implement guardrails
- ✓ Monitor policy effectiveness
- ✓ Automate remediation

TIP

Automation ensures policies are applied consistently every time.

3 VALIDATION

Validate policies before enforcing them.

VALIDATION TYPES

- ✓ Syntax validation
- ✓ Logical validation
- ✓ Security validation
- ✓ Compliance validation
- ✓ Simulation / Plan validation
- ✓ Test against real resources

VALIDATION FLOW

Write Policy → Validate Syntax → Validate Logic → Test Compliance → Approve Policy

BEST PRACTICES

- ✓ Validate in CI/CD pipeline
- ✓ Fail fast on validation errors
- ✓ Use policy unit tests
- ✓ Test with real scenarios
- ✓ Document validation rules

TIP

Validate often. Bad policies cause more damage than no policies.

4 CONTINUOUS COMPLIANCE

Ensure policies remain effective over time.

CONTINUOUS COMPLIANCE PILLARS

- ✓ Continuous assessment
- ✓ Continuous monitoring
- ✓ Drift detection
- ✓ Continuous remediation
- ✓ Continuous reporting
- ✓ Continuous improvement

CONTINUOUS COMPLIANCE FLOW

Assess Resources → Check Policy Compliance → Detect Drift → Remediate Issues → Report Results

BEST PRACTICES

- ✓ Run compliance checks regularly
- ✓ Monitor for configuration drift
- ✓ Auto-remediate safe issues
- ✓ Escalate critical violations
- ✓ Measure compliance scores

TIP

Compliance is not a one-time activity. It is continuous.

5 ENTERPRISE WORKFLOWS

Integrate policies into enterprise systems.

INTEGRATION POINTS

- ✓ CI/CD pipelines
- ✓ Infrastructure as Code
- ✓ Cloud accounts
- ✓ Kubernetes clusters
- ✓ Identity systems
- ✓ Service catalog

ENTERPRISE WORKFLOW EXAMPLE

Developer Push Code → Pipeline Runs → Policy Check → Pass Deploy / Fail Block Deploy & Alert

BEST PRACTICES

- ✓ Embed policies in workflows
- ✓ Fail builds on violations
- ✓ Provide clear feedback
- ✓ Enable self-service safely
- ✓ Measure and improve workflow

TIP

Make the right path the easy path.

POLICY AS CODE LIFECYCLE

Define Policy → Version Control → Validate Policy → Enforce Policy → Monitor Compliance → Improve Policy

KEY TAKEAWAY

Policy as Code brings consistency, automation, and scale to governance. It reduces risk, ensures compliance, and enables secure, fast, and reliable delivery at enterprise scale.

RIGHT POLICIES. AUTOMATED GOVERNANCE. CONTINUOUS COMPLIANCE. LESS RISK.

VERIQTA

DEVSECOPS METRICS

Metrics turn security from a checkbox into a measurable, improvable, and accountable part of delivery.
Measure what matters. Improve what you measure.

MEASURE SECURITY.
IMPROVE CONTINUOUSLY.
DELIVER CONFIDENTLY.
REDUCE RISK.

PAGE 41

1 MTTR (MEAN TIME TO RESPOND & RECOVER)

Measures how quickly your team responds to and recovers from security incidents.

WHAT IT MEASURES

- ✓ Incident response speed
- ✓ Containment efficiency
- ✓ Recovery effectiveness
- ✓ Overall security resilience

HOW TO CALCULATE

Incident Detected → Incident Resolved = MTTR (Average)

MTTR = Total Time to Resolve All Incidents / Number of Incidents

BEST PRACTICES

- ✓ Automate detection & alerting
- ✓ Triage and prioritize fast
- ✓ Document and refine runbooks
- ✓ Run regular incident drills
- ✓ Review and improve processes

TIP

Lower MTTR means lower impact and stronger resilience.

2 VULNERABILITY AGE

Measures how long vulnerabilities stay open before they are fixed.

WHAT IT MEASURES

- ✓ Exposure duration
- ✓ Patch efficiency
- ✓ Risk accumulation
- ✓ Remediation speed

HOW TO CALCULATE

Date Discovered → Date Resolved = Vulnerability Age (Days)

Average or 90th percentile age of open vulnerabilities

BEST PRACTICES

- ✓ Prioritize based on risk
- ✓ Fix critical issues first
- ✓ Automate patching
- ✓ Track by asset and severity
- ✓ Review and reduce backlog

TIP

Old vulnerabilities are the biggest risk to your organization.

3 SECURITY COVERAGE

Measures how much of your environment and pipeline is protected by security controls.

WHAT IT MEASURES

- ✓ Tool and control coverage
- ✓ Protected assets
- ✓ Pipeline coverage
- ✓ Visibility completeness

HOW TO CALCULATE

$$\frac{\text{Protected Assets / Pipeline Components}}{\text{Total Assets / Pipeline Components}} \times 100 = \text{Security Coverage \%}$$

Higher coverage = better visibility and protection

BEST PRACTICES

- ✓ Map assets and pipelines
- ✓ Deploy layered controls
- ✓ Eliminate blind spots
- ✓ Track coverage by environment
- ✓ Continuously improve coverage

TIP

You can't protect what you cannot see. Increase your coverage.

4 COMPLIANCE SCORE

Measures how well you meet internal and external compliance requirements.

WHAT IT MEASURES

- ✓ Control adherence
- ✓ Audit readiness
- ✓ Policy compliance
- ✓ Regulatory alignment

HOW TO CALCULATE

$$\frac{\text{Controls Compliant}}{\text{Total Applicable Controls}} \times 100 = \text{Compliance Score \%}$$

Weight controls based on risk and criticality

BEST PRACTICES

- ✓ Align with frameworks (NIST, ISO)
- ✓ Automate control checks
- ✓ Monitor continuously
- ✓ Remediate gaps quickly
- ✓ Report and review regularly

TIP

High compliance score builds trust and avoids penalties.

5 PIPELINE HEALTH

Measures the security and reliability of your CI/CD pipeline.

WHAT IT MEASURES

- ✓ Build success rate
- ✓ Security test pass rate
- ✓ Deployment frequency
- ✓ Pipeline stability

HOW TO CALCULATE

Build Success + Security Tests Passed + Deploy Success + Lead Time (Efficiency) = Pipeline Health Score

Combine reliability, security, and speed indicators

BEST PRACTICES

- ✓ Shift security left
- ✓ Automate quality gates
- ✓ Reduce flaky tests
- ✓ Monitor DORA metrics
- ✓ Continuously optimize pipeline

TIP

A healthy pipeline delivers secure software faster.

DEVSECOPS METRICS DASHBOARD EXAMPLE

MTTR	VULNERABILITY AGE	SECURITY COVERAGE	COMPLIANCE SCORE	PIPELINE HEALTH
1.6h	12.4 days	87%	92%	96%
↓ 18%	↓ 22%	↑ 9%	↑ 5%	↑ 7%

KEY TAKEAWAY

Great DevSecOps teams don't just build fast. They measure security, improve constantly, and deliver with confidence.
Measure. Improve. Secure. Repeat.

MEASURE SECURITY. IMPROVE CONTINUOUSLY. DELIVER SECURELY.

VERIQTA @veriqta

VERIQTA

SECURE LOGGING

Secure logging provides visibility, supports incident response, meets compliance requirements, and protects your organization. Good logs tell the truth. Secure logs protect that truth.

LOG WHAT MATTERS.
PROTECT SENSITIVE DATA.
ENSURE INTEGRITY.
RETAIN RESPONSIBLY.

PAGE 42

1 STRUCTURED LOGS

Use structured logs for consistency, searchability, and automation.

KEY BENEFITS

- ✓ Consistent format
- ✓ Easy to parse
- ✓ Better searchability
- ✓ Improved correlation
- ✓ Automation friendly

STRUCTURED LOG EXAMPLE (JSON)

```
{
  "timestamp": "2025-07-25T10:30:45Z",
  "level": "INFO",
  "service": "auth-service",
  "event": "user_login_success",
  "user_id": "u12345",
  "ip": "203.0.113.10",
  "status": "success",
  "message": "User logged in successfully"
}
```

BEST PRACTICES

- ✓ Use JSON or another structured format
- ✓ Include timestamp in UTC
- ✓ Use consistent field names
- ✓ Log meaningful context
- ✓ Avoid free-text logging
- ✓ Validate log schema

TIP

Structured logs turn chaos into clarity. Your future self will thank you.

2 AUDIT LOGS

Record who did what, when, where, and why.

WHAT TO LOG

- ✓ User activities
- ✓ Admin actions
- ✓ Configuration changes
- ✓ Access attempts
- ✓ Data access events
- ✓ Policy changes

AUDIT LOG EXAMPLE

Time (UTC)	User	Action	Resource	Result	Source IP
2025-07-25 10:30:45	admin	CreateUser	user: john	Success	203.0.113.10
2025-07-25 10:31:12	admin	UpdatePolicy	policy: p123	Success	203.0.113.10
2025-07-25 10:32:05	john	DeleteFile	file: report.pdf	Failure	198.51.100.23

BEST PRACTICES

- ✓ Enable audit logging everywhere
- ✓ Log both success and failure
- ✓ Include user identity
- ✓ Include source IP
- ✓ Protect audit logs from alteration

TIP

Audit logs are your first line of accountability.

3 SENSITIVE DATA

Protect sensitive information from exposure in logs.

SENSITIVE DATA EXAMPLES

- ✓ Passwords
- ✓ API Keys / Tokens
- ✓ Credit card numbers
- ✓ Personally identifiable info (PII)
- ✓ Session IDs
- ✓ Internal system details

HOW TO PROTECT

Identify Sensitive Data → Mask or Redact → Encrypt Before Logging → Review Regularly

BEST PRACTICES

- ✓ Never log secrets or credentials
- ✓ Mask or redact sensitive fields
- ✓ Use tokenization when possible
- ✓ Encrypt logs in transit and at rest
- ✓ Review logs for accidental leaks

TIP

The best log is the one that doesn't expose secrets.

4 LOG RETENTION

Retain logs for the right amount of time — not too short, not too long.

RETENTION CONSIDERATIONS

- ✓ Legal and regulatory requirements
- ✓ Incident investigation needs
- ✓ Business and compliance needs
- ✓ Storage cost
- ✓ Data lifecycle policies

RETENTION STRATEGY EXAMPLE

Hot Storage
0 – 30 days
(Fast Access)

→ Warm Storage
31 – 180 days
(Cost Optimized)

→ Cold/Archive
181+ days
(Long-term)

BEST PRACTICES

- ✓ Define retention per log type
- ✓ Automate lifecycle policies
- ✓ Balance cost and compliance
- ✓ Document retention policies
- ✓ Review and adjust regularly

TIP

Keep what you must. Archive what you should. Delete what you don't.

5 LOG INTEGRITY

Ensure logs cannot be tampered with or deleted by attackers.

INTEGRITY CONTROLS

- ✓ Write-once storage
- ✓ Immutable logs
- ✓ Digital signatures / Hashing
- ✓ Least privilege access
- ✓ Centralized logging

LOG INTEGRITY FLOW

Generate Log → Create Hash → Store in Immutable Store → Verify Integrity

BEST PRACTICES

- ✓ Use immutable storage (WORM)
- ✓ Autable object lock if available
- ✓ Sign logs or use chepliaurs
- ✓ Restrict access to log systems
- ✓ Monitor for tampering attempts

TIP

If logs can be changed, they cannot be trusted.

SECURE LOGGING CHECKLIST

Structured Format → Audit Events → Protect Sensitive Data → Define Retention → Ensure Integrity → Monitor & Review

KEY TAKEAWAY

Secure logging is not just about collecting logs. It's about collecting the right logs, protecting them, and using them to improve security continuously.

GOOD LOGS TELL YOU WHAT HAPPENED. SECURE LOGS PROVE IT.

VERIQTA

OBSERVABILITY FOR SECURITY

You cannot secure what you cannot see.
Observability for security gives you the visibility, context, and insight to detect, investigate, and respond to threats faster and with confidence.

SEE EVERYTHING.
UNDERSTAND EVERYTHING.
SECURE EVERYTHING.
OBSERVE. DETECT. RESPOND.

PAGE 43

1 METRICS

Metrics show you the health, behavior, and risk of your systems.

KEY METRICS

- ✓ Failed login rate
- ✓ Privilege escalation count
- ✓ Anomalous process count
- ✓ Vulnerability exposure
- ✓ Patch compliance rate
- ✓ Security control failures

METRICS EXAMPLES

Failed Logins	Auth Failures	High Risk Alerts	Unpatched Assets
153	87	42	19
↑ 23%	↑ 18%	↑ 35%	↓ 12%

BEST PRACTICES

- ✓ Define meaningful metrics
- ✓ Set baselines and thresholds
- ✓ Use rate, not just count
- ✓ Alert on behavior changes
- ✓ Correlate across signals
- ✓ Review and refine regularly

TIP

Metrics without context are just numbers. Contect turns numbers into meaning.

2 LOGS

Logs provide the details of what happened.

KEY LOG SOURCES

- ✓ Authentication logs
- ✓ System and application logs
- ✓ Network and firewall logs
- ✓ Cloud activity logs
- ✓ Endpoint and EDR logs
- ✓ Audit and admin logs

LOG EXAMPLE (JSON)

```
{
  "timestamp": "2025-07-25T10:30:45Z",
  "event": "user_login_failed",
  "user": "admin",
  "source_ip": "203.0.113.10",
  "location": "New York, US",
  "user_agent": "Mozilla/5.0 ...",
  "result": "failure",
  "reason": "invalid_password"
}
```

BEST PRACTICES

- ✓ Centralize logs
- ✓ Use structured logging
- ✓ Normalize and parse logs
- ✓ Keep time in UTC
- ✓ Protect logs from tampering
- ✓ Retain logs per policy

TIP

Good logs answer the 5 W's: Who, What, When, Where, Why.

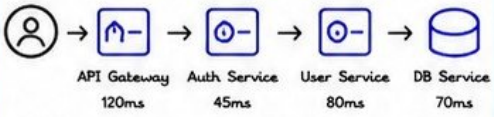
3 TRACES

Traces show the full journey of a request across systems.

KEY BENEFITS

- ✓ End-to-end visibility
- ✓ Latency and bottlenecks
- ✓ Dependency mapping
- ✓ Error and failure tracking
- ✓ Root cause analysis
- ✓ Attack path visibility

TRACE EXAMPLE



Total Time: 315ms

BEST PRACTICES

- ✓ Instrument all services
- ✓ Propagate trace context
- ✓ Sample intelligently
- ✓ Correlate traces with logs
- ✓ Monitor latency patterns
- ✓ Detect abnormal flows

TIP

Traces reveal how attacks move through your systems. Follow the path.

4 SECURITY TELEMETRY

Telemetry is the raw data that powers detection and response.

TELEMETRY SOURCES

- ✓ Cloud and infrastructure
- ✓ Endpoints and servers
- ✓ Network and firewalls
- ✓ Applications and APIs
- ✓ Identity and access
- ✓ Third-party integrations

TELEMETRY EXAMPLES

Cloud Events · Endpoint Events · Network Flows · Identity Events · App / API Events

BEST PRACTICES

- ✓ Collect high-quality data
- ✓ Normalize telemetry
- ✓ Enrich with context
- ✓ Reduce noise early
- ✓ Ensure data integrity
- ✓ Monitor data pipelines

TIP

Garbage in, garbage out. Good telemetry builds strong defenses.

5 INVESTIGATION

Observability data helps you investigate faster and smarter.

INVESTIGATION ESSENTIALS

- ✓ Correlate across signals
- ✓ Build timelines
- ✓ Ask the right questions
- ✓ Validate hypotheses
- ✓ Document findings
- ✓ Share and escalate

INVESTIGATION WORKFLOW

Detect Alert → Collect Data → Build Timeline → Analyze & Correlate → Respond & Document

BEST PRACTICES

- ✓ Start broad, then narrow
- ✓ Use timelines
- ✓ Look for anomalies
- ✓ Verify before escalating
- ✓ Document everything
- ✓ Improve after every case

TIP

The faster you investigate, the smaller the damage.

OBSERVABILITY PILLARS

METRICS (What is happening?) → LOGS (What happened?) → TRACES (How did it happen?) → TELEMETRY (What is the data?) → INVESTIGATION (What will we do?)

COMPLETE VISIBILITY. FASTER RESPONSE. STRONGER SECURITY.

KEY TAKEAWAY

Observability for security connects metrics, logs, traces, and telemetry to give you complete visibility. With the right visibility, you detect faster, investigate smarter, and respond stronger.

OBSERVE CONTINUOUSLY. DETECT EARLY. RESPOND DECISIVELY.

VERIQTA @veriqta

VERIQTA

PRODUCTION SECURITY ARCHITECTURE

Security architecture is about making the right thing easy, the wrong thing hard, and attacks expensive.
Design for resilience, visibility, and continuous protection.

BUILD SECURE BY DESIGN. OPERATE SECURE IN PRODUCTION.

PAGE 44

VERIQTA VERIQTA VERIQTA

1 SECURE PLATFORM DESIGN

Build a secure, scalable, and observable platform.

DESIGN PRINCIPLES

- ✓ Security by Design
- ✓ Least Privilege
- ✓ Automate Everything
- ✓ Immutable Infrastructure
- ✓ Continuous Monitoring
- ✓ Resilient by Default

SECURE PLATFORM LAYERS

- APPLICATION LAYER (Workloads, APIs, Services)
- PLATFORM LAYER (Kubernetes, Runtimes, Middleware)
- INFRASTRUCTURE LAYER (Compute, Storage, Network)
- FOUNDATION LAYER (Cloud Accounts, Regions, AZs)

IDENTITY & ACCESS (IAM, RBAC, SSO)

BEST PRACTICES

- ✓ Secure defaults for all services
- ✓ Harden images and runtimes
- ✓ Encrypt data everywhere
- ✓ Centralize logging and monitoring
- ✓ Automate security guardrails
- ✓ Continuously validate configuration
- ✓ Test for failure and recovery

TIP

A secure platform reduces risk for every team that builds on it.

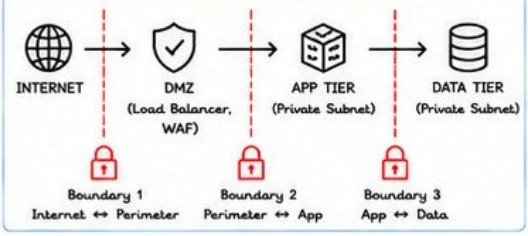
2 TRUST BOUNDARIES

Define clear trust boundaries and control every crossing.

COMMON BOUNDARIES

- Internet → Perimeter
- Perimeter → DMZ
- DMZ → Internal Network
- User → Application
- Service → Service
- Workload → Data

EXAMPLE: 3-TIER ARCHITECTURE



BEST PRACTICES

- ✓ Minimize the number of boundaries
- ✓ Enforce least privilege access
- ✓ Use network segmentation
- ✓ Inspect and log all traffic
- ✓ Control egress traffic
- ✓ Validate identity at every boundary

TIP

Every boundary is a decision point. Make every decision intentional.

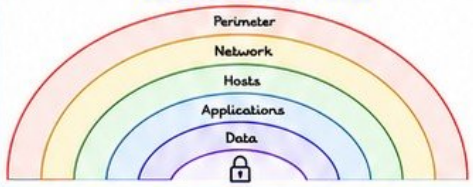
3 DEFENSE IN DEPTH

Use multiple layers of controls so failure in one layer does not mean compromise.

LAYERS OF DEFENSE

- ✓ Physical Security
- ✓ Network Security
- ✓ Host Security
- ✓ Application Security
- ✓ Data Security
- ✓ Identity Security
- ✓ Operational Security

DEFENSE IN DEPTH MODEL



BEST PRACTICES

- ✓ Layer controls across the stack
- ✓ Use overlapping controls
- ✓ Detect and respond at each layer
- ✓ Assume one layer will trafic
- ✓ Continuously test each layer

TIP

Attackers look for gaps. You build layers.

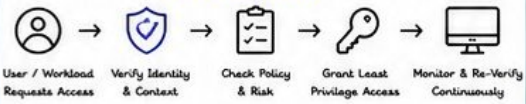
4 ZERO TRUST

Never trust, always verify.

ZERO TRUST PRINCIPLES

- ✓ Verify explicitly
- ✓ Use least privilege access
- ✓ Assume breach
- ✓ Continuously validate
- ✓ Secure every resource

ZERO TRUST FLOW



BEST PRACTICES

- ✓ Use MFA everywhere
- ✓ Enforce device trust
- ✓ Micro-segment workloads
- ✓ Use short-lived credentials
- ✓ Continuously monitor behavior

TIP

Zero Trust is not a product. It is a security model.

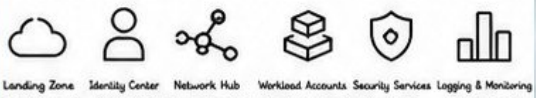
5 ENTERPRISE ARCHITECTURE

Design security at enterprise scale.

ENTERPRISE PILLARS

- ✓ Governance & Policy
- ✓ Identity & Access
- ✓ Secure Platform
- ✓ Data Protection
- ✓ Monitoring & Response
- ✓ Resilience & Recovery

ENTERPRISE REFERENCE ARCHITECTURE



BEST PRACTICES

- ✓ Use multi-account strategy
- ✓ Centralize logging and security
- ✓ Standardize guardrails
- ✓ Automate governance
- ✓ Continuously measure posture

TIP

Enterprise scale requires standardization and automation.

SECURE ARCHITECTURE CHECKLIST

- ✓ Clear trust boundaries defined
- ✓ Least privilege enforced
- ✓ Defense in depth implemented
- ✓ Zero Trust controls active
- ✓ Security automated
- ✓ Monitoring and logging in place

KEY TAKEAWAY

A strong production security architecture is intentional. It is designed, layered, automated, and continuously improved. Secure the platform. Protect the workloads. Enable the business.

STRONG ARCHITECTURE. STRONGER DEFENSE. RESILIENT BY DESIGN.

VERIQTA

THREAT DETECTION IN KUBERNETES

Kubernetes is powerful, but it's also a target.
Threat detection helps you identify suspicious behavior early, investigate with confidence, and respond before damage spreads.

DETECT EARLY.
INVESTIGATE SMART.
RESPOND FAST.
STAY AHEAD OF THREATS.

PAGE 45

1 SUSPICIOUS ACTIVITY

Identify behaviors that deviate from normal patterns.

EXAMPLES

- ✓ Unexpected Pod creation
- ✓ Privileged container usage
- ✓ Access to sensitive resources
- ✓ Unusual outbound connections
- ✓ Mass deletion of resources
- ✓ Escalation of privileges

SUSPICIOUS ACTIVITY EXAMPLES

New Pod in sensitive namespace → Privileged Container Detected → Access to Secrets or ConfigMaps → Unusual Outbound Traffic → Mass Delete Events

Baseline behavior. Detect deviation. Investigate immediately.

BEST PRACTICES

- ✓ Define normal baseline behavior
- ✓ Use real-time alerting
- ✓ Correlate multiple signals
- ✓ Reduce false positives
- ✓ Continuously tune detections
- ✓ Automate response where safe

TIP

Not all alerts are attacks. Focus on behavior, not just events.

2 AUDIT LOGS

Audit logs provide the who, what, when, where, and how.

KEY AUDIT EVENTS

- ✓ API server requests
- ✓ Authentication events
- ✓ Authorization failures
- ✓ Role and permission changes
- ✓ Resource modifications
- ✓ Secret and ConfigMap access

AUDIT LOG FLOW

User / Service Account → Kubernetes API Server → Audit Log Events → Log Storage (Elastic, Loki, Cloud Storage) → Analysis & Alerting

Enable audit logging. Store securely. Review regularly.

BEST PRACTICES

- ✓ Enable audit logging
- ✓ Log all requests
- ✓ Store logs immutably
- ✓ Encrypt audit logs
- ✓ Review logs daily
- ✓ Retain logs as per policy

TIP

Audit logs are your source of truth during investigation.

3 RUNTIME EVENTS

Runtime events show what is happening inside the cluster now.

RUNTIME EVENT SOURCES

- ✓ Pod lifecycle events
- ✓ Container process events
- ✓ File integrity changes
- ✓ Network flow events
- ✓ Syscalls and behavior
- ✓ Admission controller logs

RUNTIME MONITORING STACK

Nodes → Agents (Falco, eBPF, OSQuery) → Event Stream (Realtime) → SIEM / SOAR (Detection & Correlation) → Alerts

Collect events in real time. Correlate. Detect behavior.

BEST PRACTICES

- ✓ Use runtime security tools
- ✓ Monitor system calls
- ✓ Detect container escapes
- ✓ Track file and process changes
- ✓ Alert on anomaly
- ✓ Integrate with SIEM

TIP

Runtime events catch attacks that never touch logs.

4 INVESTIGATION

Investigate quickly and thoroughly to understand the full picture.

INVESTIGATION CHECKLIST

- ✓ What happened?
- ✓ When did it happen?
- ✓ Who or what triggered it?
- ✓ What is the impact?
- ✓ How did it spread?
- ✓ What data was accessed?

INVESTIGATION WORKFLOW

Alert Received → Triage & Validate → Collect Evidence → Analyze Timeline → Determine Impact → Document Findings

Follow a process. Don't guess. Evidence over assumptions.

BEST PRACTICES

- ✓ Use timelines
- ✓ Collect all relevant logs
- ✓ Preserve evidence
- ✓ Avoid changing the scene
- ✓ Communicate findings
- ✓ Document everything

TIP

Good investigation shortens response time next time.

5 RESPONSE

Respond fast to contain the threat and recover safely.

RESPONSE ACTIONS

- ✓ Contain the threat
- ✓ Block malicious activity
- ✓ Terminate malicious Pods
- ✓ Revoke access / tokens
- ✓ Patch and harden
- ✓ Recover and validate

RESPONSE LIFECYCLE

Detect → Contain → Eradicate → Recover → Lessons Learned

Contain. Eradicate. Recover. Improve.

BEST PRACTICES

- ✓ Have a runbook
- ✓ Automate common actions
- ✓ Verify before restoring
- ✓ Patch root cause
- ✓ Run post-incident review
- ✓ Improve detections

TIP

A good response stops today's attack and prevents tomorrow's.

THREAT DETECTION FLOW

Monitor (Collect Data) → Detect (Identify Threat) → Investigate (Understand) → Respond (Act Fast) → Improve (Strengthen)

KEY TAKEAWAY

Threat detection in Kubernetes is a cycle of visibility, detection, investigation, and response. The faster you detect, the smaller the damage. The more you investigate, the smarter your response. The more you learn, the stronger your cluster becomes.

DETECT EARLY. INVESTIGATE SMART. RESPOND FAST. STAY SECURE.

VERIQTA @veriqta

VERIQTA

SUPPLY CHAIN SECURITY

Your software supply chain is only as strong as its weakest link.
Secure every step from code to production.
Verify everything. Trust intentionally. Protect continuously.

SECURE THE CHAIN.
TRUST WITH VERIFICATION.
PROTECT EVERY ARTIFACT.
DELIVER WITH CONFIDENCE.

1 BUILD SECURITY

Secure the build process and the environment that creates your artifacts.

KEY CONTROLS

- ✓ Harden build environment
- ✓ Least privilege for builds
- ✓ Secure CI/CD pipelines
- ✓ Secrets management
- ✓ Static & dynamic analysis
- ✓ Build isolation

SECURE BUILD PIPELINE

Code → Commit & Scan → Build in Isolated Environment → Test & SAST/DAST → Sign Artifact

Build only what you need. Scan everything. Sign always.

BEST PRACTICES

- ✓ Use ephemeral build agents
- ✓ Keep tools and images updated
- ✓ Scan code and dependencies
- ✓ Fail build on high risk
- ✓ Store logs and SBOMs
- ✓ Review and approve changes

TIP

A compromised build can inject risk into every deployment.

2 DEPENDENCY TRUST

You are only as safe as the dependencies you use.

KEY CONTROLS

- ✓ Use trusted sources
- ✓ Pin dependency versions
- ✓ Vulnerability scanning
- ✓ License compliance
- ✓ Regular updates
- ✓ Remove unused deps

DEPENDENCY TRUST FLOW

Package Source → Verify Source → Scan for Vulnerabilities → Pin Version → Allow in Build

Trust, but verify. Pin, scan, and review everything.

BEST PRACTICES

- ✓ Use private package registries
- ✓ Enable dependency proxy/cache
- ✓ Review licenses and risks
- ✓ Automate vulnerability alerts
- ✓ Remove unused dependencies
- ✓ Document approved sources

TIP

Untrusted dependency is one command away from compromise.

3 ARTIFACT VALIDATION

Ensure artifacts are authentic, untampered, and trusted.

KEY CONTROLS

- ✓ Artifact signing
- ✓ Checksum verification
- ✓ Integrity scanning
- ✓ Policy enforcement
- ✓ Immutable storage
- ✓ Quarantine failures

ARTIFACT VALIDATION FLOW

Build Artifact → Verify Signature → Verify Checksum → Scan Artifact (Security) → Store in Registry

Verify before you trust. Never deploy unverified artifacts.

BEST PRACTICES

- ✓ Sign all artifacts
- ✓ Verify signatures in pipeline
- ✓ Scan images and binaries
- ✓ Use immutable artifact storage
- ✓ Block unsigned or unknown
- ✓ Enforce repository policies

TIP

If you don't verify, an attacker can deliver the payload.

4 PROVENANCE

Know where your artifacts came from and how they were built.

KEY CONTROLS

- ✓ Generate SBOM
- ✓ Capture build metadata
- ✓ Record sources & steps
- ✓ Developer identity
- ✓ Environment attestation
- ✓ Provenance attestation

PROVENANCE FLOW

Source Code → Build Environment → Build Steps → SBOM & Metadata → Signed Provenance

Provenance creates traceability. Traceability creates trust.

BEST PRACTICES

- ✓ Generate SBOM for every build
- ✓ Use SLSA / in-toto / DSSE
- ✓ Store provenance with artifacts
- ✓ Verify builder identity
- ✓ Attest environment and steps
- ✓ Make provenance tamper-proof

TIP

Without provenance, you can't prove what you built.

5 END-TO-END INTEGRITY

Protect the entire supply chain from source to runtime.

KEY CONTROLS

- ✓ Secure transport
- ✓ Access control
- ✓ Policy enforcement
- ✓ Continuous monitoring
- ✓ Drift detection
- ✓ Runtime validation

END-TO-END SUPPLY CHAIN

Plan → Code → Build → Test → Sign → Distribute → Deploy → Monitor

Every step secured. Every handoff verified. Every change monitored.

BEST PRACTICES

- ✓ Enforce policies at every stage
- ✓ Use least privilege everywhere
- ✓ Monitor for anomalies
- ✓ Verify at deploy and runtime
- ✓ Respond to risks immediately
- ✓ Continuously improve the chain

TIP

Security is not a step. It's the entire journey.

SUPPLY CHAIN SECURITY CHECKLIST

- ✓ Secure build environment
- ✓ Scan code and dependencies
- ✓ Sign all artifacts
- ✓ Verify checksums and signatures
- ✓ Generate SBOMs and provenance
- ✓ Use trusted and pinned dependencies
- ✓ Enforce policies in pipeline
- ✓ Store artifacts immutably
- ✓ Monitor and detect anomalies
- ✓ Review and improve continuously

KEY TAKEAWAY

A secure supply chain is built on verification, trust, and visibility. Secure every step, verify every artifact, and maintain end-to-end integrity. Trust is earned. Security is proven.

SECURE EVERY STEP. VERIFY EVERY HANDOFF. PROTECT EVERY DELIVERY.

VERIQTA

ENTERPRISE DEVSECOPS PIPELINE

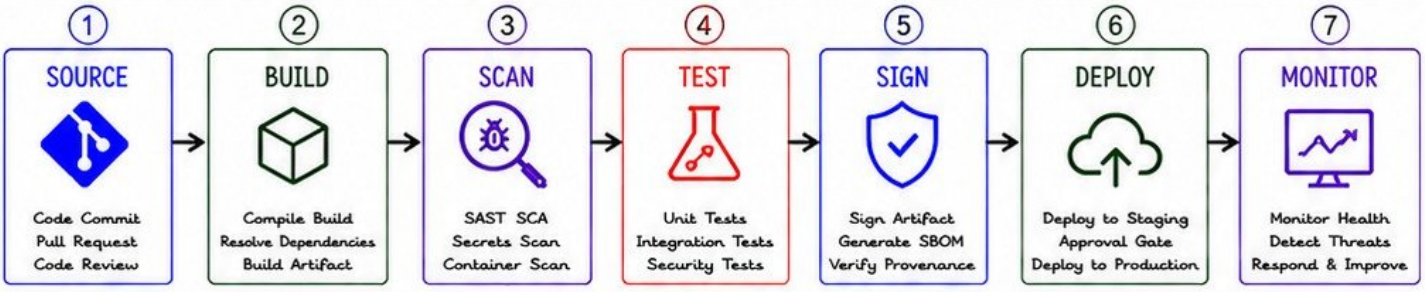
Security is built in. Automation is everywhere.
Quality is validated at every step.
Every change is trusted, tested, and traceable.
This is the modern enterprise DevSecOps pipeline.

BUILD SECURE.
DELIVER FASTER.
OPERATE SMARTER.
PROTECT CONTINUOUSLY.

PAGE 47

1 PIPELINE OVERVIEW

End-to-end secure delivery with continuous feedback.



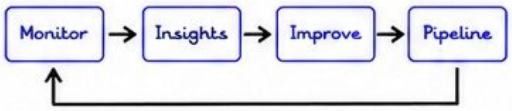
GOAL: Deliver secure, high-quality software continuously with speed, visibility, and trust.

	KEY ACTIVITIES	SECURITY CONTROLS	TOOLS EXAMPLES
1 SOURCE Manage code and changes securely.	• Branching strategy • Code commit • Pull requests • Code review	✓ Require reviews ✓ Enforce branch protection ✓ Scan secrets in commits ✓ Audit access & changes	GitHub, GitLab, Bitbucket
2 BUILD Build code into reliable artifacts.	• Compile code • Resolve dependencies • Build artifact / image • Store artifact securely	✓ Use trusted builders ✓ Pin dependencies ✓ Minimal base images ✓ Immutable build environment	Maven, Gradle, Docker
3 SCAN Find risks before they ship.	• SAST on code • SCA on dependencies • Secrets scanning • Container image scan	✓ Fail on high vulnerabilities ✓ Maintain allow/deny lists ✓ Scan IaC & configs ✓ Continuous scanning	SonarQube, Trivy, Snyk
4 TEST Validate quality and security.	• Unit tests • Integration tests • E2E tests • Security tests (DAST, API)	✓ Automate all tests ✓ Enforce quality gates ✓ Test in isolated envs ✓ Block on test failures	JUnit, Cypress, OWASP ZAP
5 SIGN Guarantee integrity and provenance.	• Sign artifact • Generate SBOM • Attach attestation • Verify provenance	✓ Use code signing ✓ Store SBOM securely ✓ Verify signatures ✓ Enforce provenance policy	cosign, Syft, in-toto
6 DEPLOY Release confidently with approvals.	• Deploy to staging • Manual/Auto approval • Deploy to production • Rollback strategy	✓ Approval gates ✓ Infrastructure as Code ✓ Least privilege deploy ✓ Canary / Blue-Green / Rollback	Argo CD, Terraform, Helm
7 MONITOR Detect, respond, and improve.	• Monitor performance • Detect security events • Alert & notify • Feed insights back	✓ Real-time alerts ✓ Centralized logging ✓ Anomaly detection ✓ Post-incident review	Prometheus, Grafana, Datadog

PIPELINE PRINCIPLES

- ✓ Security left. Security everywhere.
- ✓ Automate everything that can be automated.
- ✓ Fail fast. Fix early.
- ✓ Everything is versioned and traceable.
- ✓ Measure, learn, and continuously improve.

PIPELINE FEEDBACK LOOP



Continuously learn. Continuously improve.

KEY TAKEAWAY

A strong DevSecOps pipeline is your competitive advantage.
It delivers secure software faster, with lower risk and higher quality.
Build once. Trust always. Deliver everywhere.

AUTOMATE. SECURE. DELIVER. REPEAT. THE FUTURE IS DEVSECOPS.

VERIQTA @veriqta

VERIQTA

SHIFT RIGHT SECURITY

Security does not end at deployment.
Shift Right means applying security in production where real threats, real users, and real impact exist.
Validate continuously. Detect early. Respond faster. Improve always.

SECURITY CONTINUES IN PRODUCTION. NOT JUST IN THE PIPELINE.

PAGE 48

VERIQTA VERIQTA

1 RUNTIME VALIDATION

Validate that your systems behave securely in real production.

WHAT TO VALIDATE

- ✓ Configuration compliance
- ✓ Runtime behavior
- ✓ Access and permissions
- ✓ Network policies
- ✓ Secrets usage
- ✓ Data protection controls

RUNTIME VALIDATION FLOW

Deploy to Prod → Observe Behavior → Validate Controls → Detect Drift / Risk → Remediate Automatically

Validate continuously. Drift is a risk.

BEST PRACTICES

- ✓ Use runtime security tools
- ✓ Enforce policies in real time
- ✓ Detect config drift
- ✓ Validate least privilege
- ✓ Alert on unusual behavior
- ✓ Automate remediation

TIP

If you don't validate in production, you don't know the truth.

2 PRODUCTION MONITORING

Monitor everything that matters in production.

WHAT TO MONITOR

- ✓ System metrics
- ✓ Security events
- ✓ User activities
- ✓ API behavior
- ✓ Network traffic
- ✓ Application logs

MONITORING PILLARS

- Metrics (Health & Performance)
- Logs (Events & Activities)
- Security Telemetry (Threats & Vulnerabilities)
- Business Signals (Users & Transactions)

Correlate everything. Find the story in your data.

BEST PRACTICES

- ✓ Centralize logs
- ✓ Use real-time alerting
- ✓ Set meaningful thresholds
- ✓ Correlate across signals
- ✓ Reduce noise with context
- ✓ Review alerts regularly

TIP

Monitoring without context is noise. Monitoring with context is power.

3 ATTACK SIMULATION

Simulate real attacks in production to measure your defenses.

SIMULATION TYPES

- ✓ Red Team Exercises
- ✓ Automated Attack Tools
- ✓ Breach & Blast Radius
- ✓ Lateral Movement Tests
- ✓ Social Engineering Tests
- ✓ API Abuse Tests

ATTACK SIMULATION LIFE CYCLE

Plan Attack → Execute Attack → Detect & Observe → Measure Impact → Improve Defenses

Simulate. Measure. Improve. Repeat.

BEST PRACTICES

- ✓ Run regularly
- ✓ Include all critical paths
- ✓ Validate detection
- ✓ Measure time to respond
- ✓ Involve cross-functional teams
- ✓ Turn findings into action

TIP

Assume breach. Simulate attacks before attackers do.

4 CONTINUOUS VERIFICATION

Verify that your security posture stays strong over time.

VERIFICATION AREAS

- ✓ Policy compliance
- ✓ Vulnerability management
- ✓ Access reviews
- ✓ Configuration drift
- ✓ Data protection
- ✓ Backup & recovery

CONTINUOUS VERIFICATION LOOP

Define Baseline → Scan & Validate → Detect Deviation → Remediate Automatically → Report & Measure → Define Baseline

Never assume secure. Always verify.

BEST PRACTICES

- ✓ Automate compliance scans
- ✓ Use policy as code
- ✓ Track trends over time
- ✓ Reduce mean time to fix
- ✓ Validate changes
- ✓ Keep documentation updated

TIP

Security is a moving target. Verification keeps you on the target.

5 FEEDBACK LOOPS

Turn insights into improvements and make your system stronger.

FEEDBACK SOURCES

- ✓ Incident Postmortems
- ✓ Alert Analysis
- ✓ Attack Simulations
- ✓ User Feedback
- ✓ Threat Intelligence
- ✓ Compliance Findings

FEEDBACK LOOP FLOW

Collect Insights → Analyze Root Cause → Prioritize Actions → Implement Changes → Measure Impact

Feedback is the fuel. Improvement is the engine.

BEST PRACTICES

- ✓ Close the loop
- ✓ Track actions and results
- ✓ Share learnings
- ✓ Improve detection
- ✓ Improve prevention
- ✓ Build institutional memory

TIP

If you don't learn, you will repeat. Don't repeat.

SHIFT RIGHT SECURITY PRINCIPLES

- ✓ Security continues in production.
- ✓ Real threats happen in real time.
- ✓ Detect early. Respond fast.
- ✓ Validate continuously.
- ✓ Learn and improve constantly.

SHIFT LEFT + SHIFT RIGHT = FULL SECURITY

Shift Left (Build Secure) | Shift Right (Run Secure)

Build secure. Run secure. Stay secure.

KEY TAKEAWAY

Shift Right Security ensures your applications stay secure long after deployment. Monitor, validate, simulate, verify, and learn continuously. Security is a journey, not a checkpoint.

SECURITY DOESN'T END AT DEPLOYMENT. IT EVOLVES EVERY DAY.

VERIQTA

ZERO TRUST FOR DEVSECOPS

Zero Trust means "never trust, always verify."
No implicit trust. Verify every user, device, service, and request.
Protect your pipelines, code, infrastructure, and data everywhere.

VERIFY EXPLICITLY.
GRANT LEAST PRIVILEGE.
ASSUME BREACH.
CONTINUOUSLY MONITOR.

1 IDENTITY-FIRST SECURITY

Every access starts with a verified identity.

KEY PRINCIPLES

- ✓ Strong identity
- ✓ Multi-factor authentication
- ✓ Role-based access
- ✓ Just-in-time access
- ✓ Continuous evaluation

IDENTITY LIFECYCLE

User / Service Identity → Authenticate (MFA) → Authorize (Least Privilege) → Access Resources → Continuously Verify

Identity is the new perimeter.

BEST PRACTICES

- ✓ Enforce MFA everywhere
- ✓ Use SSO for all access
- ✓ Use short-lived credentials
- ✓ Use Roles, not users
- ✓ Review access regularly

TIP

If you can't verify it, don't trust it.

2 CONTINUOUS VERIFICATION

Trust is not permanent. Verify every request every time.

WHAT TO VERIFY

- ✓ User / Service identity
- ✓ Device health
- ✓ Location & network
- ✓ Behavior & risk
- ✓ Resource sensitivity

CONTINUOUS VERIFICATION FLOW

Initial Access → Evaluate Context → Assess Risk (High / Low) → Allow Limited Access or Deny / Step-up → Re-verify Continuously

Context changes. Trust must adapt.

BEST PRACTICES

- ✓ Evaluate risk in real time
- ✓ Use adaptive access
- ✓ Re-authenticate on risk
- ✓ Detect anomalies early
- ✓ Automate decisioning

TIP

Static access is risk. Dynamic access is safety.

3 DEVICE TRUST

Only trusted, compliant devices get access.

DEVICE TRUST FACTORS

- ✓ Device posture
- ✓ OS & patch level
- ✓ Endpoint protection
- ✓ Encryption status
- ✓ Jailbreak / Root check

DEVICE TRUST FLOW

Device Connects → Check Health & Compliance → Assess Trust → Allow Access (Limited) → Deny / Quarantine

Unhealthy devices are blocked or isolated.

BEST PRACTICES

- ✓ Use device posture checks
- ✓ Enforce MDM / EDR
- ✓ Require full disk encryption
- ✓ Block non-compliant devices
- ✓ Monitor device continuously

TIP

A secure user on an insecure device is still a risk.

4 LEAST PRIVILEGE

Give minimum access needed. Nothing more.

LEAST PRIVILEGE PRINCIPLES

- ✓ Need-to-know access
- ✓ Need-to-do access
- ✓ Just-in-time elevation
- ✓ Use roles, not users
- ✓ Remove unused access

ACCESS MODEL

User / Service → Request Access → Policy Decision → Allow (Least) → Resources (Scoped)

Policy Decision → Deny (Too Much)

BEST PRACTICES

- ✓ Scope access to resources
- ✓ Use RBAC / ABAC
- ✓ Use temporary access
- ✓ Review and revoke access
- ✓ Automate access reviews

TIP

Too much access is the #1 security risk.

5 SECURE ACCESS

Secure connections and controlled access to everything.

SECURE ACCESS CONTROLS

- ✓ Encrypted connections
- ✓ Private networks
- ✓ VPN / Zero Trust Network Access (ZTNA)
- ✓ Micro-segmentation
- ✓ API access control

SECURE ACCESS PATH

User / Service → Encrypted Connection → Zero Trust Gateway → Policy Enforcement → Application / Resource

All access is authenticated, authorized, and encrypted.

BEST PRACTICES

- ✓ Use ZTNA for all access
- ✓ Encrypt everything
- ✓ Segment networks
- ✓ Protect APIs
- ✓ Monitor access patterns

TIP

Never expose direct access. Always go through trust.

ZERO TRUST PILLARS

- ✓ Verify explicitly
- ✓ Use least privilege access
- ✓ Assume breach
- ✓ Inspect and log everything
- ✓ Automate and adapt
- ✓ Continuously improve

ZERO TRUST IN DEVSECOPS

Developers → Code Repo → CI/CD Pipeline → Infrastructure → Applications → Data

Secure every identity, pipeline, workload, and data flow.
Zero Trust is the foundation of modern DevSecOps.

KEY TAKEAWAY

Zero Trust for DevSecOps means securing every identity, device, pipeline, and workload with continuous verification, least privilege, and secure access. Trust is never assumed. It is earned every time.

ZERO TRUST IS NOT A PRODUCT. IT'S A STRATEGY.

VERIQTA

PRODUCTION FAILURE CASE STUDIES

Real failures. Real impact. Real lessons.
We don't learn from theory. We learn from what broke in production.
These cases show how security failures happen, how we investigated, and what we changed to make sure it never happens again.

FAILURES WILL HAPPEN.
PREPARE. RESPOND.
LEARN. IMPROVE.
PREVENT NEXT TIME.

PAGE 50

1 SECRET EXPOSURE

What happened:
A developer committed an AWS access key to a public GitHub repository.

IMPACT
- Unauthorized access
- Data exfiltration
- Resources abused
- High cloud bills
- Reputational damage

ROOT CAUSE
Secrets stored in code.
No secret scanning.

WHAT WE DID
- ✓ Revoked the key immediately
- ✓ Rotated all credentials
- ✓ Scanned all repos for secrets
- ✓ Implemented secret scanning
- ✓ Moved secrets to a vault
- ✓ Enforced pre-commit scanning

LESSONS LEARNED
- ✓ Never store secrets in code
- ✓ Scan early. Scan always
- ✓ Rotate credentials
- ✓ Use a secrets manager
- ✓ Train developers

2 VULNERABLE DEPENDENCY

What happened:
A vulnerable library (log4j) was used in production without updates.

IMPACT
- Remote code execution
- System compromise
- Service outage
- Customer impact

ROOT CAUSE
No dependency scanning.
Outdated libraries.

WHAT WE DID
- ✓ Identified all affected systems
- ✓ Patched and updated libraries
- ✓ Added dependency scanning
- ✓ Enabled automatic updates
- ✓ Created vulnerability SLAs

LESSONS LEARNED
- ✓ Scan dependencies in CI/CD
- ✓ Keep libraries updated
- ✓ Review alerts, not ignore
- ✓ Use SBOM for visibility
- ✓ Automate security checks

3 MISCONFIGURED IAM

What happened:
An IAM role had excessive permissions and was assumed by a compromised workload.

IMPACT
- Full account access
- Data accessed
- Resources modified
- Compliance violation

ROOT CAUSE
Overly permissive policies.
No access review.

WHAT WE DID
- ✓ Revoked excessive permissions
- ✓ Applied least privilege
- ✓ Enabled IAM Access Analyzer
- ✓ Set up permission boundaries
- ✓ Implemented access reviews

LESSONS LEARNED
- ✓ Least privilege always
- ✓ Review access regularly
- ✓ Use permission boundaries
- ✓ Monitor privilege usage
- ✓ Automate access governance

4 CONTAINER ESCAPE

What happened:
A container broke out of isolation due to a misconfigured privileged container.

IMPACT
- Host access gained
- Lateral movement
- Data accessed
- Cluster risk

ROOT CAUSE
Privileged container.
Weak runtime security.

WHAT WE DID
- ✓ Removed privileged mode
- ✓ Enabled Pod Security Standards
- ✓ Updated runtime policies
- ✓ Implemented seccomp & AppArmor
- ✓ Added runtime threat detection

LESSONS LEARNED
- ✓ Don't run privileged containers
- ✓ Follow least privilege in K8s
- ✓ Use runtime security
- ✓ Harden node configurations
- ✓ Continuously validate runtime

5 LESSONS LEARNED

What happened:
Security gaps stacked up over time without addressing root causes.

IMPACT
- Major incident
- Extended downtime
- Lost trust
- High recovery cost

ROOT CAUSE
Security debt.
No continuous improvement.

WHAT WE DID
- ✓ Conducted post-mortem
- ✓ Strengthened security program
- ✓ Automated detection & response
- ✓ Invested in people & tools
- ✓ Built a security culture

LESSONS LEARNED
- ✓ Security is a process
- ✓ Fix root cause, not symptoms
- ✓ Measure and track metrics
- ✓ Improve continuously
- ✓ Share lessons across teams

CASE STUDY PRINCIPLES

- ✓ Blameless post-mortems
- ✓ Focus on systems, not people
- ✓ Document everything
- ✓ Share lessons learned
- ✓ Prevent, detect, respond, recover

INCIDENT RESPONSE FLOW

Detect → Investigate → Contain → Eradicate → Recover → Learn

Respond fast. Learn deeply. Improve continuously.

KEY TAKEAWAY

Every failure is a teacher. The goal is not to avoid all failures, but to ensure each one makes you stronger, smarter, and more secure than before.

LEARN FAST. FIX FAST. IMPROVE FOREVER.

VERIQTA

ENTERPRISE SECURITY ARCHITECTURE CASE STUDY

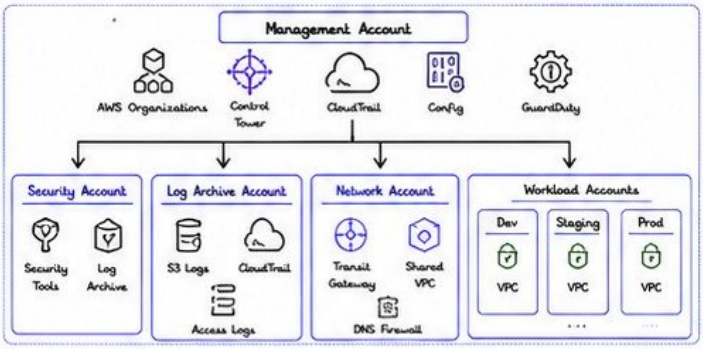
A real-world enterprise platform built with security at every layer.
Multi-account AWS environment. Secure networking. Hardened CI/CD.
Kubernetes security. Governance and compliance.
This is how modern enterprises build and operate secure systems.

SECURE BY DESIGN.
PROTECT EVERY LAYER.
GOVERN EVERY ACCOUNT.
TRUST, VERIFY, MONITOR.

PAGE 51

1 MULTI-ACCOUNT CLOUD ARCHITECTURE

Isolate environments. Separate responsibilities. Enforce guardrails.

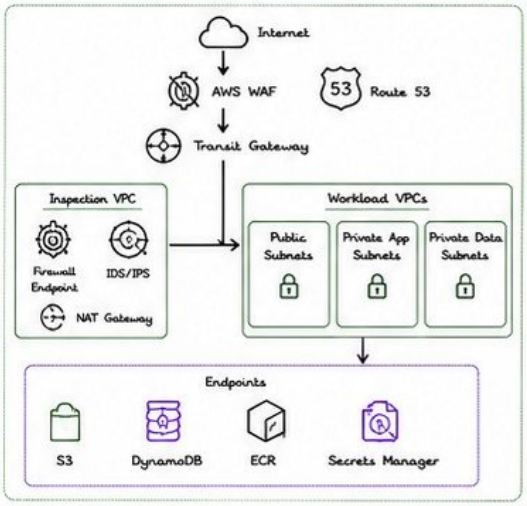


KEY CONTROLS

- ✓ SCPs enforce guardrails
- ✓ Centralized logging & monitoring
- ✓ Least privilege between accounts
- ✓ Tagging and cost allocation
- ✓ AWS Config rules
- ✓ Centralized security tooling
- ✓ Automated account provisioning

2 SECURE NETWORKING

Segment everything. Control every path. Inspect all traffic.

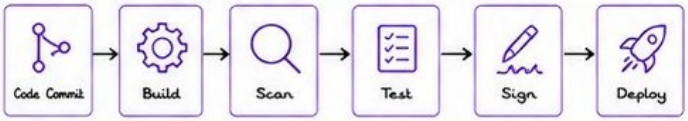


CONTROLS

- ✓ VPC segmentation
- ✓ Private subnets
- ✓ NAT + Egress control
- ✓ AWS WAF
- ✓ Network Firewall
- ✓ Route 53 DNSSEC
- ✓ TLS everywhere
- ✓ VPC Flow Logs
- ✓ Endpoint policies
- ✓ SGs / NACLs
- ✓ Zero trust networking

3 CI/CD SECURITY

Secure pipeline. Secure artifacts. Secure deployments.



SECURITY CONTROLS

- ✓ Branch protection
- ✓ SAST on code
- ✓ Dependency scanning
- ✓ Secrets scanning
- ✓ IaC scanning
- ✓ Container image scanning
- ✓ Policy as Code (OPA)
- ✓ Artifact signing (Sigstore)
- ✓ SBOM generation
- ✓ Provenance (in-toto)
- ✓ Approval gates
- ✓ Least privilege deploy

Fail early. Fail safely. Never deploy untrusted code.

4 KUBERNETES SECURITY

Secure the cluster. Secure the workloads. Secure the runtime.

CLUSTER LAYERS

- API Server Security
- etcd Encryption
- RBAC & AuthN
- Network Policies
- Pod Security Standards
- Admission Controllers
- Secrets Management
- Runtime Security

IMPLEMENTATION

- ✓ MFA for kubectl access
- ✓ Least privilege RBAC
- ✓ Namespace isolation
- ✓ NetworkPolicies (default deny)
- ✓ Pod Security Standards (restricted)
- ✓ Image scanning (Trivy)
- ✓ Secrets in external store
- ✓ mTLS for service-to-service
- ✓ Runtime protection (Falco/AppArmor)
- ✓ audit logs to SIEM

TOOLS

Falco · Trivy · Kyverno · OPA Gatekeeper · External Secrets · Prometheus

5 GOVERNANCE & COMPLIANCE MODEL

Policies. Guardrails. Continuous compliance.

GOVERNANCE PILLARS

- Identity & Access
- Data Protection
- Infrastructure Protection
- Threat Detection
- Compliance & Audit

GOVERNANCE CONTROLS

- ✓ SCPs and Guardrails
- ✓ IAM least privilege
- ✓ Encryption everywhere
- ✓ Data classification
- ✓ Backup & DR strategy
- ✓ Logging & audit
- ✓ Compliance frameworks
- ✓ Continuous monitoring
- ✓ Risk management
- ✓ Incident response

COMPLIANCE FRAMEWORKS

- CIS Benchmark
- ISO 27001
- SOC 2
- NIST 800-53
- PCI DSS

ENTERPRISE ARCHITECTURE SUMMARY

Multi-account foundation
↓
Secure networking
↓
Secure CI/CD pipeline
↓
Kubernetes security
↓
Governance & compliance

OUTCOMES

- ✓ Strong security posture
- ✓ Blast radius reduction
- ✓ Compliance ready
- ✓ Continuous visibility
- ✓ Faster and safer delivery
- ✓ Business resilience

KEY TAKEAWAY

Enterprise security is not a product.
It is an architecture, a process,
and a culture. Build secure.
Operate secure. Improve
continuously.

DESIGN PRINCIPLES

- ✓ Security by Design
- ✓ Least Privilege
- ✓ Assume Breach
- ✓ Defense in Depth
- ✓ Automate Security
- ✓ Continuous Compliance
- ✓ Observe Everything
- ✓ Improve Continuously

OPERATIONAL EXCELLENCE LOOP

Monitor → Detect → Investigate → Respond → Improve

Measure. Learn. Improve. Repeat.

SECURE ARCHITECTURE TODAY. RESILIENT ENTERPRISE TOMORROW.

VERIQTA @veriqta

VERIQTA

BUILDING AN ENTERPRISE DEVSECOPS PLATFORM

A platform that empowers developers to ship fast and secure.
Built on automation. Designed for scale. Secured by default.
This is how we build a modern DevSecOps platform for the enterprise.

EMPOWER DEVELOPERS.
AUTOMATE SECURITY.
STANDARDIZE EVERYTHING.
DELIVER SECURELY AT SCALE.

1 DEVELOPER WORKFLOW

Simple. Fast. Secure by default.

Code → Commit & PR → CI Pipeline → Deploy → Release → Monitor

WORKFLOW PRINCIPLES

- ✓ Developers own the code. Platform handles the security.
- ✓ Automated checks at every step.
- ✓ Fast feedback. Fail early.
- ✓ Secure defaults. No exceptions.
- ✓ Observability and traceability built-in.

TIP

Make the secure path the easy path.

2 SECURITY AUTOMATION

Automate security at every stage.

Plan → Code → Build → Test → Deploy → Run

SHIFT LEFT	BUILD SECURITY	TEST SECURITY	DEPLOY SECURITY	RUNTIME SECURITY
✓ SAST	✓ Image Scan	✓ DAST	✓ Policy Check	✓ Runtime Threat Detection
✓ Secrets Scan	✓ SBOM Gen	✓ API Security	✓ Admission Ctrl	✓ File Integrity
✓ IaC Scan	✓ Artifact Sign	✓ Container Scan	✓ Image Verify	✓ Network Monitor
✓ Dependency Scan	✓ Vulnerability Scan	✓ Config Scan	✓ Infra Guardrails	✓ Behavior Analysis
✓ Policy as Code		✓ License Check	✓ Canary Analysis	

Automate. Enforce. Detect. Respond. Continuously.

3 PLATFORM ENGINEERING

Build the golden path. Hide the complexity.

PLATFORM CAPABILITIES

- CI/CD Pipelines
- Security Jobs
- Infrastructure as Code
- Policy Enforcement
- Secrets Management
- Observability Stack
- Approval Workflows
- Audit & Compliance

TIP Treat the platform like a product. Version it. Improve it. Measure it.

PLATFORM ARCHITECTURE

Developer Portal | APIs | CLI

↓

Platform Services Layer: CI/CD | Security | IaC | Policy | Observability

↓

Shared Services: Identity | Secrets | Logging | Monitoring | Storage

↓

Infrastructure: Kubernetes | Cloud | Network | Storage | Compute

4 SELF-SERVICE SECURITY

Empower teams with secure self-service.

SELF-SERVICE CAPABILITIES

- Project Onboarding — One-click project setup
- Pipeline Templates — Secure CI/CD templates
- Policy Library — Reusable security policies
- Security Dashboard — Findings and posture
- Exception Management — Request, approve, track
- Security Guardrails — Pre-built secure controls

DEVELOPER EXPERIENCE

Developer → Portal / CLI / API

- ☑ Choose Template
- ☑ Configure Variables
- ☑ Deploy Pipeline
- ☑ Get Security Feedback
- ☑ Ship with Confidence ✓

TIP Give developers autonomy within secure boundaries.

5 STANDARDIZATION

Consistent by design. Scalable by default.

STANDARD PIPELINES	STANDARD POLICIES	STANDARD PLATFORMS	STANDARD COMPLIANCE
✓ Build	✓ Code Quality	✓ Kubernetes	✓ CIS Benchmarks
✓ Scan	✓ Secrets	✓ Cloud Accounts	✓ ISO 27001
✓ Test	✓ Dependencies	✓ Networks	✓ NIST 800-53
✓ Sign	✓ Containers	✓ Storage	✓ SOC 2
✓ Deploy	✓ IaC	✓ Monitoring	✓ PCI DSS
✓ Monitor	✓ Runtime	✓ Logging	

TIP Standardization reduces risk, accelerates delivery, and improves compliance.

ENTERPRISE DEVSECOPS PLATFORM OUTCOMES

- Faster, safer software delivery
- Reduced security risk
- Consistent security posture
- Full visibility and traceability
- Lower MTTR and better resilience
- Developer productivity at scale
- Audit-ready and compliant

SUCCESS METRICS

- Deployment Frequency ↑
- Change Failure Rate ↓
- Lead Time for Change ↓
- MTTR ↓
- Vulnerability Exposure ↓
- Security Coverage ↑
- Developer Satisfaction ↑

PLATFORM DESIGN PRINCIPLES

- ✓ Secure by default
- ✓ Automation over manual
- ✓ Self-service with guardrails
- ✓ Standardize everything
- ✓ Observability everywhere
- ✓ Continuously improve

CONTINUOUS IMPROVEMENT LOOP

Measure → Feedback → Improve → Release → Adopt

Build. Measure. Learn. Improve. Repeat.

KEY TAKEAWAY

An enterprise DevSecOps platform is not a tool. It is a system of people, processes, and technology that enables secure speed at scale.
Build the platform. Empower the teams. Deliver value securely. Every time.

A STRONG PLATFORM EMPOWERS TEAMS. AUTOMATION PROTECTS YOUR FUTURE.

VERIQTA

END-TO-END SECURE SDLC PROJECT

Security is not a phase. It is the entire lifecycle.
This project shows how to build and deliver a secure application from idea to production and beyond.
Secure by design. Secure by default. Secure forever.

BUILD SECURE.
TEST SECURE.
DEPLOY SECURE.
OPERATE SECURE.
IMPROVE CONTINUOUSLY.

VERIQTA

1 PLANNING

Define what to build,
how to build it securely,
and what success
looks like.

KEY ACTIVITIES

- ✓ Define business goals
- ✓ Asset & data classification
- ✓ Threat modeling (STRIDE)
- ✓ Security requirements
- ✓ Risk assessment
- ✓ Compliance mapping
- ✓ Architecture review

SECURITY CONTROLS

- Security champions
- Security policies
- Data protection strategy
- Privacy impact assessment
- Secure architecture principles

DELIVERABLES

- Threat model
- Security requirements
- Risk register
- Compliance checklist
- Secure architecture

2 DEVELOPMENT

Build the code right.
Secure coding.
Secure configurations.
Secure dependencies.

KEY ACTIVITIES

- ✓ Secure coding standards
- ✓ Code reviews
- ✓ Dependency management
- ✓ Secrets management
- ✓ Static code analysis
- ✓ IaC & config as code
- ✓ Developer security training

SECURITY CONTROLS

- SAST (Static Analysis)
- Dependency scanning
- Secrets scanning
- Pre-commit hooks
- Code signing

DELIVERABLES

- Source code
- SBOM (Software Bill of Materials)
- Code analysis reports
- IaC templates
- Developer checklist

3 TESTING

Find issues early.
Break it safely.
Fix it completely.

KEY ACTIVITIES

- ✓ SAST & SCA scans
- ✓ DAST scanning
- ✓ Container image scanning
- ✓ API security testing
- ✓ Fuzz testing
- ✓ Penetration testing
- ✓ Security regression testing

SECURITY CONTROLS

- DAST (Dynamic Analysis)
- Container scanning
- License compliance
- Secrets validation
- Vulnerability gating

DELIVERABLES

- Security test reports
- Vulnerability report
- Pen test report
- Compliance report
- Go/No-Go approval

4 DEPLOYMENT

Deploy securely.
Protect the pipeline.
Verify the release.

KEY ACTIVITIES

- ✓ CI/CD pipeline with security gates
- ✓ Artifact signing
- ✓ Infrastructure validation
- ✓ Policy as Code
- ✓ Environment hardening
- ✓ Deployment approval
- ✓ Rollback plan

SECURITY CONTROLS

- Image signing & verification
- Policy enforcement (OPA/Gatekeeper)
- IaC scanning
- Secrets injection
- Change management

DELIVERABLES

- Signed artifacts
- Pipeline logs
- IaC compliance report
- Deployment checklist
- Release notes

5 OPERATIONS

Run securely.
Monitor everything.
Respond fast.
Improve always.

KEY ACTIVITIES

- ✓ Continuous monitoring
- ✓ Incident response
- ✓ Vulnerability management
- ✓ Log aggregation
- ✓ Backup & recovery
- ✓ Compliance monitoring
- ✓ Continuous improvement

SECURITY CONTROLS

- Runtime protection
- SIEM & alerting
- WAF & DDoS protection
- Patch management
- Access reviews

DELIVERABLES

- Monitoring dashboard
- Incident reports
- Vulnerability reports
- Audit reports
- Post-mortem reports

END-TO-END SECURE SDLC FLOW

1 Planning → 2 Development → 3 Testing → 4 Deployment → 5 Operations → Secure Application

Security is Continuous. Not a Phase.

KEY TAKEAWAY

A secure SDLC is repeatable,
measurable, and automatable.
Security is built in, not bolted on.
Secure every step. Every time.

SECURE SDLC PRINCIPLES

- ✓ Security by Design
- ✓ Shift Left Security
- ✓ Automate Security
- ✓ Everyone Owns Security
- ✓ Measure and Improve

SECURITY METRICS

Vulnerabilities Found (Early)	↑
Vulnerability Escape Rate	↓
Mean Time to Detect (MTTD)	↓
Mean Time to Respond (MTTR)	↓
Compliance Score	↑

TOOLS EXAMPLES

SAST: SonarQube, CodeQL
SCA: Snyk, Dependabot
DAST: OWASP ZAP, Burp Suite
IaC: Checkov, tfsec
Containers: Trivy, Grype
CI/CD: GitHub Actions, GitLab CI

REMEMBER

People + Process + Technology = Secure Software at Scale.

SECURITY IS EVERYONE'S JOB. BUILD SECURE. DELIVER CONFIDENTLY.

VERIQTA

ENTERPRISE DEVSECOPS TROUBLESHOOTING

Issues will happen. Fast, systematic troubleshooting is what separates junior engineers from senior operators.
Detect early. Investigate deeply. Fix correctly. Prevent forever.

OBSERVE. INVESTIGATE.
ANALYZE. RESOLVE.
DOCUMENT. PREVENT.
CONTINUOUSLY IMPROVE.

PAGE 54

1 FAILED SCANS

What it looks like
- SAST / DAST fails
- Container scan fails
- Dependency scan fails
- Policy check fails

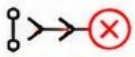
COMMON CAUSES
- ✓ Tool misconfiguration
- ✓ Network / proxy issues
- ✓ Invalid credentials / tokens
- ✓ Large artifacts / timeouts
- ✓ Tool or DB update issues
- ✓ Unsupported file types

TROUBLESHOOTING STEPS
1. Check pipeline logs for exact error
2. Validate tool configuration
3. Verify network connectivity
4. Check credentials / tokens
5. Retry scan locally
6. Check tool status page
7. Check resource limits / timeouts

TOOLS & COMMANDS
```
curl -I <tool-endpoint>
ping <host>
nslookup <host>
docker logs <scanner>
trivy --debug image:tag
sonar-scanner -X
cat pipeline.log
```

2 PIPELINE FAILURES

What it looks like
- Build fails
- Tests fail
- Deploy fails
- Pipeline stops unexpectedly



COMMON CAUSES
- ✓ Code compilation errors
- ✓ Failing unit / integration tests
- ✓ Artifact not found
- ✓ Wrong environment variables
- ✓ Service / dependency down
- ✓ Timeout / resource limits

TROUBLESHOOTING STEPS
1. Check stage where it failed
2. Read full logs, not just the error
3. Re-run with debug / verbose
4. Validate artifacts exist
5. Verify environment variables
6. Check dependent services
7. Fix and re-run with confidence

TOOLS & COMMANDS
```
docker-compose logs
kubectl get pods
kubectl logs <pod>
grep -i error pipeline.log
printenv | sort
jq . <file.json>
npm test / mvn test -e
```

3 PERMISSION ISSUES

What it looks like
- Access denied
- 403 / 401 errors
- Cannot deploy / push
- Secrets / roles not working

COMMON CAUSES
- ✓ IAM role missing policies
- ✓ Expired / invalid tokens
- ✓ Wrong account / project
- ✓ Missing RBAC permissions
- ✓ Secrets not mounted
- ✓ Least privilege blocking

TROUBLESHOOTING STEPS
1. Identify who/what is failing
2. Check error message carefully
3. Validate identity / token
4. Check IAM / RBAC permissions
5. Verify role trust relationship
6. Test with least privilege
7. Apply correct permission

TOOLS & COMMANDS
```
aws sts get-caller-identity
aws iam simulate-principal-policy
kubectl auth can-i --list
kubectl auth can-i <verb> <res>
gcloud auth list
az account show
echo $TOKEN | cut -c1-20
```

4 RUNTIME ALERTS

What it looks like
- High CPU / Memory
- Pod CrashLoopBackOff
- 5xx errors
- Security runtime alert

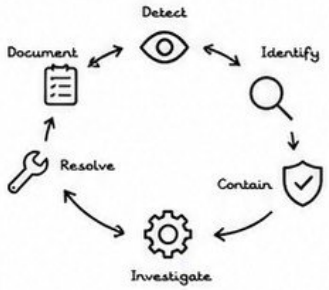
COMMON CAUSES
- ✓ Resource exhaustion
- ✓ Application bugs
- ✓ Misconfiguration
- ✓ Probe failures
- ✓ Dependency failures
- ✓ Security threats detected

TROUBLESHOOTING STEPS
1. Check alert details
2. Identify affected service/pod
3. Review dashboards & logs
4. Check resource usage
5. Check dependencies
6. Check recent changes / deploys
7. Contain, fix and validate

TOOLS & COMMANDS
```
kubectl top pods
kubectl describe pod <pod>
kubectl logs <pod> -f
docker stats
prometheus / grafana
aws cloudwatch logs
kubectl get events
--sort-by=.lastTimestamp
```

5 INVESTIGATION WORKFLOW

Follow this every time.



SENIOR INVESTIGATION CHECKLIST
- ☑ What is the problem?
- ☑ When did it start?
- ☑ What changed recently?
- ☑ Who is affected?
- ☑ What evidence do we have?
- ☑ Which systems are involved?
- ☑ What are the logs saying?
- ☑ What is the root cause?
- ☑ What is the impact?
- ☑ What is our resolution?
- ☑ How do we prevent this again?

INVESTIGATION COMMANDS CHEAT SHEET

Kubernetes
```
kubectl get all -A
kubectl describe <resource> <name>
kubectl logs <pod> -n <ns> --previous
kubectl get events -A --sort-by=.lastTimestamp
```

Linux / Containers
```
ps aux | sort -nrk 3
df -h
free -m
journalctl -xe
docker ps -a
docker logs <container>
```

Cloud / Network
```
curl -I <url>
dig <domain>
traceroute <host>
nslookug <host>
mtr <host>
```

QUICK TIPS
- ☑ Read the full error.
- ☑ Check the recent changes first.
- ☑ Reproduce the issue locally if possible.
- ☑ Don't guess. Collect evidence.
- ☑ Document everything.
- ☑ Fix the root cause, not the symptom.

TROUBLESHOOT LIKE A SENIOR
- ✓ Be calm. Follow the process.
- ✓ Ask the right questions.
- ✓ Use data, not assumptions.
- ✓ Fix once. Fix the root.
- ✓ Share learnings with the team.

TROUBLESHOOTING PRINCIPLES

Observe everything → Ask why → Collect evidence → Analyze deeply → Fix correctly → Prevent recurrence

Good engineers solve problems. Great engineers prevent them.

KEY TAKEAWAY

A structured troubleshooting approach saves time, reduces downtime and builds trust. Strong processes + strong tools = reliable systems.

PROBLEMS ARE INEVITABLE. CHAOS IS OPTIONAL. PROCESS IS POWER.

VERIQTA @veriqta

VERIQTA

PRODUCTION DEBUGGING PLAYBOOK

When production fails, senior engineers follow a process, not panic.
This playbook is your step-by-step guide to investigate, resolve, and prevent issues in modern cloud-native systems.

OBSERVE. COLLECT. ANALYZE. RESOLVE. LEARN. IMPROVE. REPEAT.

1 EVIDENCE COLLECTION

Before touching anything, collect high-quality evidence.

WHAT TO COLLECT

- ✓ Timestamps (UTC)
- ✓ Error messages
- ✓ Logs (all relevant services)
- ✓ Metrics & dashboards
- ✓ Configuration snapshots
- ✓ Deployment history
- ✓ System state (ps, df, top)
- ✓ Network info (ip, routes, dns)
- ✓ User impact (who, what, when)

TOOLS & COMMANDS

- kubectl get pods -A -o wide
- kubectl describe <resource>
- kubectl logs <pod> --all-containers
- kubectl logs <pod> -p (previous)
- journalctl -xeu <service> --since "-1h"
- dmesg | tail -n 200
- ss -tulpen
- curl -v <endpoint>
- aws cloudwatch get-metric-data ...
- aws logs describe-log-groups

BEST PRACTICES

- ✓ Do not restart anything yet
- ✓ Capture before the system changes
- ✓ Note exact time of incident
- ✓ Use UTC everywhere
- ✓ Preserve logs (don't truncate)
- ✓ Store evidence in a safe place
- ✓ Communicate: let the team know

2 LOG ANALYSIS

Logs tell the story.
Find the signal.
Ignore the noise.

WHAT TO LOOK FOR

- ✓ Error patterns & stack traces
- ✓ First occurrence (not last)
- ✓ Correlated events
- ✓ Latency or timeout spikes
- ✓ Failed dependencies
- ✓ Authentication / authorization
- ✓ Resource exhaustion
- ✓ Unusual changes

TOOLS & COMMANDS

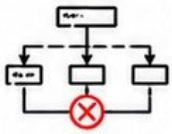
- grep -i "error" app.log
- grep -i "exception" app.log
- jq (parse JSON logs)
- kubectl logs <pod> | grep -i error
- aws logs filter-log-events ...
- elasticsearch / kibana
- loki query
- splunk search
- cloudwatch logs insights

BEST PRACTICES

- ✓ Search by time window
- ✓ Use multiple log sources
- ✓ Correlate across services
- ✓ Pivot from errors to causes
- ✓ Look for the FIRST error
- ✓ Follow the full request path
- ✓ Document key findings

3 ROOT CAUSE ANALYSIS

Find the real cause, not the symptom.



ANALYSIS FRAMEWORKS

- ✓ 5 Whys
- ✓ Ishikawa (Fishbone)
- ✓ Fault Tree Analysis
- ✓ Event Correlation
- ✓ Change Impact Analysis
- ✓ Dependency Mapping
- ✓ Timeline Reconstruction

KEY QUESTIONS

- ✓ What changed?
- ✓ What failed first?
- ✓ What is the impact?
- ✓ Which system is responsible?
- ✓ Why did the protection fail?
- ✓ Could we have detected earlier?
- ✓ How do we prevent recurrence?

BEST PRACTICES

- ✓ Blame the system, not people
- ✓ Validate assumptions
- ✓ Use data, not guesswork
- ✓ Involve SMEs early
- ✓ Keep a timeline
- ✓ Record your hypothesis
- ✓ Prove the root cause

4 RECOVERY

Restore service safely.
Minimize impact.

RECOVERY STRATEGIES

- ✓ Rollback deployment
- ✓ Restart service / pod
- ✓ Scale resources
- ✓ Failover to healthy zone/region
- ✓ Clear cache / queues
- ✓ Revert configuration
- ✓ Patch or hotfix
- ✓ Data restore (if needed)

TOOLS & COMMANDS

- kubectl rollout undo deployment/<name>
- kubectl delete pod <pod-name>
- kubectl scale deployment <name> --replicas=...
- systemctl restart <service>
- terrafarm apply -target=...
- aws elbv2 set-target-group-health ...
- redis-cli flushall (use with caution)
- aws rds failover-db-cluster ...

BEST PRACTICES

- ✓ Recover the service first
- ✓ Communicate status clearly
- ✓ Verify full functionality
- ✓ Monitor closely after recovery
- ✓ Do not apply permanent fixes until root cause is confirmed
- ✓ Document what you did

5 POST-INCIDENT REVIEW

Every incident is a lesson.
Make it count.

REVIEW CHECKLIST

- ✓ What happened?
- ✓ When & how long?
- ✓ Who was impacted?
- ✓ Root cause
- ✓ What worked well?
- ✓ What failed?
- ✓ Action items
- ✓ Owners & deadlines

IMPROVEMENT ACTIONS

- ✓ Update runbooks
- ✓ Add alerts / dashboards
- ✓ Improve tests
- ✓ Automate detection
- ✓ Fix documentation gaps
- ✓ Train the team
- ✓ Update architecture
- ✓ Prevent recurrence

TIP

A 1-hour review today prevents a 10-hour incident tomorrow.

DOCUMENTATION TEMPLATE

Incident ID: _____
Date/Time: _____
Duration: _____
Impact: _____
Root Cause: _____
Resolution: _____
Action Items: _____
_____

GOLDEN RULES

- ✓ Stay calm. Think. Then act.
- ✓ Collect before you change.
- ✓ Look for the first failure.
- ✓ Data over assumptions.
- ✓ Learn and make it better.

INCIDENT INVESTIGATION FLOW

Collect → Analyze Logs → Find Root Cause → Recover → Review & Improve

Observe → Collect → Analyze → Resolve → Learn → Improve

KEY TAKEAWAY

A structured debugging process turns chaos into clarity. Follow the playbook, respect the data, and always improve the system.
Good engineers solve incidents.
Great engineers prevent them.

INVESTIGATE WITH DATA. RESOLVE WITH CONFIDENCE. IMPROVE ALWAYS.

VERIQTA

SENIOR ENGINEER TIPS

Security is not a tool. It is a mindset.
The best engineers build systems that are secure, reliable, observable, and maintainable for years.

DESIGN SECURELY.
AUTOMATE EVERYTHING.
MAKE SMART TRADE-OFFS.
OPERATE EXCELLENTLY.
BUILD FOR THE LONG TERM.

PAGE 56

1 SECURITY DESIGN PRINCIPLES

Design security in from the start, not as an afterthought.

- ✓ Secure by Design, not by default.
- ✓ Defense in Depth across all layers.
- ✓ Zero Trust: never trust, always verify.
- ✓ Least Privilege everywhere.
- ✓ Fail Secure, not Open.
- ✓ Keep the attack surface as small as possible.
- ✓ Encrypt data in transit and at rest.
- ✓ Assume breach. Plan for it.

REMEMBER
Good security is invisible when everything works and priceless when things go wrong.

SENIOR TIP
If you didn't design it, someone will have to defend it.
Design like you are going to operate it.

2 AUTOMATION MINDSET

Automate the repeatable. Engineered systems don't rely on memory.

- ✓ Everything as Code (IaC, Policy, Pipeline).
- ✓ Automate security controls and compliance.
- ✓ Shift security left with automated checks.
- ✓ Use declarative tools. Avoid manual drift.
- ✓ Build self-healing and self-validating systems.
- ✓ Automate evidence collection and reporting.
- ✓ Eliminate toil. Focus on high-value work.

REMEMBER
Automation reduces risk, increases speed, and frees engineers to solve real problems.

SENIOR TIP
If it can be automated, write the code.
If it can be templated, template it.
If it can be standardized, standardize it.

3 RISK-BASED DECISIONS

Perfect security does not exist. Manage risk intieligently.

- ✓ Identify assets and what matters most.
- ✓ Understand threats and business impact.
- ✓ Evaluate likelihood vs impact.
- ✓ Prioritize fixes that reduce the highest risk.
- ✓ Document risk acceptance with reason.
- ✓ Reassess risk continuously.
- ✓ Security is about smart trade-offs.

REMEMBER
The goal is not zero risk. The goal is managed, measured, and accepted risk.

SENIOR TIP
Know what you are protecting, why it matters, and what you are willing to accept.
That is real maturity.

4 OPERATIONAL EXCELLENCE

Run reliable systems. Operational excellence is security.

- ✓ Monitor everything that matters.
- ✓ Have alerts with context, not noise.
- ✓ Test recovery, failover, and backups.
- ✓ Practice incident response regularly.
- ✓ Keep runbooks updated and simple.
- ✓ Measure SLIs, SLOs, and error budgets.
- ✓ Blameless post-incident reviews.

REMEMBER
You don't rise to the occasion. You fall to your level of preparation.

SENIOR TIP
If you can't operate it at 3AM, you don't really own it.
Make it observable, predictable and boring.

5 LONG-TERM MAINTAINABILITY

Build systems that last. Future you is your next customer.

- ✓ Keep architectures simple and modular.
- ✓ Write clear documentation.
- ✓ Adopt standards. Don't reinvent.
- ✓ Refactor when complexity grows.
- ✓ Version everything. Track changes.
- ✓ Design for scalability and evolve safely.
- ✓ Leave the codebase better than you found it.

REMEMBER
Code and systems are read more than they are written.
Clarity scales.

SENIOR TIP
Think about the next engineer who will read your code or operate your system.
Make their life easier.

THE SENIOR ENGINEER COMPASS

- Secure by Design
- Automate Everything
- Manage Risk
- Operate Excellent
- Build for the Long Term

DAILY SENIOR ENGINEER HABITS

- ☑ Read logs and metrics daily.
- ☑ Review security alerts.
- ☑ Automate one thing every day.
- ☑ Improve documentation.
- ☑ Validate assumptions.
- ☑ Learn something new.
- ☑ Mentor and share knowledge.

A NOTE TO YOUR FUTURE SELF

You won't remember every command, every tool, or every bug you fixed.
But you will be remembered for the systems you built, the incidents you handled, and the engineers you helped grow.
Keep building. Keep improving.
Leave a legacy.

GREAT ENGINEERS BUILD SYSTEMS. SENIOR ENGINEERS BUILD LEGACY.

VERIQTA @veriqta

VERIQTA

ENTERPRISE BEST PRACTICES

Best practices are habits that protect systems, people, and data. Build them into your engineering culture. Follow them. Automate them. Improve them continuously.

SECURE BY DEFAULT.
AUTOMATE EVERYTHING.
VALIDATE CONTINUOUSLY.
DOCUMENT CLEARLY.
COLLABORATE EFFECTIVELY.

PAGE 57

1 SECURE DEFAULTS

Start secure.
Stay secure.

KEY PRACTICES

- ✓ Use secure configurations by default
- ✓ Disable unnecessary services and ports
- ✓ Encrypt data in transit and at rest
- ✓ Apply least privilege access
- ✓ Harden systems and containers
- ✓ Use private networks and endpoints
- ✓ Enable logging and monitoring

EXAMPLES

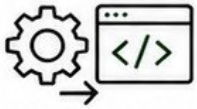
- Deny all inbound by default
- TLS 1.2+ everywhere
- IMDSv2 on AWS
- Read-only filesystems
- Non-root containers
- Secure headers enabled

REMEMBER

In secure environments, the default answer is NO. Allow only what is needed. Nothing more.

2 AUTOMATION FIRST

Humans make mistakes.
Automation doesn't forget.



KEY PRACTICES

- ✓ Automate builds, tests, and deployments
- ✓ Use Infrastructure as Code
- ✓ Automate security scans and checks
- ✓ Automate policy enforcement
- ✓ Automate backups and recovery
- ✓ Automate notifications and alerts
- ✓ Eliminate manual, repetitive tasks

EXAMPLES

- CI/CD pipelines
- Terraform / CloudFormation
- SAST, DAST, IaC scans
- Policy as Code (OPA, Conftest)
- Auto-scaling and auto-healing
- ChatOps / Alerting automation

REMEMBER

If a task can be automated, automate it. If it can be standardized, standardize it.

3 CONTINUOUS VALIDATION

Trust, but verify.
Continuously.

KEY PRACTICES

- ✓ Continuously test security and functionality
- ✓ Validate configurations and compliance
- ✓ Monitor systems and user behavior
- ✓ Detect, alert, and respond quickly
- ✓ Perform regular dependency updates
- ✓ Run chaos engineering exercises
- ✓ Verify backups and disaster recovery

EXAMPLES

- Continuous vulnerability scans
- Compliance as Code
- Real-time monitoring
- Threat detection (EDR, IDS)
- GameDays / Tabletop exercises
- DR tests and restore drills

REMEMBER

Systems change.
Threats change.
Validate continuously or risk increases.

4 DOCUMENTATION

If it's not documented, it doesn't exist.

KEY PRACTICES

- ✓ Document architecture and designs
- ✓ Document runbooks and procedures
- ✓ Keep documentation up to date
- ✓ Store docs in version control
- ✓ Use diagrams to simplify complexity
- ✓ Document decisions and changes
- ✓ Make docs discoverable and accessible

EXAMPLES

- Architecture diagrams (Diagrams as Code)
- Runbooks in Markdown
- ADRs (Architecture Decision Records)
- Change logs and release notes
- README for every repo
- Internal knowledge base

REMEMBER

Good documentation saves time, reduces risk, and eliminates tribal knowledge.

5 TEAM COLLABORATION

Security and reliability are a team sport.

KEY PRACTICES

- ✓ Share knowledge and learn together
- ✓ Define clear roles and responsibilities
- ✓ Communicate early and often
- ✓ Use standardized processes and tools
- ✓ Review code, configs, and designs
- ✓ Encourage feedback and improvement
- ✓ Celebrate wins and learn from failures

EXAMPLES

- Pair programming
- Code reviews & peer reviews
- Blameless post-incident reviews
- Cross-functional collaboration
- Shared dashboards and metrics
- Communities of practice

REMEMBER

Build a culture of trust, transparency, and ownership. Great teams build great systems.

ENTERPRISE BEST PRACTICES CHECKLIST

- ☑ Secure defaults applied
- ☑ Automation covers key workflows
- ☑ Continuous validation enabled
- ☑ Documentation is updated
- ☑ Team collaborates effectively
- ☑ Metrics and logs are monitored
- ☑ Risks identified and mitigated
- ☑ Processes are reviewed regularly

THE 5 PILLARS

Strong pillars. Strong systems. Strong impact.

KEY TAKEAWAY

Best practices are not optional.
They are force multipliers.
Apply them consistently.
Improve them continuously.
Build secure. Build reliable.
Build for the long term.

BEST PRACTICES TODAY. STRONG SYSTEMS TOMORROW. LASTING IMPACT ALWAYS.

VERIQTA @veriqta

VERIQTA

DEVSECOPS ARCHITECTURE CHEAT SHEET

Build secure. Ship fast. Operate safely.
Security is built in, not bolted on.

SECURE BY DESIGN.
AUTOMATE SECURITY.
CONTINUOUSLY VERIFY.
RESPOND FAST.
IMPROVE ALWAYS.

PAGE 58

1 SECURITY CONTROLS (DEFENSE IN DEPTH)

- ✓ Identity & Access Management (IAM)
- ✓ Least Privilege Access
- ✓ Multi-Factor Authentication (MFA)
- ✓ Network Segmentation
- ✓ Encryption in Transit (TLS)
- ✓ Encryption at Rest
- ✓ Secrets Management
- ✓ Vulnerability Management
- ✓ Container Image Scanning
- ✓ Runtime Protection
- ✓ Continuous Monitoring & Logging
- ✓ Threat Detection & Response
- ✓ Backup & Disaster Recovery
- ✓ Compliance & Governance

ZERO TRUST PRINCIPLES

- ✓ Verify explicitly
- ✓ Use least privilege
- ✓ Assume breach
- ✓ Continuous verification
- ✓ Monitor everything

REMEMBER

People + Process + Technology
=
Strong Security

2 DEVSECOPS PIPELINE FLOW

PLAN	CODE	BUILD	TEST	RELEASE	DEPLOY	OPERATE
• Threat Modeling • Security Requirements • Risk Assessment	• Secure Coding • SAST • IDE Security Plugins	• SCA • Dependency Scan • Secrets Scan • Code Quality	• DAST • IAST • Container Scan • Policy as Code	• Artifact Signing • Provenance (SBOM) • Approval Gate	• IaC Scan • Policy Check • K8s Admission Control	• Runtime Security • Monitoring • Logging • Incident Response

KEY PRINCIPLE

Automate security checks at every stage.
Do not wait for the end.

KUBERNETES SECURITY CHECKLIST

- ☑ RBAC enabled and least privilege
- ☑ Pod Security Standards enforced
- ☑ NetworkPolicies applied
- ☑ Secrets encrypted (KMS / SealedSecrets)
- ☑ Service Accounts scoped
- ☑ Image scanning in registry
- ☑ Admission Controllers enabled
- ☑ Audit logging enabled
- ☑ Resource limits and requests set
- ☑ Readiness & Liveness probes configured
- ☑ Namespaces isolated
- ☑ Kubelet authentication & authorization
- ☑ API server secured (TLS, AuthN, AuthZ)
- ☑ Regular backup of etcd
- ☑ Node hardening and patching

COMMON TOOLS

- ✓ Kyverno
- ✓ OPA Gatekeeper
- ✓ Falco
- ✓ Trivy
- ✓ Kube-bench
- ✓ kube-hunter
- ✓ Kubescape

4 CLOUD SECURITY CHECKLIST (AWS EXAMPLE)

- ☑ Use Multi-Account Strategy
- ☑ IAM least privilege and roles
- ☑ MFA for all human access
- ☑ S3 Block Public Access enabled
- ☑ VPC with private subnets
- ☑ Security Groups least privilege
- ☑ KMS encryption enabled
- ☑ CloudTrail enabled in all regions
- ☑ GuardDuty enabled
- ☑ Config rules enforced
- ☑ WAF for public endpoints
- ☑ Secrets in Secrets Manager
- ☑ Regular patch management
- ☑ Backup & cross-region replication
- ☑ Cost & resource monitoring enabled

KEY SERVICES

- IAM
- VPC
- S3
- KMS
- CloudTrail
- GuardDuty
- Config
- WAF
- Secrets Manager

5 INCIDENT RESPONSE CHECKLIST

DETECT
- ✓ Alert triggered
- ✓ Validate the alert
- ✓ Identify affected system
- ✓ Start incident log

CONTAIN
- ✓ Isolate affected resources
- ✓ Revoke compromised access
- ✓ Block malicious activity
- ✓ Preserve evidence

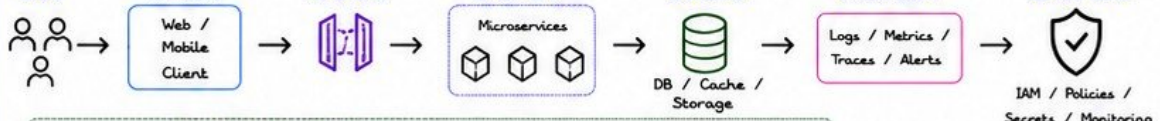
ERADICATE
- ✓ Remove root cause
- ✓ Patch vulnerabilities
- ✓ Rotate secrets/keys
- ✓ Clean affected systems

RECOVER
- ✓ Restore services
- ✓ Monitor closely
- ✓ Validate functionality
- ✓ Return to normal

QUICK COMMANDS

```
kubectl get pods -A                          # All pods
kubectl describe pod <pod>                   # Pod details
kubectl logs <pod> -f                        # Pod logs
kubectl get events --sort-by='.lastTimestamp' # Recent events
kubectl top pod -A                           # Resource usage

aws sts get-caller-identity                  # Who am I?
aws s3 ls s3://bucket                        # List S3 bucket
aws iam list-attached-role-policies --
role-name <role>                             # Role policies
aws ec2 describe-instances                   # EC2 instances
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=EventName,AttributeValue=ConsoleLogin # CloudTrail login

docker images                                # List images
trivy image <image>                          # Scan image
```

DEVSECOPS ARCHITECTURE OVERVIEW



CI/CD & AUTOMATION (GitHub Actions / GitLab CI / Jenkins / Argo CD)
Security Scans | Policy Checks | Artifact Signing | Automated Deployments

GOLDEN RULE

Secure every layer.
Automate every check.
Monitor everything.
Trust no one. Verify always.

SECURITY IS A TEAM SPORT. EVERY COMMIT. EVERY DEPLOYMENT. EVERY TIME.

VERIQTA @veriqta

VERIQTA

ENTERPRISE READINESS ASSESSMENT

Measure today. Improve intentionally.

Assess across people, process, platform, security, and operations.

ASSESS. SCORE. PRIORITIZE. IMPROVE. BUILD SECURE, RESILIENT, SCALABLE PLATFORMS.

1 SECURITY MATURITY

How strong is your security posture across the SDLC and cloud?

KEY AREAS

- ✓ Identity & Access Management
- ✓ Least Privilege & Zero Trust
- ✓ Vulnerability Management
- ✓ Secrets Management
- ✓ Secure SDLC / Shift Left
- ✓ Infrastructure Security
- ✓ Runtime Protection
- ✓ Security Monitoring & Detection
- ✓ Incident Response Preparedness
- ✓ Compliance & Governance

MATURITY LEVEL

1	2	3	4	5
INITIAL	BASIC	DEFINED	MANAGED	OPTIMIZED
Ad-hoc, reactive, limited visibility, high risk.	Some controls in place, inconsistent implementation.	Documented processes, regular reviews.	Measured, automated controls, low risk.	Continuous improvement, proactive, resilient.

YOUR SCORE __/5

NOTES / GAPS

2 PLATFORM MATURITY

How mature is your platform engineering and automation capability?

KEY AREAS

- ✓ Infrastructure as Code
- ✓ CI/CD Automation
- ✓ Self-Service Platform
- ✓ Observability & Monitoring
- ✓ Standardization & Reusability
- ✓ Workload Orchestration (K8s)
- ✓ Developer Experience
- ✓ Platform Reliability / SLOs
- ✓ Cost Management

MATURITY LEVEL

1	2	3	4	5
INITIAL	BASIC	DEFINED	MANAGED	OPTIMIZED
Manual, siloed, no automation, high toil.	Some automation, duplicate efforts.	Standard tools, repeatable processes.	Self-service, metrics driven, reliable.	Product-like platform, continuous innovation.

YOUR SCORE __/5

NOTES / GAPS

3 COMPLIANCE READINESS

How prepared are you for compliance and audit requirements?

KEY AREAS

- ✓ Policies & Standards
- ✓ Data Classification
- ✓ Access Reviews
- ✓ Audit Logging
- ✓ Evidence Collection
- ✓ Regulatory Mapping
- ✓ Continuous Compliance
- ✓ Third-Party Risk
- ✓ Document Management

MATURITY LEVEL

1	2	3	4	5
INITIAL	BASIC	DEFINED	MANAGED	OPTIMIZED
Unaware, no policies, high risk.	Basic policies, manual evidence.	Documented controls, periodic audits.	Automated evidence, continuous compliance.	Continuous monitoring, audit-ready.

YOUR SCORE __/5

NOTES / GAPS

4 OPERATIONAL READINESS

How ready are your operations to run, recover, and scale reliably?

KEY AREAS

- ✓ Monitoring & Alerting
- ✓ Incident Management
- ✓ SRE Practices / SLOs
- ✓ Capacity & Performance
- ✓ Backup & Disaster Recovery
- ✓ Change Management
- ✓ Runbooks & Documentation
- ✓ On-Call & Escalation
- ✓ Chaos Engineering / Resilience

MATURITY LEVEL

1	2	3	4	5
INITIAL	BASIC	DEFINED	MANAGED	OPTIMIZED
Firefighting, no process, frequent outages.	Some monitoring, manual response.	Documented processes, basic SLAs.	Proactive monitoring, reliable ops.	Resilient, self-healing, continuous improvement.

YOUR SCORE __/5

NOTES / GAPS

5 IMPROVEMENT ROADMAP

Prioritize gaps. Take action. Measure progress.

PRIORITY	AREA / GAP	CURRENT STATE	TARGET STATE	INITIATIVE / ACTION	OWNER	TIMELINE
HIGH						
MEDIUM						
LOW						

SCORING GUIDE

1 = Initial	(0 - 1.9)
2 = Basic	(2.0 - 2.9)
3 = Defined	(3.0 - 3.9)
4 = Managed	(4.0 - 4.4)
5 = Optimized	(4.5 - 5.0)

OVERALL MATURITY SUMMARY

Security	__/5
Platform	__/5
Compliance	__/5
Operations	__/5

OVERALL SCORE

__/20

☆☆☆☆☆

KEY TAKEAWAY

Readiness is not a destination.
It is a discipline.
Assess honestly.
Improve intentionally.
Build a secure, resilient, future-ready organization.

QUICK WINS (START TODAY)

- ☑ Enable MFA for all users
- ☑ Enforce least privilege
- ☑ Scan code & containers
- ☑ Centralize logging
- ☑ Automate backups
- ☑ Document runbooks

MEASURE HONESTLY. IMPROVE CONTINUOUSLY. DELIVER SECURELY.

VERIQTA @veriqta

VERIQTA

FINAL PRODUCTION REVIEW

The last mile is not the end.
It is the beginning of reliable, secure, and scalable systems.
Review everything. Improve continuously. Lead like a senior.

REVIEW. VALIDATE.
SECURE. AUTOMATE.
OBSERVE. IMPROVE.
DELIVER VALUE.

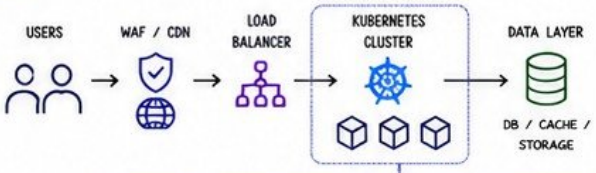
PAGE 60

1 COMPLETE DEVSECOPS WORKFLOW

PLAN	CODE	BUILD	SECURE	TEST	RELEASE	DEPLOY	OPERATE
• Threat Modeling	• Secure Coding	• Dependency Scan	• Policy as Code	• DAST	• Approval	• GitOps / CD	• Monitoring
• Requirements	• SAST	• Container Build	• IaC Scan	• IAST	• Provenance (SLSA)	• Policy Check	• Logging
• Risk Assessment	• Secrets Scan	• Image Scan	• Compliance	• Unit Tests	• Signed Artifacts	• Progressive Delivery	• Alerts
• Security Goals	• Pre-commit	• SBOM	• License Check	• Integration Tests	• Change Log	• Canary	• Response
	• Code Review	• Artifact Sign	• Vulnerability Gate	• Security Tests	• Release Notes	• Rollback Plan	• Improve

Security is not a phase. It is a continuous workflow.

2 END-TO-END ARCHITECTURE REVIEW



NETWORK SECURITY	WORKLOAD SECURITY	PLATFORM SERVICES	IDENTITY & ACCESS
• VPC	• RBAC	• CI/CD	• IAM
• Subnets	• NetworkPolicies	• Registry	• OIDC
• Security Groups	• Pod Security	• Monitoring	• Least Privilage
• NACLs	• Secrets	• Logging	• MFA
• Private Link	• Images	• GitOps	• Audit Logs

Review every layer. Understand every trust boundary.

3 PRODUCTION SECURITY CHECKLIST

- [x] All images scanned and signed (SBOM generated)
- [x] Secrets stored in Vault / KMS encrypted
- [x] Least privilege IAM / RBAC enforced
- [x] NetworkPolicies restrict traffic
- [x] WAF rules enabled and tested
- [x] TLS everywhere (1.2+)
- [x] Audit logging enabled (CloudTrail / K8s Audit)
- [x] Monitoring and alerting configured
- [x] Backups encrypted and tested
- [x] Disaster recovery plan validated
- [x] Vulnerability management process active
- [x] Compliance controls mapped and verified
- [x] Dependencies up to date and scanned
- [x] Access reviews performed
- [x] Incident response plan tested

REMEMBER

Good security is invisible when everything works and priceless when something breaks.

4 COMMON FAILURE PATTERNS

PATTERN	WHY IT HAPPENS	IMPACT	HOW TO PREVENT
Over-Permissive Access	Default roles, no reviews	Breach, data leak	Least privilage, regular access reviews
Unscanned Dependencies	No automation, no SBOM	Vulnerabilities, exploits	Automated scans, update regularly
Exposed Services	Misconfigured security groups	Unauthorized access	Restrict inbound, use NACLs / WAF
Secrets in Code	Developer convenience	Credential leak	Use Vault / KMS, scan & block
No Observability	Monitoring not prioritized	Slow detection, outages	Centralize logs, alerts, dashboards
No Rollback Strategy	Fast deployments without plan	Long outages, customer impact	Canary, blue/green, tested rollback

Most incidents are preventable with discipline and automation.

5 SENIOR ENGINEER PRINCIPLES

- **SECURITY IS EVERYONE'S JOB** — Embed security in culture and process.
- **AUTOMATE EVERYTHING** — If it can be automated, it should be.
- **OBSERVE CONTINUOUSLY** — You can't fix what you can't see.
- **RISK-BASED DECISIONS** — Focus on what matters most.
- **ITERATE AND IMPROVE** — Perfect is the enemy of good.
- **DOCUMENT AND SHARE** — Knowledge compounds when shared.
- **OWN THE SYSTEM END-TO-END** — From code to customer, take responsibility.
- **BUILD FOR RESILIENCE** — Design for failure, recover gracefully.

REMEMBER

Senior engineers don't just solve problems. They prevent them, scale solutions, and make the system better for everyone.

6 FINAL PRODUCTION VERIFICATION

- [x] All pipelines green and stable
- [x] All security scans passing
- [x] All policies enforced
- [x] Monitoring and alerts active
- [x] Dashboards reviewed
- [x] Runbooks tested
- [x] Backups and DR validated
- [x] Access and permissions verified
- [x] Cost and performance optimized
- [x] Stakeholders sign-off obtained

YOU'RE READY!

You have built a secure, automated, observable, and resilient platform.
Keep learning,
Keep improving.
Lead the change.

Shipping secure software is the start.
Operating it well is the real engineering.

DEVSECOPS MANTRA

D	Design secure
E	Everything automated
V	Validate continuously
S	Secure by default
E	Every change matters
C	Comply and audit
O	Observe everything
P	Plan for failure
S	Scale with confidence
S	Sustain and improve

FINAL CHECK

Secure.
Automated.
Observable.
Resilient.
Reliable.
Production Ready.
VERIQTA Approved.

CONTINUOUS IMPROVEMENT LOOP

Never stop improving. Never stop learning. Never ship insecure.

ENGINEER SECURELY. AUTOMATE INTELLIGENTLY. OPERATE EXCELLENTLY.

VERIQTA @veriqta